

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل چهاردهم: Vector Space Classification

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

14.1 Document representations and measures of relatedness in vector spaces

14.2 Rocchio classification

14.3 k nearest neighbor

14.3.1 Time complexity and optimality of kNN

14.4 Linear versus nonlinear classifiers

14.5 Classification with more than two classes

14.6 The bias-variance tradeoff

14.7 Chapter 14 Summary

14.8 End of Chapter Exercises

14 طبقه‌بندی فضای برداری: چگونه با بردارها، متون را درک کنیم؟

گوگل ترنسلیت و چالش طبقه‌بندی متون: چگونه زبان‌ها را از هم تشخیص دهیم؟

تصور کنید در تیم گوگل ترنسلیت کار می‌کنید و باید سیستمی طراحی کنید که متون‌ها را به زبان‌های مختلفی (انگلیسی، فرانسوی، آلمانی، اسپانیایی و...) طبقه‌بندی کند. چگونه می‌توانید با استفاده از بردارهای عددی، زبان متن را بدون نیاز به درک معنای آن تشخیص دهید؟

این دقیقاً همان چالشی است که مدل فضای برداری با آن روبرو است: چگونه می‌توانیم با استفاده از ویژگی‌های عددی (مانند وزن tf-idf) به جای وجود یا عدم وجود واژه‌ها، متون‌ها را از هم تشخیص دهیم؟

در فصل 13، با مدل بیز ساده کار کردیم که در آن هر سند به صورت دنباله‌ای از واژه‌ها یا بردار دوتایی نمایش داده می‌شد. در این فصل، از نمایش متفاوتی استفاده می‌کنیم: **مدل فضای برداری** که در فصل 6 توسعه داده شد. در این مدل، هر سند به صورت یک بردار با یک مولفه حقیقی برای هر واژه (معمولاً وزن tf-idf) نمایش داده می‌شود. بنابراین، فضای سند X ، دامنه تابع طبقه‌بندی γ ، به صورت $\mathbb{R}^V$ است.

🗨️ فرضیه تداوم (Contiguity hypothesis): مرزهای نامرئی بین زبان‌ها

فرضیه اصلی در استفاده از مدل فضای برداری، **فرضیه تداوم** است: اسناد یک کلاس ناحیه‌ای پیوسته تشکیل می‌دهند و مناطق کلاس‌های مختلف با هم همپوشانی ندارند.

در بسیاری از وظایف طبقه‌بندی متن، به خصوص نوعی که در فصل 13 با آن مواجه شدیم، کلاس‌ها را می‌توان با الگوهای واژگانی متمایز کرد. برای مثال، اسناد کلاس China معمولاً مقادیر بالایی در ابعاد مربوط به Chinese، Beijing و Mao دارند، در حالی که اسناد کلاس UK مقادیر بالایی برای London، British و Queen دارند. بنابراین، اسناد دو کلاس مناطق مجزایی را تشکیل می‌دهند که در شکل 14.1 نشان داده شده است و می‌توانیم مرزهایی را برای جداسازی آن‌ها ترسیم کرد.

مثال واقعی: در سیستم طبقه‌بندی مقالات علمی آرکایو (ArXiv)، مقالات مربوط به هوش مصنوعی معمولاً واژه‌هایی مانند "deep learning"، "neural network" و "algorithm" دارند، در حالی که مقالات مربوط به فیزیک نظری واژه‌هایی مانند "particle"، "quantum" و "field" دارند. این دو مجموعه در فضای برداری مناطق مجزایی را تشکیل می‌دهند که با مرزهای خطی قابل تفکیک هستند.

📌 اهمیت نمایش سند: چرا کلمات توقف مهم هستند؟

اینکه آیا یک مجموعه از اسناد به مناطق مجزایی نگاشته می‌شود یا نه، به انتخاب‌هایی که برای نمایش سند انجام می‌دهیم بستگی دارد. برای مثال، وقوع مکرر ضمیر اول شخص "I" شواهدی برای کلاس "نوشته توسط یک نفر" است. اما این اطلاعات ممکن است از نمایش سند حذف شود اگر از لیست کلمات توقف استفاده کنیم.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان آمازون، کلماتی مانند "the"، "is"، "and" و "a" تقریباً در همه نظرات وجود دارند اما اطلاعات کمی در مورد دسته محصول ارائه می‌دهند. حذف این کلمات به سیستم کمک می‌کند تا روی کلمات کلیدی مانند "poor"، "excellent" یا "packaging" تمرکز شود که اطلاعات بیشتری در مورد نظر مشتری ارائه می‌دهند.

🌟 روش‌های طبقه‌بندی فضای برداری

این فصل دو روش طبقه‌بندی فضای برداری را معرفی می‌کند:

- **طبقه‌بندی Rocchio** (بخش 14.2): فضای برداری را به مناطقی تقسیم می‌کند که در مرکز مراکز یا نمونه‌های اولیه قرار دارند، یک برای هر کلاس که به صورت مرکز جرم تمام اسناد آن کلاس محاسبه می‌شود. طبقه‌بندی Rocchio ساده و کارآمد است، اما اگر کلاس‌ها تقریباً کروی با شعاع‌های مشابه نباشند، دقیق نیست.

- **طبقه‌بندی kNN** (بخش 14.3): به سند آزمون، کلاس اکثریت از k نزدیک‌ترین همسایه‌ها را اختصاص می‌دهد. kNN نیازی به آموزش صریح ندارد و می‌تواند مستقیماً از مجموعه آموزشی استفاده کند. اگر مجموعه آموزشی بزرگ باشد، kNN می‌تواند کلاس‌های غیرکروی و پیچیده‌تر را بهتر از Rocchio مدیریت کند.

مثال واقعی: در سیستم طبقه‌بندی ایمیل‌های گوگل، Rocchio برای فیلتر اسپم‌های واضح (مانند ایمیل‌های تبلیغاتی که الگوی مشخصی دارند) استفاده می‌شود، در حالی که kNN برای فیلتر اسپم‌های پیچیده‌تر که الگوهای پنهان دارند یا از تکنیک‌های جدیدی استفاده می‌کنند، بهتر عمل می‌کند.

طبقه‌بندهای خطی: مرزهای مستقیم در فضای برداری

تعداد زیادی از طبقه‌بندهای متن را می‌توان به عنوان **طبقه‌بندهای خطی** در نظر گرفت - طبقه‌بندهایی که بر اساس ترکیب خطی ساده‌ای از ویژگی‌ها طبقه‌بندی می‌کنند. چنین طبقه‌بندهایی فضای ویژگی‌ها را به مناطقی جدا شده توسط هایپرپلن‌های تصمیم خطی تقسیم می‌کنند.

وقتی از طبقه‌بندهای دو کلاسی برای مسائل با بیش از دو کلاس استفاده می‌کنیم، دو نوع وظیفه وجود دارد:

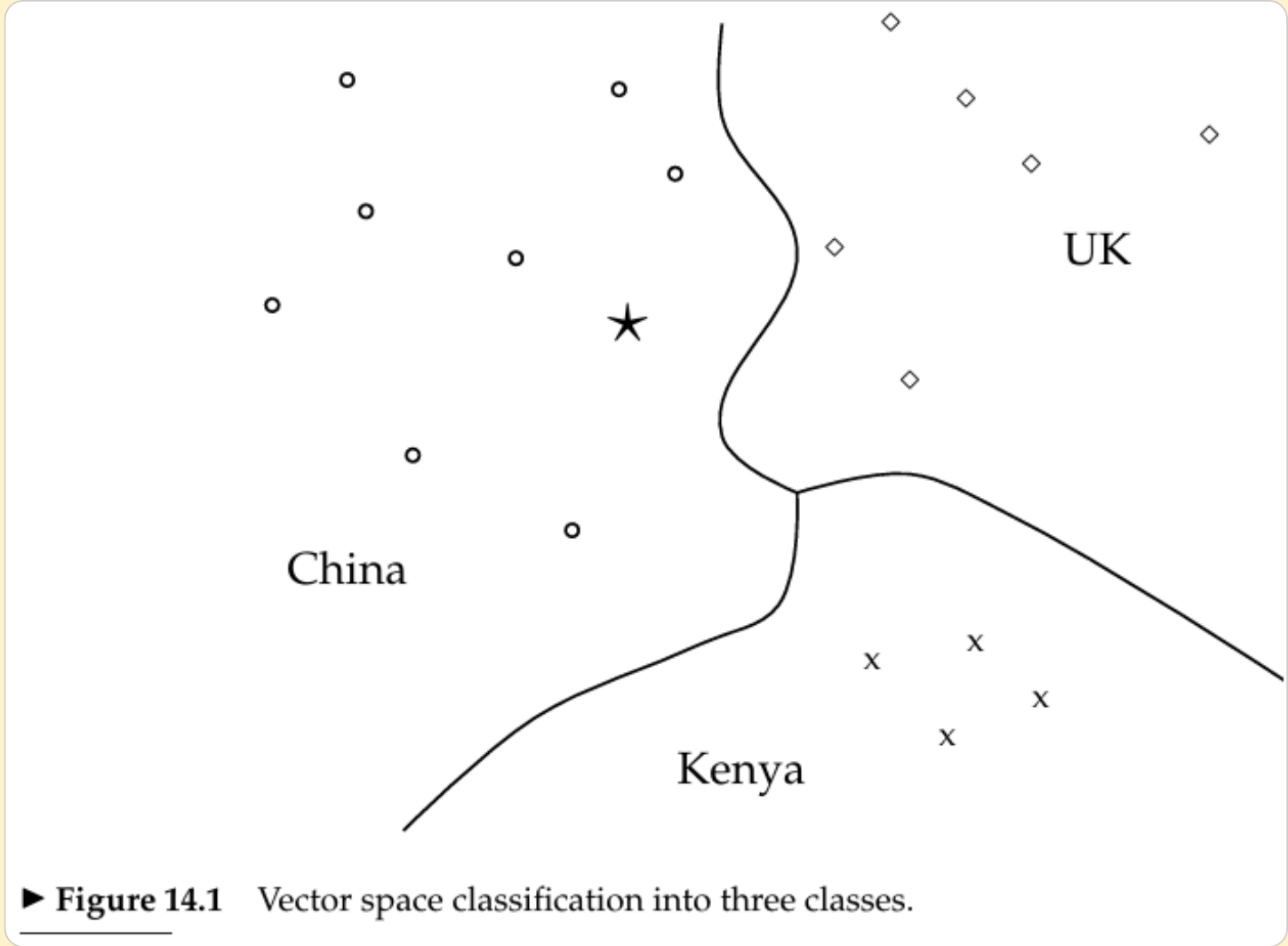
- **وظایف یک-از-چند (one-of):** سند باید دقیقاً به یکی از چندین کلاس انحصاری متقابل اختصاص داده شود.
- **وظایف هر-از-چند (any-of):** سند می‌تواند به هر تعدادی از کلاس‌ها اختصاص داده شود (در بخش 14.5 توضیح داده خواهد شد).

طبقه‌بندهای دو کلاسی وظایف هر-از-چند را حل می‌کنند و می‌توانند برای حل وظایف یک-از-چند ترکیب شوند.

14.1 نمایش اسناد و معیارهای وابستگی در فضاهای برداری

بردارها به صورت نقاطی: از ابعاد به مرزها

برای نمایش ویژگی‌های بردار اسناد در طبقه‌بندی فضای برداری، این بردارها را به صورت نقاطی در یک صفحه نمایش می‌دهیم مانند مثال در شکل 14.1. در واقعیت، بردارهای سند بردارهای یکانی نرمال‌شده هستند که به سطح یک ابرکره (hypersphere) اشاره می‌کنند.



► **Figure 14.1** Vector space classification into three classes.

شکل 14.2 تصویر ابعاد کوچک از سطح یک ابرکره را به یک صفحه نشان می‌دهد. سمت چپ: تصویر دایره‌ای دو بعدی به یک بعدی. برای نقاط x_1, x_2, x_3, x_4, x_5 در مختصات x ، فاصله $|x_2x_3|$ تقریباً 0.201 است و تنها 0.5% با $|x|$ تفاوت دارد. سمت راست: تصویر مربوط به نیمکره سه بعدی به دو بعدی.

فاصله‌ها و شباهت‌ها: چگونه "نزدیکی" را اندازه‌گیری کنیم؟

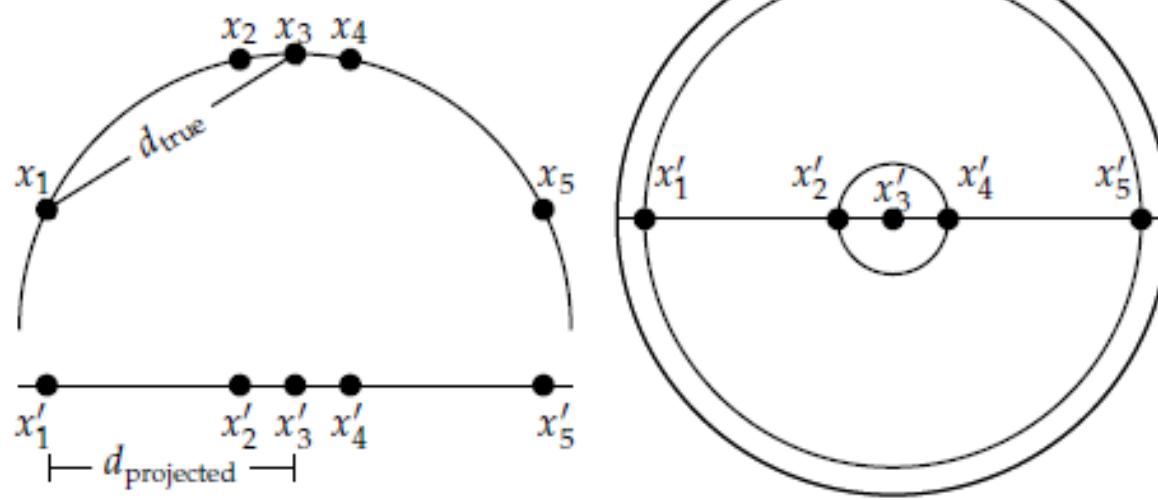
تصمیمات بسیاری از طبقه‌بندی‌های فضای برداری بر اساس مفهوم **فاصله** است، مثلاً هنگام محاسبه نزدیک‌ترین همسایه‌ها در طبقه‌بندی kNN. در این فصل، از فاصله اقلیدسی (Euclidean distance) به عنوان معیار فاصله اصلی استفاده می‌کنیم. همانطور که قبلاً مشاهده شد (تمرین 6.18، صفحه 131 کتاب)، ارتباط مستقیمی بین شباهت کسینوسی (cosine similarity) و فاصله اقلیدسی برای بردارهای نرمال‌شده وجود دارد. در طبقه‌بندی فضای برداری، به ندرست اهمیت ندارد که وابستگی دو سند به صورت شباهت یا فاصله بیان شود.

مراکزها و میانگین‌ها: نقاط مرجع در فضای برداری

علاوه بر اسناد، **مراکزها** یا میانگین بردارها نیز نقش مهمی در طبقه‌بندی فضای برداری ایفا می‌کنند. مراکزها نرمال‌شده نیستند. برای بردارهای نرمال‌شده، حاصل ضرب داخلی، شباهت کسینوسی و فاصله اقلیدسی رفتار متفاوتی در حالت کلی دارند.

مثال واقعی: در سیستم طبقه‌بندی مقالات علمی آرکایو (ArXiv)، مراکز هر حوزه (مانند یادگیری ماشین، فیزیک نظری، زیست‌شناسی) به صورت میانگین بردارهای تمام مقالات آن حوزه محاسبه می‌شود. وقتی مقاله جدیدی به سیستم داده می‌شود، با محاسبه فاصله آن از مراکزهای مختلف، می‌توان تشخیص داد که به کدام حوزه نزدیک‌تر است.

شکل 14.2: تصویر ابعاد کوچک از سطح ابرکره



► **Figure 14.2** Projections of small areas of the unit sphere preserve distances. Left: A projection of the 2D semicircle to 1D. For the points x_1, x_2, x_3, x_4, x_5 at x coordinates $-0.9, -0.2, 0, 0.2, 0.9$ the distance $|x_2 x_3| \approx 0.201$ only differs by 0.5% from $|x'_2 x'_3| = 0.2$; but $|x_1 x_3| / |x'_1 x'_3| = d_{\text{true}} / d_{\text{projected}} \approx 1.06 / 0.9 \approx 1.18$ is an example of a large distortion (18%) when projecting a large area. Right: The corresponding projection of the 3D hemisphere to 2D.

شکل 14.2 تصویر ابعاد کوچک از سطح یک ابرکره را به یک صفحه نشان می‌دهد. سمت چپ: تصویر دایره‌ای دو بعدی به یک بعدی. برای نقاط x_1, x_2, x_3, x_4, x_5 در مختصات x ، فاصله $|x_2 x_3|$ تقریباً 0.201 است و تنها 0.5% با $|x|$ تفاوت دارد. سمت راست: تصویر مربوط به نیمکره سه بعدی به دو بعدی.

این تصویر نشان می‌دهد که چگونه تصویر کردن به یک صفحه می‌تواند باعث تغییر شکل شود. برای مثال، تصویر دایره‌ای دو بعدی به یک بعدی به دلیل انحنای آن، کشیده شده و به صورت بیضی در می‌آید. این همان پدیده‌ای است که در سیستم‌های طبقه‌بندی متون رخ می‌دهد: وقتی زبان‌های مختلف را به فضای برداری کاهش می‌دهیم، ممکن است اطلاعات مهمی از دست بدهیم.

تمرین 14.1: انحنای در تصویر کردن

برای مناطق کوچک، فاصله‌ها روی سطح ابرکره به خوبی توسط فاصله‌ها روی تصویر آن‌ها تقریب می‌شوند زیرا برای زوایای کوچک، $\alpha \approx \sin \alpha$. برای چه اندازه زاویه، تغییر شکل $\alpha / \sin(\alpha)$ است:

برای زوایای کوچک:

- (i) $\alpha / \sin(\alpha) \approx 1.01$ (%تغییر شکل حدود 1)
- (ii) $\alpha / \sin(\alpha) \approx 1.05$ (%تغییر شکل حدود 5)
- (iii) $\alpha / \sin(\alpha) \approx 1.1$ (%تغییر شکل حدود 10)

این تمرین نشان می‌دهد که حتی برای مناطق کوچک، تصویر کردن به یک صفحه کمتر باعث تغییر شکل قابل توجهی می‌شود. برای مثال، تصویر نیمکره سه بعدی به دو بعدی می‌تواند باعث تغییر شکل 18% شود (مثال سمت راست).

تفسیر عملی: در سیستم طبقه‌بندی متون گوگل، وقتی زبان‌های مختلف را به فضای برداری با ابعاد کمتر کاهش می‌دهیم (برای افزایش سرعت)، ممکن است برخی ویژگی‌های ظریف که زبان‌ها را از هم متمایز می‌کنند از دست بدهیم. این می‌تواند باعث کاهش دقت طبقه‌بندی شود، به خصوص برای زبان‌هایی با ویژگی‌های ظریف که ساختار پیچیده‌تری دارند.

```
def euclidean_distance(v1, v2):
    """
    محاسبه فاصله اقلیدسی بین دو بردار

    این تابع فاصله اقلیدسی استاندارد بین دو بردار را محاسبه می‌کند.

    پارامترها:
    v1, v2: دو بردار که می‌خواهیم فاصله بین آن‌ها را محاسبه کنیم

    خروجی: مقدار فاصله اقلیدسی
    """
    return np.sqrt(np.sum((v1 - v2) ** 2))

def cosine_similarity(v1, v2):
    """
    محاسبه شباهت کسینوسی بین دو بردار

    این تابع شباهت کسینوسی را بین دو بردار محاسبه می‌کند.

    پارامترها:
    v1, v2: دو بردار که می‌خواهیم شباهت بین آن‌ها را محاسبه کنیم

    خروجی: مقدار شباهت کسینوسی (بین 0 و 1)
    """
    dot_product = np.dot(v1, v2)
    norm_v1 = np.linalg.norm(v1)
    norm_v2 = np.linalg.norm(v2)

    if norm_v1 == 0 or norm_v2 == 0:
        return 0

    return dot_product / (norm_v1 * norm_v2)

def visualize_vectors_2d(vectors, labels, centroids=None):
    """
    نمایش بردارهای دو بعدی در یک صفحه

    این تابع بردارهای دو بعدی را به صورت نقاطی در یک صفحه نمایش می‌دهد.
    همچنین می‌تواند مراکزها را در صورت وجود نمایش دهد.

    پارامترها:
    vectors: لیست بردارهای دو بعدی
    labels: لیست برچسب‌های مربوط به هر بردار
    centroids: لیست مراکزها (اختیاری) برای نمایش

    خروجی: نمایش بردارها و مراکزها در یک صفحه
    """
    plt.figure(figsize=(10, 8))

    # رسم بردارها
    colors = ['red', 'blue', 'green', 'purple', 'orange']
    for i, (vec, label) in enumerate(zip(vectors, labels)):
        plt.quiver(0, 0, vec[0], vec[1], angles='xy', scale_units='xy', scale=1,
                  color=colors[i % len(colors)], alpha=0.7, width=0.005)
        plt.scatter(vec[0], vec[1], s=100, c=colors[i % len(colors)], label=label)

    # رسم مراکزها در صورت وجود
    if centroids is not None:
        for i, (centroid, label) in enumerate(zip(centroids, ['Centroid 1', 'Centroid 2'])):
            plt.scatter(centroid[0], centroid[1], s=200, c='black', marker='x', label=label)

    plt.axhline(0, color='gray', linestyle='--', alpha=0.3)
    plt.axvline(0, color='gray', linestyle='--', alpha=0.3)
    plt.grid(True, alpha=0.3)
    plt.xlabel('first dimension')
    plt.ylabel('second dimension')
    plt.title('2D vector demonstration')
    plt.legend()
    plt.axis('equal')
    plt.show()

def angle_distortion(angle_deg):
    """
    برای یک زاویه  $\alpha/\sin(\alpha)$  محاسبه نسبت تغییر شکل

    را برای یک زاویه محاسبه می‌کند  $\alpha/\sin(\alpha)$  این تابع نسبت تغییر شکل.
    این معیار نشان می‌دهد که چقدر تصویر یک زاویه کوچک در یک صفحه تغییر شکل می‌دهد.

    پارامترها:
    angle_deg: زاویه بر حسب درجه

    خروجی: مقدار  $\alpha/\sin(\alpha)$ 
    """
    angle_rad = np.radians(angle_deg)
    return angle_rad / np.sin(angle_rad)

# تمرین 14.1: محاسبه تغییر شکل برای زوایای کوچک
print("تمرین 14.1: محاسبه تغییر شکل برای زوایای کوچک")
print("="*50)
angles = [1, 5, 10]
for angle in angles:
    distortion = angle_distortion(angle)
    print(f"زاویه {angle} درجه: نسبت تغییر شکل  $\alpha/\sin(\alpha)$  = {distortion:.4f}")

# ایجاد چند بردار نمونه برای نمایش
vectors = np.array([
    [2, 3], # کلاس 1
    [4, 1], # کلاس 2
    [1, 4], # کلاس 3
    [3, 2] # کلاس 4
])
labels = ["class1", "class 2", "class 3", "class 4"]
```

```
# محاسبه و نمایش بردارها
print("\nنمایش بردارهای دو بعدی:")
visualize_vectors_2d(vectors, labels)

# محاسبه شباهت کسینوسی و فاصله اقلیدسی بین بردارها
print("\nمعیارهای وابستگی بین بردارها:")
for i in range(len(vectors)):
    for j in range(i+1, len(vectors)):
        cos_sim = cosine_similarity(vectors[i], vectors[j])
        euc_dist = euclidean_distance(vectors[i], vectors[j])
        print(f"شباهت کسینوسی بین بردار {i+1} و {j+1}: {cos_sim:.4f}")
        print(f"فاصله اقلیدسی بین بردار {i+1} و {j+1}: {euc_dist:.4f}")

# نرمال‌سازی بردارها و مقایسه مجدد
normalized_vectors = normalize(vectors, norm='l2')
print("\nپس از نرمال‌سازی:")
for i in range(len(normalized_vectors)):
    for j in range(i+1, len(normalized_vectors)):
        cos_sim = cosine_similarity(normalized_vectors[i], normalized_vectors[j])
        euc_dist = euclidean_distance(normalized_vectors[i], normalized_vectors[j])
        print(f"شباهت کسینوسی بین بردار نرمال {i+1} و {j+1}: {cos_sim:.4f}")
        print(f"فاصله اقلیدسی بین بردار نرمال {i+1} و {j+1}: {euc_dist:.4f}")

# نمایش تفاوت بین بردارهای نرمال‌شده و نرمال‌نشده
print("\nمقایسه بردارهای نرمال‌شده و نرمال‌نشده:")
plt.figure(figsize=(12, 5))
orig_vectors = np.array([[2, 3], [4, 1]]) # بردارهای اصلی
norm_vectors = normalize(orig_vectors, norm='l2') # بردارهای نرمال‌شده

# نمایش بردارهای اصلی
plt.subplot(1, 2, 1)
plt.quiver(0, 0, orig_vectors[0, 0], orig_vectors[0, 1], angles='xy', scale_units='xy', scale=1,
           color='red', alpha=0.7, width=0.005)
plt.quiver(0, 0, orig_vectors[1, 0], orig_vectors[1, 1], angles='xy', scale_units='xy', scale=1,
           color='blue', alpha=0.7, width=0.005)
plt.scatter(orig_vectors[:, 0], orig_vectors[:, 1], s=100, c=['red', 'blue'])
plt.title('Original vectors')
plt.axis('equal')
plt.grid(True, alpha=0.3)

# نمایش بردارهای نرمال‌شده
plt.subplot(1, 2, 2)
plt.quiver(0, 0, norm_vectors[0, 0], norm_vectors[0, 1], angles='xy', scale_units='xy', scale=1,
           color='red', alpha=0.7, width=0.005)
plt.quiver(0, 0, norm_vectors[1, 0], norm_vectors[1, 1], angles='xy', scale_units='xy', scale=1,
           color='blue', alpha=0.7, width=0.005)
plt.scatter(norm_vectors[:, 0], norm_vectors[:, 1], s=100, c=['red', 'blue'])
plt.title('Normalized vectors')
plt.axis('equal')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

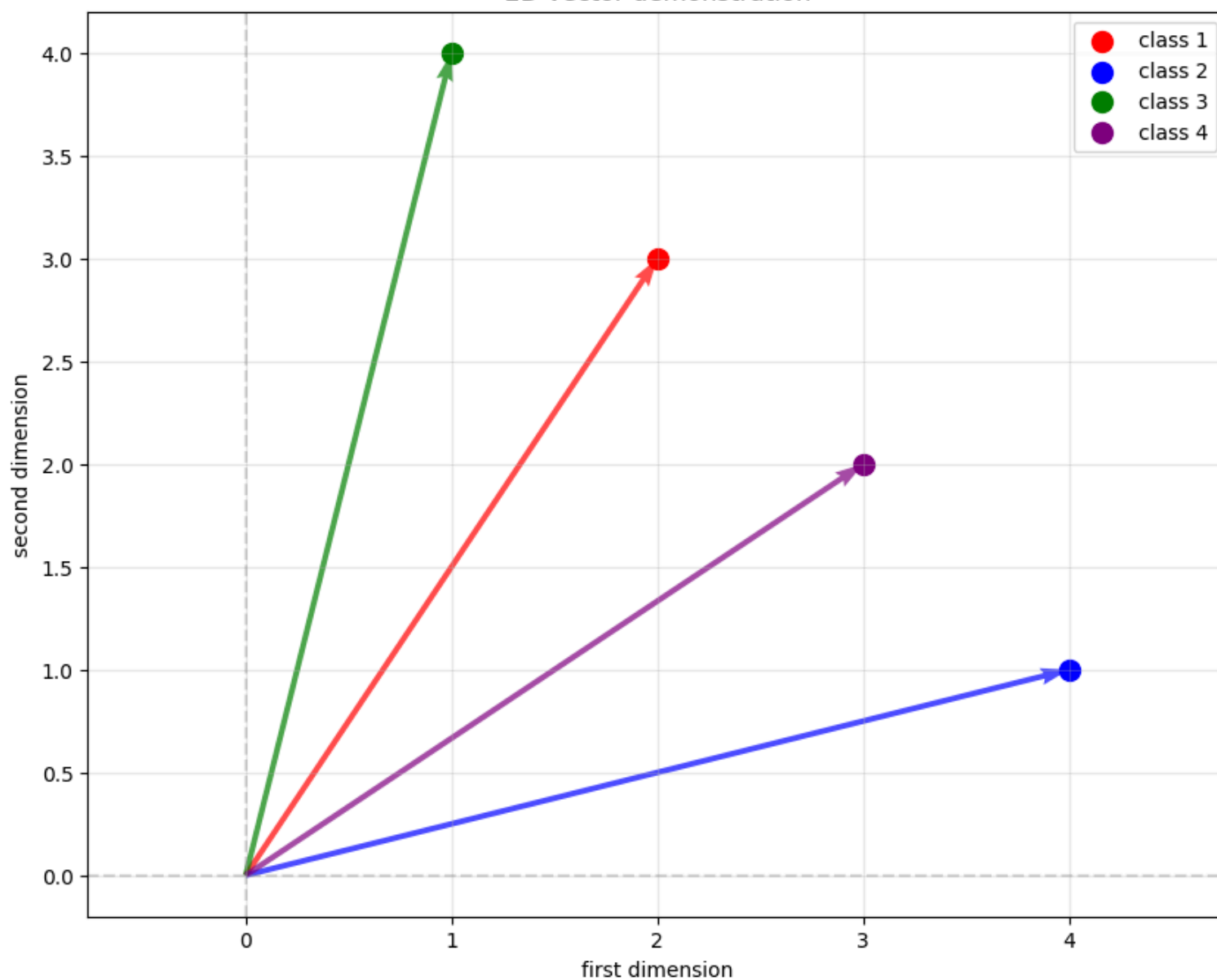
# نمایش تفاوت در معیارهای وابستگی
print("\nتفسیر عملی:")
print("\n*50)
print("1. برای زوایای کوچک، تصویر کردن به یک صفحه کمتر باعث تغییر شکل قابل توجهی می‌شود.
print("2. در سیستم طبقه‌بندی متون، وقتی زبان‌ها را به فضای برداری کاهش می‌دهیم، ممکن است برخی ویژگی‌های ظریف که زبان‌ها را از هم متمایز می‌کنند از دست بدهیم.
print("3. این می‌تواند باعث کاهش دقت طبقه‌بندی شود، به خصوص برای زبان‌هایی با ویژگی‌های ظریف که ساختار پیچیده‌تری دارند.
print("4. با این حال، سیستم‌هایی مانند گوگل ترنسلیت ممکن است بین سرعت و دقت تعادل برقرار کنند.
```

تمرین 14.1: محاسبه تغییر شکل برای زوایای کوچک
=====

$\alpha/\sin(\alpha)$	=	1.0001
$\alpha/\sin(\alpha)$	=	1.0013
$\alpha/\sin(\alpha)$	=	1.0051

نمایش بردارهای دو بعدی:

2D vector demonstration



معیارهای وابستگی بین بردارها

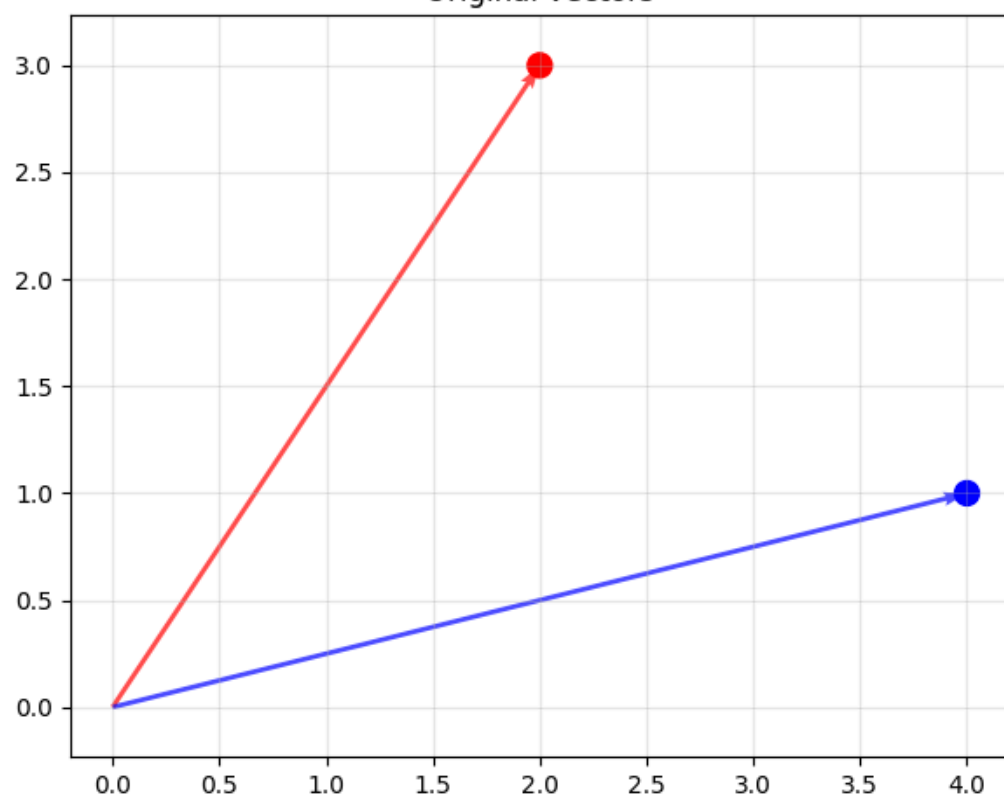
شباهت کسینوسی بین بردار 1 و 2: 0.7399
 فاصله اقلیدسی بین بردار 1 و 2: 2.8284
 شباهت کسینوسی بین بردار 1 و 3: 0.9417
 فاصله اقلیدسی بین بردار 1 و 3: 1.4142
 شباهت کسینوسی بین بردار 1 و 4: 0.9231
 فاصله اقلیدسی بین بردار 1 و 4: 1.4142
 شباهت کسینوسی بین بردار 2 و 3: 0.4706
 فاصله اقلیدسی بین بردار 2 و 3: 4.2426
 شباهت کسینوسی بین بردار 2 و 4: 0.9417
 فاصله اقلیدسی بین بردار 2 و 4: 1.4142
 شباهت کسینوسی بین بردار 3 و 4: 0.7399
 فاصله اقلیدسی بین بردار 3 و 4: 2.8284

پس از نرمال‌سازی

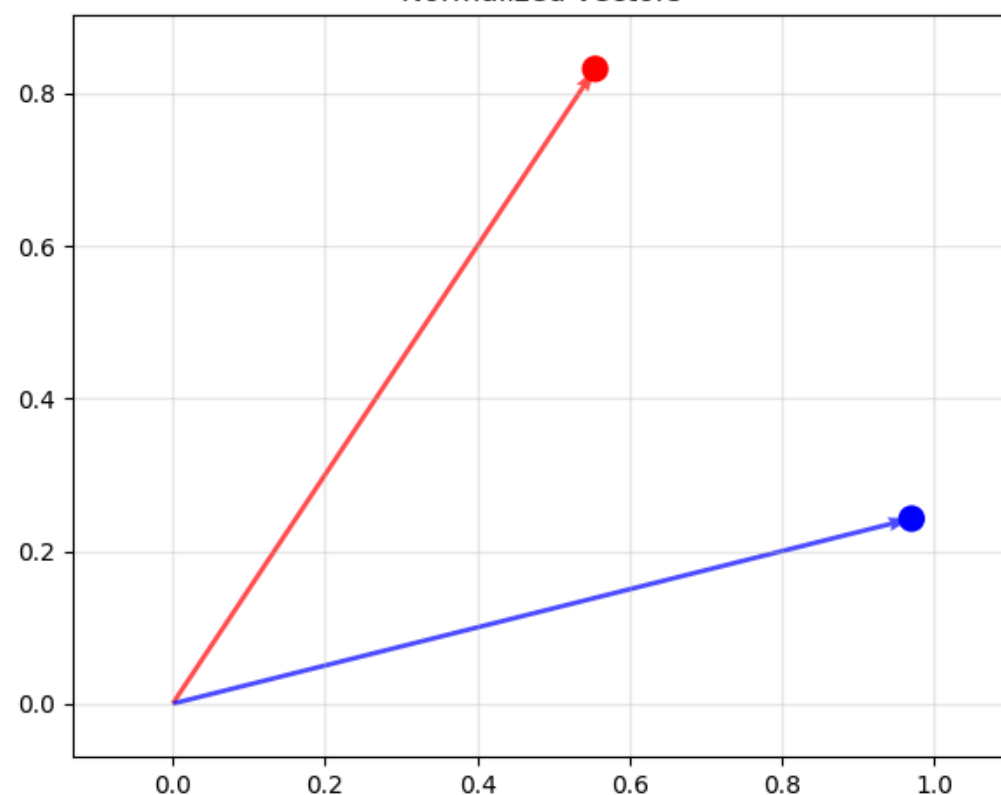
شباهت کسینوسی بین بردار نرمال 1 و 2: 0.7399
 فاصله اقلیدسی بین بردار نرمال 1 و 2: 0.7212
 شباهت کسینوسی بین بردار نرمال 1 و 3: 0.9417
 فاصله اقلیدسی بین بردار نرمال 1 و 3: 0.3413
 شباهت کسینوسی بین بردار نرمال 1 و 4: 0.9231
 فاصله اقلیدسی بین بردار نرمال 1 و 4: 0.3922
 شباهت کسینوسی بین بردار نرمال 2 و 3: 0.4706
 فاصله اقلیدسی بین بردار نرمال 2 و 3: 1.0290
 شباهت کسینوسی بین بردار نرمال 2 و 4: 0.9417
 فاصله اقلیدسی بین بردار نرمال 2 و 4: 0.3413
 شباهت کسینوسی بین بردار نرمال 3 و 4: 0.7399
 فاصله اقلیدسی بین بردار نرمال 3 و 4: 0.7212

مقایسه بردارهای نرمال‌شده و نرمال‌نشده

Original vectors



Normalized vectors



- =====
1. برای زوایای کوچک، تصویر کردن به یک صفحه کمتر باعث تغییر شکل قابل توجهی می‌شود.
 2. در سیستم طبقه‌بندی متون، وقتی زبان‌ها را به فضای برداری کاهش می‌دهیم، ممکن است برخی ویژگی‌های ظریف که زبان‌ها را از هم متمایز می‌کنند از دست بدهیم.
 3. این می‌تواند باعث کاهش دقت طبقه‌بندی شود، به خصوص برای زبان‌هایی با ویژگی‌های ظریف که ساختار پیچیده‌تری دارند.
 4. با این حال، سیستم‌هایی مانند گوگل ترنسلیت ممکن است بین سرعت و دقت تعادل برقرار کنند.

14.2 طبقه‌بندی Rocchio: مرزهای مستقیم در فضای برداری

🌐 **گوگل نیوز و چالش طبقه‌بندی اخبار: چگونه اخبار مرتبط با یک کشور را از اخبار دیگر جدا کنیم؟**

تصور کنید در تیم گوگل نیوز کار می‌کنید و باید سیستمی طراحی کند که اخبار مرتبط با کشورهای مختلف (چین، بریتانیا، آمریکا، ژاپن و...) را از هم جدا کند. این چالش پیچیده‌ای است زیرا اخبار مربوط به یک کشور ممکن است واژه‌های مشابهی با اخبار کشور دیگر داشته باشد.

طبقه‌بندی Rocchio یکی از بهترین روش‌ها برای محاسبه مرزهای تصمیم خوب در فضای برداری است. این روش از **مراکزها** یا **نمونه‌های اولیه** برای تعریف کلاس‌ها استفاده می‌کند و سپس بر اساس فاصله، سند را به کلاسی اختصاص می‌دهد که مراکز آن به سند نزدیک‌تر است.

در طبقه‌بندی Rocchio، مراکز کلاس c به صورت زیر محاسبه می‌شود:

$$\tilde{\mu}(c) = \frac{1}{|D_c|} \sum_{d \in D_c} \tilde{\mathbf{v}}(d)$$

که در آن D_c مجموعه اسنادی از کلاس c و $\tilde{\mathbf{v}}(d)$ بردار نرمال‌شده سند d است.

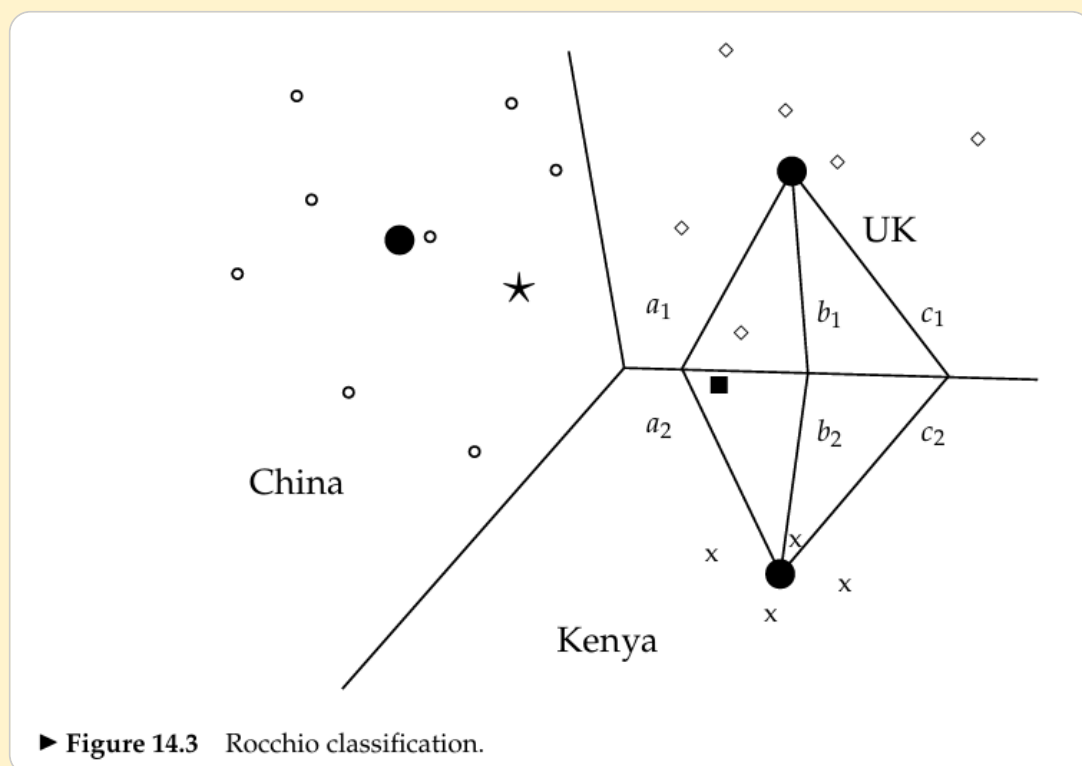
مرز بین دو کلاس در طبقه‌بندی Rocchio، مجموعه‌ای از نقاط است که فاصله آن‌ها از دو مراکز برابر است. در فضای دو بعدی، این مرز یک خط است. در فضای M بعدی، این مرز یک هایپرپلن است که به صورت زیر تعریف می‌شود:

$$\tilde{\mathbf{w}} \cdot \tilde{\mathbf{x}} = b$$

که در آن $\tilde{\mathbf{v}}(w)$ بردار نرمال‌شده هایپرپلن و b یک ثابت است. این تعریف شامل خطوط (هر خط در فضای 2D) و صفحات (هر صفحه در فضای 3D) و هایپرپلن‌ها در فضاهای با ابعاد بالاتر می‌شود.

📐 شکل 14.3: مرزهای تصمیم Rocchio برای سه کلاس

شکل 14.3 مرزهای تصمیم Rocchio را برای سه کلاس (UK، China و Kenya) نشان می‌دهد. اسناد به صورت دایره‌ها، الماس‌ها و نماد X نمایش داده شده‌اند. مرز بین دو کلاس مجموعه‌ای از نقاط است که فاصله آن‌ها از دو مراکز برابر است. برای مثال، مرز بین کلاس China و Kenya یک خط است که به صورت $|\tilde{\mathbf{v}}(d_1)| = |\tilde{\mathbf{v}}(d_2)|$ تعریف می‌شود.



قانون طبقه‌بندی در Rocchio به این صورت است که سند به کلاسی اختصاص داده می‌شود که مراکز آن به سند نزدیک‌تر است. به طور معادل، ما مراکزی را پیدا می‌کنیم که سند به آن نزدیک‌تر است و سپس سند را به کلاس مربوطه اختصاص می‌دهیم.

مثال عملی: طبقه‌بندی اخبار مربوط به چین

جدول 14.1 بردارها و مراکزها را برای داده‌های جدول 13.1 نشان می‌دهد. برای مثال، بردار سند $d_5 (0, 0.71, 0.71, 0, 0)$ مربوط به خبری است که حاوی واژه‌های "Tokyo"، "Japan"، "Chinese" و "Macao" است. مراکز کلاس China بر اساس این داده‌ها $\tilde{v}(\mu_{\text{China}}) = (0.33, 0.33, 0.33, 0)$ است.

برای طبقه‌بندی این سند، فاصله آن از مراکزهای مختلف محاسبه می‌شود:

vector	term weights					
	Chinese	Japan	Tokyo	Macao	Beijing	Shanghai
$\vec{d}_1$	0	0	0	0	1.0	0
$\vec{d}_2$	0	0	0	0	0	1.0
$\vec{d}_3$	0	0	0	1.0	0	0
$\vec{d}_4$	0	0.71	0.71	0	0	0
$\vec{d}_5$	0	0.71	0.71	0	0	0
$\vec{\mu}_c$	0	0	0	0.33	0.33	0.33
$\vec{\mu}_{\bar{c}}$	0	0.71	0.71	0	0	0

► **Table 14.1** Vectors and class centroids for the data in Table 13.1.

$$|\tilde{v}(d_5) - \tilde{v}(\mu_{\text{China}})| \approx 1.15$$

$$|\tilde{v}(d_5) - \tilde{v}(\mu_{\text{UK}})| \approx 1.00$$

$$|\tilde{v}(d_5) - \tilde{v}(\mu_{\text{Kenya}})| \approx 1.00$$

بنابراین، سند به کلاس China اختصاص داده می‌شود زیرا به مراکز China نزدیک‌تر است.

شکل 14.4: الگوریتم Rocchio در شبکه‌کد

شکل 14.4 الگوریتم Rocchio را در شبکه‌کد نشان می‌دهد:

```
TRAINROCCHIO(C, ID)
1  for each  $c_j \in C$ 
2  do  $D_j \leftarrow \{d : \langle d, c_j \rangle \in ID\}$ 
3      $\vec{\mu}_j \leftarrow \frac{1}{|D_j|} \sum_{d \in D_j} \vec{v}(d)$ 
4  return  $\{\vec{\mu}_1, \dots, \vec{\mu}_J\}$ 

APPLYROCCHIO( $\{\vec{\mu}_1, \dots, \vec{\mu}_J\}, d$ )
1  return  $\arg \min_j |\vec{\mu}_j - \vec{v}(d)|$ 
```

► **Figure 14.4** Rocchio classification: Training and testing.

الگوریتم به این صورت عمل می‌کند که برای هر کلاس c_j :

1. تمام اسناد مربوط به آن کلاس (D_j) را جمع می‌کند.

2. مراکز $\tilde{v}(\mu_j)$ را به صورت میانگین بردارهای آن اسناد محاسبه می‌کند.

3. برای طبقه‌بندی یک سند جدید d ، بردار آن $\tilde{v}(d)$ را محاسبه کرده و به هر مراکز $\tilde{v}(\mu_j)$ فاصله را محاسبه می‌کند.

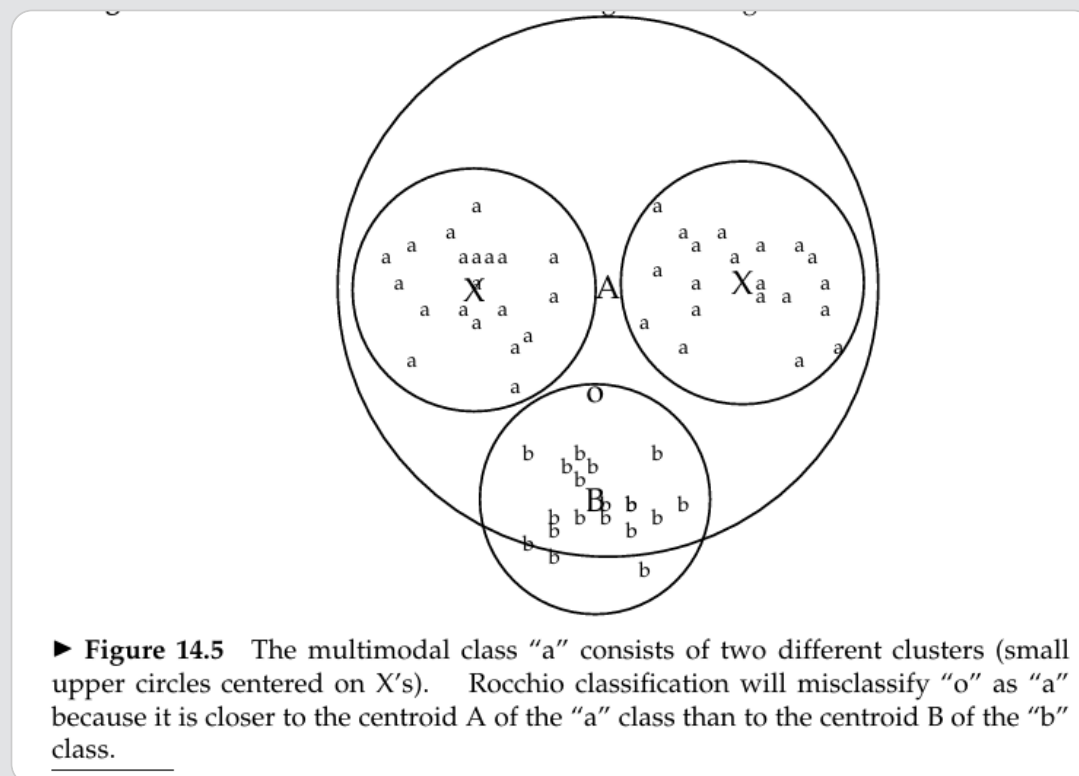
4. سند را به کلاسی اختصاص می‌دهد که مراکز آن به سند نزدیک‌تر است.

با این الگوریتم، سیستم می‌تواند تمام مراکزها و سپس قوانین طبقه‌بندی را برای طبقه‌بندی اسناد جدید محاسبه و ذخیره کند.

🎯 پیچیدگی‌های Rocchio: چه زمانی کار نمی‌کند؟

با وجود سادگی و کارایی بالا، طبقه‌بندی Rocchio محدودیت‌هایی دارد:

- **کلاس‌های چندوجهی (multimodal):** Rocchio فرض می‌کند که کلاس‌ها تقریباً کروی با شعاع‌های مشابه هستند. این فرض برای بسیاری از وظایف طبقه‌بندی متن درست نیست. مثال کلاسی "a" در شکل 14.5 شامل دو خوشه مجزا با شعاع‌های مشابه است.



- **توزیع نابرابر:** Rocchio فقط از فاصله از مرکز استفاده می‌کند و جزئیات توزیع نقاط در هر کلاس را نادیده می‌گیرد. در شکل 14.3، Rocchio کلاس Kenya را به China اختصاص می‌دهد زیرا توزیع نقاط آن پراکندگی بیشتری دارد، در حالی که توزیع نقاط China فشرده‌تر است.

مثال واقعی: در سیستم طبقه‌بندی مقالات علمی آرکایو (ArXiv)، برخی حوزه‌ها مانند "یادگیری ماشین" ممکن است شامل زیرشاخه‌های متفاوتی (مانند یادگیری عمیق در مقابل یادگیری تقویتی) باشند که ساختار پیچیده‌ای دارند. طبقه‌بندی Rocchio ممکن است در تشخیص این زیرشاخه‌ها دچار شود و همه آن‌ها را به یک کلاس اصلی اختصاص دهد.

⚙️ پیچیدگی زمانی: چگونه کارایی را محاسبه کنیم؟

جدول 14.2 زمان آموزش و آزمون Rocchio را با روش‌های دیگر مقایسه می‌کند. پیچیدگی زمانی Rocchio به صورت زیر محاسبه می‌شود:

$$\mathcal{O}(|D|L_{ave} + |C||V|)$$

mode	time complexity
training	$\Theta(\mathcal{D} L_{\text{ave}} + \mathcal{C} V)$
testing	$\Theta(L_{\text{a}} + \mathcal{C} M_{\text{a}}) = \Theta(\mathcal{C} M_{\text{a}})$

► **Table 14.2** Training and test times for Rocchio classification. L_{ave} is the average number of tokens per document. L_{a} and M_{a} are the numbers of tokens and types, respectively, in the test document. Computing Euclidean distance between the class centroids and a document is $\Theta(|\mathcal{C}|M_{\text{a}})$.

که در آن $|\mathcal{D}|$ تعداد اسناد، L_{ave} میانگین تعداد توکن در هر سند، $|\mathcal{C}|$ تعداد کلاس‌ها و $|V|$ تعداد ویژگی‌ها است. این نشان می‌دهد که پیچیدگی زمانی Rocchio خطی در اندازه مجموعه آموزشی است.

در مقایسه، طبقه‌بندی Rocchio و بیز ساده هر دو پیچیدگی زمانی خطی یکسانی دارند، زیرا هر دو روش تنها نیاز به محاسبه مراکزها و سپس مقایسه فاصله‌ها دارد. این در حالی که هر دو روش به اندازه مجموعه آموزشی خطی است.

In [5]: *# پیاده‌سازی طبقه‌بندی Rocchio*

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import normalize

def train_rocchio(documents, labels):
    """
    Rocchio آموزش طبقه‌بند

    این تابع مراکزها را برای هر کلاس محاسبه کرده و برمی‌گرداند.
    """
    # tf-idf تبدیل اسناد به بردارهای
    vocab = set()
    for doc in documents:
        vocab.update(doc)

    vocab_list = list(vocab)
    # برای هر سند و واژه tf محاسبه
    tf_matrix = np.zeros((len(documents), len(vocab_list)))
    for i, doc in enumerate(documents):
        for j, term in enumerate(vocab_list):
            tf_matrix[i, j] = doc.count(term)

    # برای هر واژه df محاسبه
    df = np.zeros(len(vocab_list))
    for j, term in enumerate(vocab_list):
        df[j] = sum(1 for doc in documents if term in doc)

    # محاسبه idf
    n_docs = len(documents)
    idf = np.log(n_docs / (df + 1))

    # محاسبه tf-idf
    tfidf_matrix = tf_matrix * idf

    # نرمال‌سازی بردارها
    normalized_vectors = normalize(tfidf_matrix, norm='l2')

    # محاسبه مراکز برای هر کلاس
    classes = list(set(labels))
    centroids = {}

    for c in classes:
        # شاخص‌های مربوط به کلاس c
        class_indices = [i for i, label in enumerate(labels) if label == c]
        class_vectors = normalized_vectors[class_indices]

        # محاسبه مراکز به صورت میانگین بردارهای آن کلاس
        centroids[c] = np.mean(class_vectors, axis=0)

    # برای استفاده در مرحله آزمون idf بازگرداندن مراکزها، واژگان، و مقادیر
    return centroids, vocab_list, idf, normalized_vectors

def apply_rocchio(centroids, vocab_list, idf, test_doc):
    """
    روی یک سند آزمون Rocchio اعمال طبقه‌بند

    این تابع سند آزمون را به کلاسی اختصاص می‌دهد که مراکز آن به سند نزدیکتر است.
    """
    # tf-idf تبدیل سند آزمون به بردار
    test_vector = np.zeros(len(vocab_list))
    for j, term in enumerate(vocab_list):
        test_vector[j] = test_doc.count(term)

    # از مرحله آموزش idf با استفاده از tf-idf محاسبه
    tfidf_vector = test_vector * idf

    # نرمال‌سازی بردار آزمون
    normalized_test_vector = normalize(tfidf_vector.reshape(1, -1), norm='l2')

    # محاسبه فاصله از هر مراکز
```

```
min_distance = float('inf')
predicted_class = None

for c, centroid in centroids.items():
    distance = np.linalg.norm(normalized_test_vector - centroid)
    if distance < min_distance:
        min_distance = distance
        predicted_class = c

return predicted_class, normalized_test_vector

# مثال کاربردی: طبقه‌بندی اخبار مربوط به کشورها
documents = [
    ["Chinese", "Beijing", "Mao"],
    ["Chinese", "Shanghai", "Mao"],
    ["Chinese", "Taiwan", "Mao"],
    ["Chinese", "Tokyo", "Mao"],
    ["British", "London", "Queen"],
    ["British", "London", "Queen"],
    ["Kenyan", "Nairobi", "tourism"],
    ["Kenyan", "Nairobi", "safari"]
]
labels = ["China", "China", "China", "China", "UK", "UK", "Kenya", "Kenya"]

# Rocchio آموزش طبقه‌بند
centroids, vocab, idf_values, normalized_vectors = train_rocchio(documents, labels)

# نمایش مراکزها
print("مراکزهای محاسبه شده برای هر کلاس:")
for c, centroid in centroids.items():
    print(f"کلاس {c}: {centroid}")

# اعمال روی سند آزمون
test_doc = ["Chinese", "Beijing", "economy"]
predicted_class, normalized_test_vector = apply_rocchio(centroids, vocab, idf_values, test_doc)
print(f"\nسند آزمون: {test_doc}")
print(f"کلاس پیش‌بینی شده: {predicted_class}")

# نمایش فضای برداری و مرزهای تصمیم
plt.figure(figsize=(12, 8))
colors = {'China': 'red', 'UK': 'blue', 'Kenya': 'green'}

# نمایش اسناد آموزشی
for i, label in enumerate(labels):
    plt.scatter(normalized_vectors[i, 0], normalized_vectors[i, 1],
                c=colors[label], alpha=0.7, label=label)

# نمایش مراکزها
for c, centroid in centroids.items():
    plt.scatter(centroid[0], centroid[1], s=200, c='black', marker='x',
                label=f"Centroid {c}")

# نمایش سند آزمون
plt.scatter(normalized_test_vector[0, 0], normalized_test_vector[0, 1],
            c='purple', marker='*', s=200, label="Test document")

# رسم مرزهای تصمیم (فقط بین دو کلاس اول)
centroid_china_2d = np.array([centroids["China"][0], centroids["China"][1]])
centroid_uk_2d = np.array([centroids["UK"][0], centroids["UK"][1]])

# محاسبه نقطه میانی خط
midpoint = (centroid_china_2d + centroid_uk_2d) / 2
# محاسبه بردار جهت
direction = centroid_uk_2d - centroid_china_2d
# محاسبه بردار عمود بر خط
normal = np.array([-direction[1], direction[0]])
normal = normal / np.linalg.norm(normal)

# رسم بخشی از خط
line_length = 0.8
line_start = midpoint - normal * line_length / 2
line_end = midpoint + normal * line_length / 2

plt.plot([line_start[0], line_end[0]], [line_start[1], line_end[1]],
        '--', alpha=0.5, color='gray')

plt.xlabel('first dimension')
plt.ylabel('second dimension')
plt.title('Rocchio classification for news articles')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.show()

# کلاس‌های چندوجهی Rocchio: نمایش پیچیدگی
print("\nRocchio پیچیدگی‌های:")
print("1. فرض می‌کند که کلاس‌ها تقریباً کروی با شعاع‌های مشابه هستند (multimodal): Rocchio کلاس‌های چندوجهی")
print("2. فقط از فاصله از مرکز استفاده می‌کند و جزئیات توزیع نقاط در هر کلاس را نادیده می‌گیرد Rocchio: توزیع نابر")
print("3. مثال واقعی: در سیستم طبقه‌بندی مقالات علمی، برخی حوزه‌ها ممکن است شامل زیرشاخه‌های متفاوتی (مانند یادگیری ماشین) باشند که ساختار پیچیده‌ای دارند")

# نمایش پیچیدگی زمانی
print("\nپیچیدگی زمانی:")
print("به صورت زیر محاسبه می‌شود Rocchio پیچیدگی زمانی:")
print("O(|D|Lave + |C||V|)")
print("تعداد ویژگی‌ها است |V| تعداد کلاس‌ها و |C|، میانگین تعداد توکن در هر سند Lave، تعداد اسناد |D| که در آن")
print("خطی در اندازه مجموعه آموزشی است Rocchio این نشان می‌دهد که پیچیدگی زمانی")
print("و بیز ساده هر دو پیچیدگی زمانی خطی یکسانی دارند Rocchio در مقایسه، طبقه‌بندی")
```


الگوریتم kNN بر اساس فرضیه تداوم

TRAIN-KNN($\mathcal{C}, \mathcal{ID}$)

```
1  $\mathcal{ID}' \leftarrow \text{PREPROCESS}(\mathcal{ID})$ 
2  $k \leftarrow \text{SELECT-K}(\mathcal{C}, \mathcal{ID}')$ 
3 return  $\mathcal{ID}', k$ 
```

APPLY-KNN($\mathcal{C}, \mathcal{ID}', k, d$)

```
1  $S_k \leftarrow \text{COMPUTENEARESTNEIGHBORS}(\mathcal{ID}', k, d)$ 
2 for each  $c_j \in \mathcal{C}$ 
3 do  $p_j \leftarrow |S_k \cap c_j|/k$ 
4 return  $\arg \max_j p_j$ 
```

► Figure 14.7 kNN training (with preprocessing) and testing. p_j is an estimate for $P(c_j|S_k) = P(c_j|d)$. c_j denotes the set of all documents in the class c_j .

الگوریتم kNN بر اساس فرضیه تداوم است که اسناد نزدیک به هم در فضای برداری به هم تعلق دارند. این روش به طور مستقیم از مجموعه آموزشی استفاده می‌کند و نیازی به مرحله آموزش صریح ندارد.

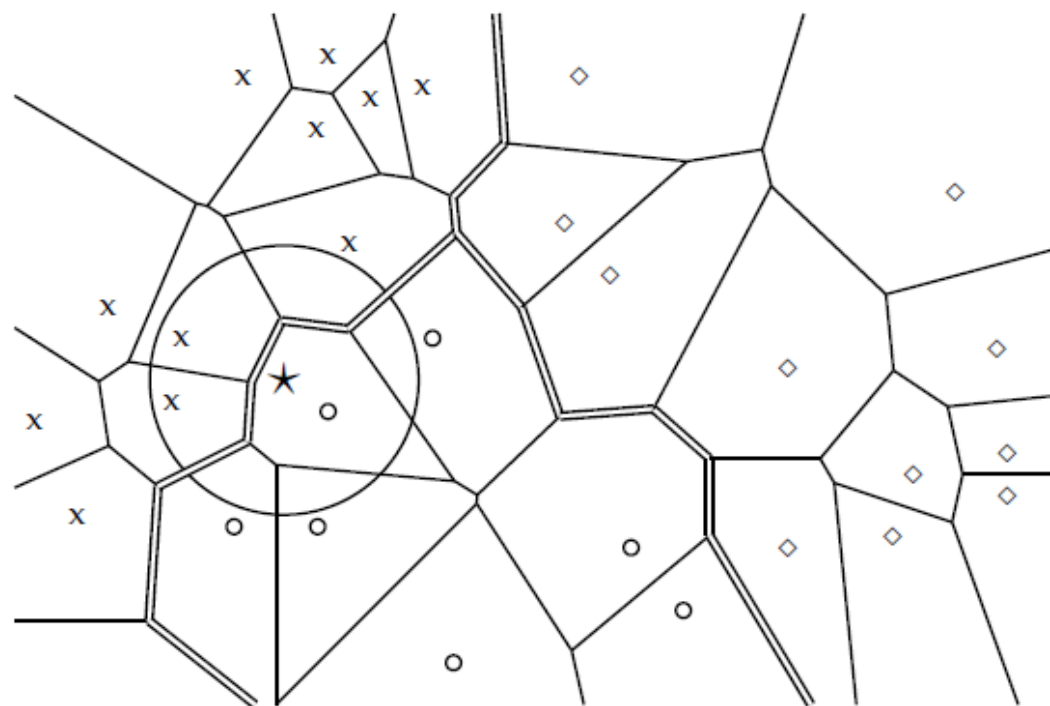
الگوریتم kNN به این صورت عمل می‌کند که برای هر سند آزمون d ، k نزدیک‌ترین همسایه‌های آن در مجموعه آموزشی را پیدا می‌کند:

1. فاصله هر سند آزمون d از هر کلاس c محاسبه می‌شود.

2. سند به کلاسی اختصاص داده می‌شود که دارای کمترین فاصله.

مثال واقعی: در سیستم اسپاتیفای، kNN برای پیدا کردن موسیقی مشابه بر اساس ویژگی‌های صوتی بسیار مفید است. این سیستم می‌تواند دو آهنگ با ریتم‌های متفاوت را به عنوان "مشابه" شناسایی کند، حتی اگر متون آن‌ها کاملاً متفاوت باشند. این کار به ویژگی‌های صوتی که مستقل از زبان و فرهنگ موسیقی هستند تکیه می‌دهد.

شکل 14.6: تقسیم فضای برداری به موزاییک‌های ورونویی



► Figure 14.6 Voronoi tessellation and decision boundaries (double lines) in 1NN classification. The three classes are: X, circle and diamond.

شکل 14.6 تقسیم فضای برداری را به موزاییک‌های ورونویی (Voronoi tessellation) تقسیم می‌کند. در این تقسیم، هر سند به یک سلول اختصاص داده می‌شود و هر سلول شامل اسنادی است که به آن کلاس تعلق دارند. مرزهای بین سلول‌ها هایپرپلن‌ها هستند که فضای بین سلول‌ها را به مناطقی تقسیم می‌کنند.

در M بعدی، هایپرپلن‌ها (M-1) فضای بین سلول‌ها را تقسیم می‌کنند که به صورت چندوجهی (M-1) بعدی تقسیم می‌شوند. این تقسیم‌ها می‌توانند ساختارهای پیچیده‌ای مانند کره‌ها، دونات و اشکال را نمایش دهند.

🚩 پیچیدگی‌های kNN: چه زمانی کار نمی‌کند؟

با وجود سادگی و کارایی بالا، طبقه‌بندی kNN محدودیت‌هایی دارد:

- **حساسیت بالا در مجموعه‌های بزرگ:** برای مجموعه‌های بزرگ، محاسبه فاصله برای هر سند آزمون ممکن است زمان‌بر باشد.
- **حساسیت ذخیره:** kNN به حافظه نیاز دارد تا تمام اسناد آموزشی را ذخیره کند.
- **عدم تعادل وزن‌دهی ویژگی‌ها:** ویژگی‌هایی با مقادیر بزرگ ممکن است بر ویژگی‌های با مقادیر کوچک تأثیر بیشتری داشته باشند.

مثال واقعی: در سیستم طبقه‌بندی تصاویر پینترست (Pinterest)، kNN برای پیدا کردن تصاویر مشابه بر اساس ویژگی‌های بصری ممکن است بسیار کارآمد باشد. با این حال، سیستم می‌تواند تصاویر مشابه را حتی اگر برچسب‌ها یا توضیحات متفاوتی داشته باشند پیدا کند.

🌀 پیچیدگی زمانی: چگونه کارایی را بهبود آوریم؟

پیچیدگی زمانی kNN به صورت زیر محاسبه می‌شود:

$$\Theta(|D|L_{ave} + |C||V|)$$

که در آن $|D|$ تعداد اسناد، L_{ave} میانگین تعداد توکن در هر سند، $|C|$ تعداد کلاس‌ها و $|V|$ تعداد ویژگی‌ها است. این نشان می‌دهد که پیچیدگی زمانی kNN خطی در اندازه مجموعه آموزشی است.

در مقایسه، پیچیدگی زمانی kNN معمولاً از Rocchio بدتر است زیرا نیاز به محاسبه و ذخیره تمام اسناد آموزشی دارد. با این حال، پیچیدگی زمانی Rocchio خطی در اندازه مجموعه آموزشی است، زیرا هر دو روش تنها نیاز به محاسبه مراکزها و سپس مقایسه فاصله‌ها دارد.

In [9]: *# (kNN) نزدیک‌ترین همسایه k پیاده‌سازی طبقه‌بندی*

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.decomposition import PCA

def euclidean_distance(v1, v2):
    """
    محاسبه فاصله اقلیدسی بین دو بردار

    این تابع فاصله اقلیدسی استاندارد بین دو بردار را محاسبه می‌کند.

    پارامترها:
    v1, v2: دو بردار که می‌خواهیم فاصله بین آن‌ها را محاسبه کنیم

    خروجی: مقدار فاصله اقلیدسی
    """
    return np.sqrt(np.sum((v1 - v2) ** 2))
```

```
def find_k_nearest_neighbors(train_docs, train_labels, test_doc, k, vocab_list):
    """
    نزدیکترین همسایه‌ها برای یک سند آزمون k پیدا کردن
    """
    # برای سند آزمون tf محاسبه
    test_vector = np.zeros(len(vocab_list))
    for j, term in enumerate(vocab_list):
        test_vector[j] = test_doc.count(term)

    # محاسبه فاصله از هر سند آزمون به تمام اسناد آموزشی
    distances = []
    for i, (doc, label) in enumerate(zip(train_docs, train_labels)):
        # تبدیل سند آموزشی به بردار tf
        train_vector = np.zeros(len(vocab_list))
        for j, term in enumerate(vocab_list):
            train_vector[j] = doc.count(term)

        # محاسبه فاصله اقلیدسی
        distance = euclidean_distance(test_vector, train_vector)
        distances.append((distance, label))

    # نزدیکترین k مرتب‌سازی بر اساس فاصله و انتخاب
    distances.sort(key=lambda x: x[0])
    k_nearest = distances[:k]

    # نزدیکترین همسایه‌ها k شمارش آرا برای هر کلاس در
    class_votes = Counter([label for _, label in k_nearest])

    # انتخاب کلاس با بیشترین آرا
    predicted_class = class_votes.most_common(1)[0][0]

    return predicted_class, k_nearest

def classify_knn(train_docs, train_labels, test_docs, k):
    """
    kNN طبقه‌بندی اسناد با استفاده از
    """
    # ایجاد واژگان مشترک
    vocab = set()
    for doc in train_docs + test_docs:
        vocab.update(doc)
    vocab_list = list(vocab)

    predictions = []
    for test_doc in test_docs:
        predicted_class, _ = find_k_nearest_neighbors(train_docs, train_labels, test_doc, k, vocab_list)
        predictions.append(predicted_class)

    return predictions

def evaluate_knn(train_docs, train_labels, test_docs, test_labels, k):
    """
    kNN ارزیابی عملکرد
    """
    predictions = classify_knn(train_docs, train_labels, test_docs, k)
    correct = sum(1 for predicted, true_label in zip(predictions, test_labels) if predicted == true_label)

    accuracy = correct / len(test_docs)
    return accuracy

def plot_knn_decision_boundary(train_docs, train_labels, k=3):
    """
    Voronoi tessellation نسخه ساده از) kNN نمایش مرزهای تصمیم
    """
    plt.figure(figsize=(10, 8))

    # ایجاد واژگان مشترک
    vocab = set()
    for doc in train_docs:
        vocab.update(doc)
    vocab_list = list(vocab)

    # تبدیل اسناد به بردارهای tf
    doc_vectors = []
    for doc in train_docs:
        doc_vector = np.zeros(len(vocab_list))
        for j, term in enumerate(vocab_list):
            doc_vector[j] = doc.count(term)
        doc_vectors.append(doc_vector)

    # برای نمایش در فضای دو بعدی PCA کاهش ابعاد با
    pca = PCA(n_components=2)
    doc_vectors_2d = pca.fit_transform(doc_vectors)

    # نمایش اسناد آموزشی
    colors = {'China': 'red', 'UK': 'blue', 'Kenya': 'green'}

    for i, (vector_2d, label) in enumerate(zip(doc_vectors_2d, train_labels)):
        plt.scatter(vector_2d[0], vector_2d[1],
                    c=colors[label], alpha=0.7, label=label)

    unique_labels = list(set(train_labels))
    label_to_numeric = {label: i for i, label in enumerate(unique_labels)}

    # نمایش مرزهای تصمیم (فقط بین دو کلاس اول)
    # ایجاد یک شبکه برای نمایش مرزهای تصمیم
    x_min, x_max = doc_vectors_2d[:, 0].min() - 1, doc_vectors_2d[:, 0].max() + 1
    y_min, y_max = doc_vectors_2d[:, 1].min() - 1, doc_vectors_2d[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                        np.arange(y_min, y_max, 0.1))

    # طبقه‌بندی هر نقطه در شبکه
    Z = []
```

```
for point in np.c_[xx.ravel(), yy.ravel()]:
    # بازگشت به فضای اصلی با استفاده از معکوس PCA
    point_original = pca.inverse_transform(point.reshape(1, -1))

    # پیدا کردن نزدیک‌ترین همسایه
    min_distance = float('inf')
    nearest_label = None
    for i, (vector, label) in enumerate(zip(doc_vectors, train_labels)):
        distance = euclidean_distance(point_original, vector)
        if distance < min_distance:
            min_distance = distance
            nearest_label = label

    Z.append(label_to_numeric[nearest_label])

Z = np.array(Z).reshape(xx.shape)

cmap = plt.cm.colors.ListedColormap([colors[label] for label in unique_labels])

# نمایش مرزهای تصمیم
plt.contourf(xx, yy, Z, alpha=0.2, cmap=cmap)

plt.xlabel('first principal component')
plt.ylabel('second principal component')
plt.title('Decision boundary of kNN (k=3)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.show()

# مثال کاربردی: طبقه‌بندی اخبار مربوط به کشورها
documents = [
    ["Chinese", "Beijing", "Mao"],
    ["Chinese", "Shanghai", "Mao"],
    ["Chinese", "Taiwan", "Mao"],
    ["Chinese", "Tokyo", "Mao"],
    ["British", "London", "Queen"],
    ["British", "London", "Queen"],
    ["Kenyan", "Nairobi", "tourism"],
    ["Kenyan", "Nairobi", "safari"]
]
labels = ["China", "China", "China", "China", "UK", "UK", "Kenya", "Kenya"]

# k=1 طبقه‌بندی با
predicted_1nn = classify_knn(documents, labels, documents, 1)
print(f"پیش‌بینی kNN (k=1): {predicted_1nn}")

# k=3 طبقه‌بندی با
predicted_3nn = classify_knn(documents, labels, documents, 3)
print(f"پیش‌بینی kNN (k=3): {predicted_3nn}")

# ارزیابی عملکرد
accuracy_1nn = evaluate_knn(documents, labels, documents, labels, 1)
accuracy_3nn = evaluate_knn(documents, labels, documents, labels, 3)
print(f"\ndقت kNN (k=1): {accuracy_1nn:.2f}")
print(f"dقت kNN (k=3): {accuracy_3nn:.2f}")

# نمایش مرزهای تصمیم
plot_knn_decision_boundary(documents, labels, k=3)

# kNN نمایش پیچیدگی‌های
print("\nkNN پیچیدگی‌های:")
print("1. حساسیت بالا در مجموعه‌های بزرگ: برای مجموعه‌های بزرگ، محاسبه فاصله برای هر سند آزمون ممکن است زمان‌بر باشد.")
print("2. به حافظه نیاز دارد تا تمام اسناد آموزشی را ذخیره کند: حساسیت ذخیره.")
print("3. عدم تعادل وزن‌دهی ویژگی‌ها: ویژگی‌هایی با مقادیر بزرگ ممکن است بر ویژگی‌های با مقادیر کوچک تأثیر بیشتری داشته باشند.")

# نمایش پیچیدگی زمانی
print("\nkNN پیچیدگی زمانی:")
print("به صورت زیر محاسبه می‌شود kNN پیچیدگی زمانی")
print("O(|D|L<sub>ave</sub> + |C||V|)")
print("تعداد ویژگی‌ها است |V| تعداد کلاس‌ها و |C| میانگین تعداد توکن در هر سند L<sub>ave</sub>، تعداد اسناد |D| که در آن")
print("خطی در اندازه مجموعه آموزشی است kNN این نشان می‌دهد که پیچیدگی زمانی")
print("بدتر است زیرا نیاز به محاسبه و ذخیره تمام اسناد آموزشی دارد Rocchio معمولاً از kNN در مقایسه، پیچیدگی زمانی")

# مثال واقعی: سیستم اسپاتیفای
print("\nمثال واقعی: سیستم اسپاتیفای")
print("برای پیدا کردن موسیقی مشابه بر اساس ویژگی‌های صوتی بسیار مفید است kNN، در سیستم اسپاتیفای")
print("این سیستم می‌تواند دو آهنگ با ریتم‌های متفاوت را به عنوان "مشابه" شناسایی کند، حتی اگر متون آن‌ها کاملاً متفاوت باشند")
print("این کار به ویژگی‌های صوتی که مستقل از زبان و فرهنگ موسیقی هستند تکیه می‌دهد")

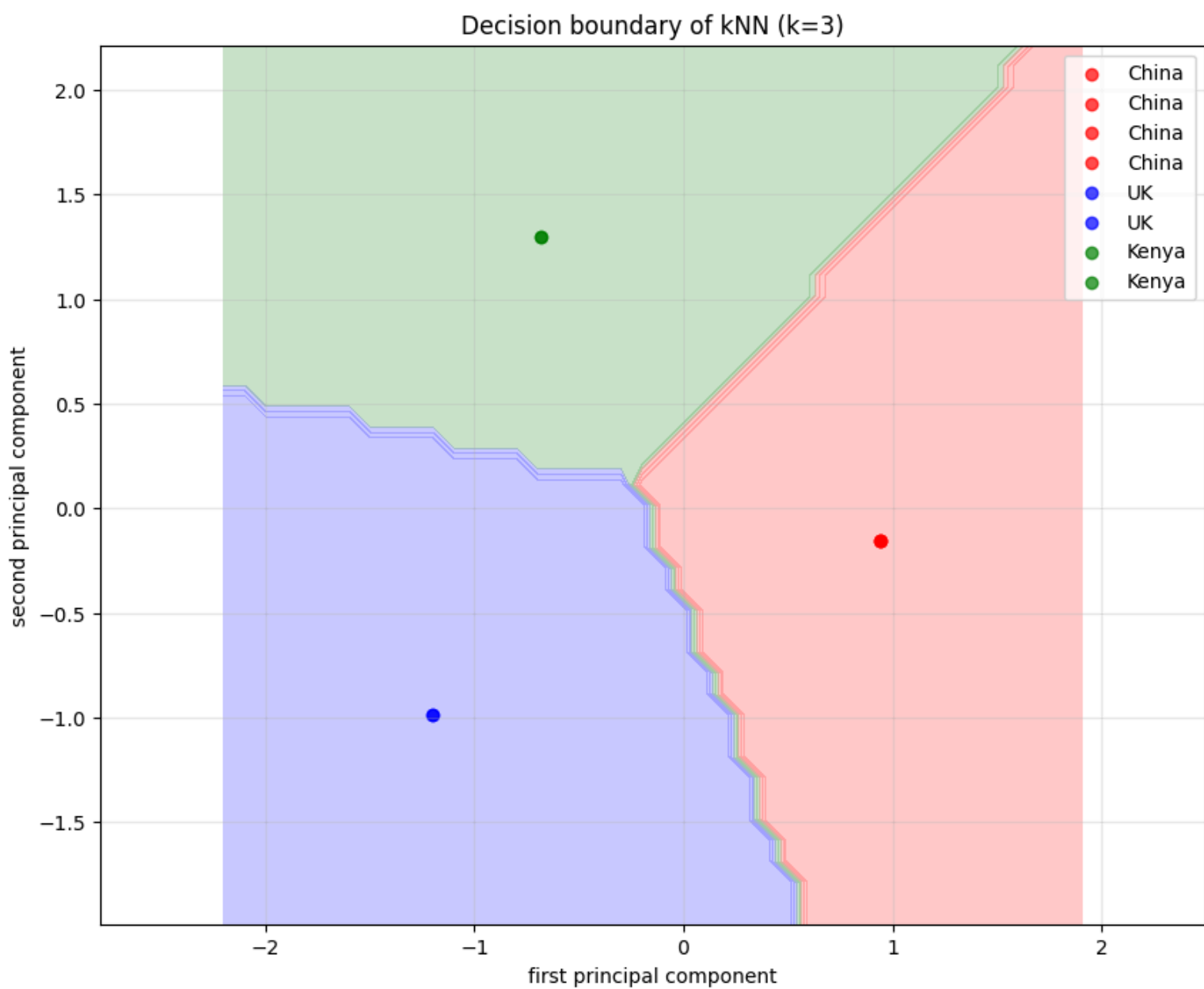
# نمایش پیچیدگی زمانی
print("\nkNN پیچیدگی زمانی: چگونه کارایی را بهبود آوریم؟")
print("1. برای جستجوی سریع‌تر KD-tree بهینه‌سازی ساختار داده: استفاده از ساختارهای داده کارآمد مانند")
print("2. به تعداد معقولی برای کاهش محاسبات k استفاده از تقریب‌دهی تقریبی: محدود کردن")
print("3. استفاده از فاصله‌های جایگزین مانند فاصله کسینوسی که به وزن‌دهی ویژگی‌ها توجه بیشتری به موارد مهم می‌دهد")
print("4. که به ویژگی‌های با وزن کمتر وزن بیشتری می‌دهد Local Sensitive Hashing (LSH) استفاده از روش‌های احتمالی مانند")
```

پیش‌بینی kNN (k=1): ['China', 'China', 'China', 'China', 'UK', 'UK', 'Kenya', 'Kenya']

پیش‌بینی kNN (k=3): ['China', 'China', 'China', 'China', 'UK', 'UK', 'Kenya', 'Kenya']

dقت kNN (k=1): 1.00

dقت kNN (k=3): 1.00



kNN پیچیدگی‌های

- حساسیت بالا در مجموعه‌های بزرگ: برای مجموعه‌های بزرگ، محاسبه فاصله برای هر سند آزمون ممکن است زمان‌بر باشد.
- به حافظه نیاز دارد تا تمام اسناد آموزشی را ذخیره کند: حساسیت ذخیره.
- عدم تعادل وزن‌دهی ویژگی‌ها: ویژگی‌هایی با مقادیر بزرگ ممکن است بر ویژگی‌های با مقادیر کوچک تأثیر بیشتری داشته باشند.

kNN پیچیدگی زمانی

:به صورت زیر محاسبه می‌شود kNN پیچیدگی زمانی

$$\Theta(|D|L_{\text{ave}} + |C||V|)$$

تعداد ویژگی‌ها است $|V|$ تعداد کلاس‌ها و $|C|$ میانگین تعداد توکن در هر سند L_{ave} ، تعداد اسناد $|D|$ که در آن

خطی در اندازه مجموعه آموزشی است kNN این نشان می‌دهد که پیچیدگی زمانی

بدتر است زیرا نیاز به محاسبه و ذخیره تمام اسناد آموزشی دارد Rocchio معمولاً از kNN در مقایسه، پیچیدگی زمانی

مثال واقعی: سیستم اسپاتیفای

برای پیدا کردن موسیقی مشابه بر اساس ویژگی‌های صوتی بسیار مفید است kNN، در سیستم اسپاتیفای

این سیستم می‌تواند دو آهنگ با ریتم‌های متفاوت را به عنوان "مشابه" شناسایی کند، حتی اگر متون آن‌ها کاملاً متفاوت باشند

این کار به ویژگی‌های صوتی که مستقل از زبان و فرهنگ موسیقی هستند تکیه می‌دهد

پیچیدگی زمانی: چگونه کارایی را بهبود آوریم؟

- برای جستجوی سریع‌تر KD-tree بهینه‌سازی ساختار داده: استفاده از ساختارهای داده کارآمد مانند.
- به تعداد معقولی برای کاهش محاسبات k استفاده از تقریبندی تقریبی: محدود کردن.
- استفاده از فاصله‌های جایگزین مانند فاصله کسینوسی که به وزن‌دهی ویژگی‌ها توجه بیشتری به موارد مهم می‌دهد.
- که به ویژگی‌های با وزن کمتر وزن بیشتری می‌دهد Local Sensitive Hashing (LSH) استفاده از روش‌های احتمالی مانند.

14.3.1 پیچیدگی زمانی و بهینگی kNN: چالش‌های عملی در دنیای واقعی ⌚

🔍 Netflix و چالش توصیه فیلم: چگونه فیلم‌های مشابه را به سرعت پیدا کنیم؟

تصور کنید در تیم توصیه فیلم Netflix کار می‌کنید و باید سیستمی طراحی کنید که به کاربران فیلم‌های مشابه بر اساس علاقه‌مندی‌هایشان پیشنهاد دهد. با میلیون‌ها کاربر و هزاران فیلم، چگونه می‌توانید به سرعت k نزدیک‌ترین همسایه‌ها را برای هر کاربر پیدا کنید؟ این دقیقاً همان چالشی است که kNN در مقیاس بزرگ با آن روبرو است.

در حالی که kNN یک روش بسیار قدرتمند است، اما پیچیدگی زمانی آن می‌تواند یک چالش جدی برای سیستم‌های بزرگ باشد. بیایید ببینیم چگونه می‌توانیم این چالش را مدیریت کنیم.

📌 پیچیدگی زمانی kNN: آموزش سریع، تست زمان‌بر

جدول 14.3 پیچیدگی زمانی kNN را نشان می‌دهد. ویژگی‌های kNN کاملاً با اکثر الگوریتم‌های طبقه‌بندی دیگر متفاوت است:

kNN with preprocessing of training set	
training	$\Theta(D L_{ave})$
testing	$\Theta(L_a + D M_{ave}M_a) = \Theta(D M_{ave}M_a)$
kNN without preprocessing of training set	
training	$\Theta(1)$
testing	$\Theta(L_a + D L_{ave}M_a) = \Theta(D L_{ave}M_a)$

► Table 14.3 Training and test times for kNN classification. M_{ave} is the average size of the vocabulary of documents in the collection.

- **زمان آموزش:** آموزش یک طبقه‌بند kNN صرفاً شامل تعیین k و پیش‌پردازش اسناد است. در واقع، اگر مقدار k را از قبل انتخاب کنیم و پیش‌پردازش انجام ندهیم، kNN اصلاً نیازی به آموزش ندارد!
- **زمان تست:** زمان تست برای kNN برابر با $\Theta(|D|M_{ave}M_a)$ است. این زمان خطی با اندازه مجموعه آموزشی است، زیرا باید فاصله هر سند آموزشی از سند تست را محاسبه کنیم.
- **مزیت برای کلاس‌های زیاد:** زمان تست مستقل از تعداد کلاس‌های L است. بنابراین kNN برای مسائلی با L بزرگ یک مزیت بالقوه دارد.

مثال واقعی: در سیستم توصیه فیلم Amazon Prime، زمان آموزش kNN تقریباً صفر است، زیرا سیستم فقط نیاز به ذخیره تمام بردارهای ویژگی فیلم دارد. اما وقتی کاربر جدیدی وارد سیستم می‌شود، برای پیشنهاد 5 فیلم برتر، باید فاصله بین بردار ویژگی کاربر و تمام فیلم‌ها را محاسبه کند که با میلیون‌ها فیلم در مجموعه، این کار می‌تواند زمان‌بر باشد.

📁 یادگیری مبتنی بر حافظه: نقاط قوت و ضعف

در طبقه‌بندی kNN، برخلاف طبقه‌بندی Rocchio (مراکز) یا بیز ساده (اولویت‌ها و احتمالات شرطی)، هیچ تخمین پارامتری انجام نمی‌دهیم. kNN صرفاً تمام مثال‌های مجموعه آموزشی را به خاطر می‌سپارد و سپس سند تست را با آن‌ها مقایسه می‌کند. به همین دلیل، kNN همچنین یادگیری مبتنی بر حافظه یا یادگیری مبتنی بر نمونه نامیده می‌شود.

مثال واقعی: در سیستم تشخیص چهره فیس‌بوک، kNN می‌تواند چهره یک فرد را با مقایسه با تمام چهره‌های موجود در پایگاه داده شناسایی کند. این سیستم نیازی به استخراج ویژگی‌های کلی از چهره‌ها ندارد، بلکه مستقیماً تصاویر را با هم مقایسه می‌کند. این کار دقت بالایی دارد، اما با افزایش تعداد کاربران، نیاز به حافظه و قدرت پردازش بیشتر می‌شود.

🚀 بهینه‌سازی kNN: چگونه می‌توانیم کارایی را بهبود دهیم؟

آیا می‌توان تست kNN را کارآمدتر از $\Theta(|D|M_{ave}M_a)$ یا نادیده گرفتن طول اسناد، کارآمدتر از $\Theta(|D|)$ کرد؟

- **الگوریتم‌های سریع برای ابعاد کوچک:** الگوریتم‌های سریع kNN برای ابعاد کوچک M وجود دارد (تمرین 14.12).
- **تقریب‌ها برای ابعاد بزرگ:** تقریب‌هایی برای M بزرگ وجود دارد که کران‌های خطا برای بهره‌وری مشخصی ارائه می‌دهند (بخش 14.7).
- **استفاده از نمایه معکوس:** شباهت بین پیدا کردن نزدیک‌ترین همسایگان یک سند تست و بازیابی ad hoc، جایی که ما به دنبال اسناد با بیشترین شباهت به کوئری هستیم، قابل توجه است.

مثال واقعی: در سیستم جستجوی گوگل، برای پیدا کردن صفحات وب مشابه با یک کوئری، از نمایه معکوس استفاده می‌شود تا جستجو را به اسنادی محدود کند که حداقل یک کلمه مشترک با کوئری دارند. این رویکرد می‌تواند زمان جستجو را به طور

قابل توجهی کاهش دهد، به خصوص برای کوئری‌های کوتاه که با استفاده از لیست متوقف کلمات بزرگ، همپوشانی کمی با اسناد دارند.

📊 بهینگی kNN: چقدر دقیق است؟

همانطور که در فصل بعد خواهیم دید، اثربخشی kNN نزدیک به دقیق‌ترین روش‌های یادگیری در طبقه‌بندی متن است (جدول 15.2، صفحه 334). یک معیار برای کیفیت یک روش یادگیری، نرخ خطای بیز آن است، یعنی میانگین نرخ خطای طبقه‌بندی‌های یادگرفته شده توسط آن برای یک مسئله خاص.

kNN برای مسائلی با نرخ خطای بیز غیرصفر بهینه نیست - یعنی مسائلی که حتی بهترین طبقه‌بند ممکن نرخ خطای طبقه‌بندی غیرصفر دارد. خطای 1NN به صورت مجانبی (با افزایش مجموعه آموزشی) توسط دو برابر نرخ خطای بیز محدود می‌شود. یعنی اگر طبقه‌بند بهینه نرخ خطای x داشته باشد، 1NN نرخ خطای مجانبی کمتر از $2x$ خواهد داشت.

مثال واقعی: در سیستم تشخیص اسپم جیمیل، حتی بهترین طبقه‌بند ممکن است به دلیل تکنیک‌های پیچیده اسپمرها، نرخ خطای 1% داشته باشد. طبقه‌بند 1NN در این حالت ممکن است نرخ خطای تا 2% داشته باشد، که هنوز برای کاربران قابل قبول است. اما برای مسائل با نرخ خطای بیز صفر (مانند تشخیص اعداد دستنویس در شرایط ایده‌آل)، خطای 1NN با افزایش اندازه مجموعه آموزشی به صفر نزدیک می‌شود.

14.4 طبقه‌بندی‌های خطی در مقابل غیرخطی: مرزهای ساده در برابر پیچیدگی‌های دنیای واقعی 📊

🎬 چالش طبقه‌بندی فیلم در نتفلیکس: یک خط ساده کافی است؟

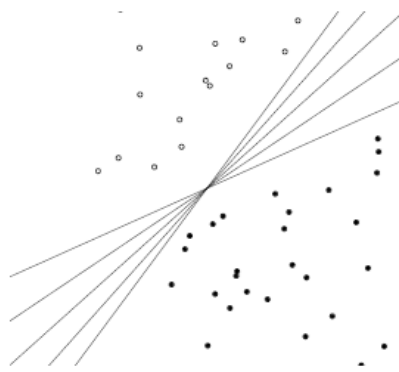
تصور کنید در تیم طبقه‌بندی محتوای نتفلیکس (Netflix) کار می‌کنید. وظیفه شما این است که فیلم‌ها را به دو دسته "اکشن" و "کمدی" تقسیم کنید. شاید بتوانید با ویژگی‌هایی مانند "تعداد انفجارها در دقیقه" یا "تعداد جوک‌ها در دقیقه" یک خط مرزی ساده بین این دو ژانر رسم کنید. این یک **طبقه‌بند خطی** است.

حالا تصور کنید وظیفه شما پیشنهاد فیلم به کاربران است. مرز بین "فیلم‌هایی که کاربر دوست دارد" و "فیلم‌هایی که دوست ندارد" بسیار پیچیده و شخصی است. آیا یک خط ساده می‌تواند این تمایز را ایجاد کند؟ به احتمال زیاد خیر. این یک **مسئله غیرخطی** است که به الگوریتم‌های پیچیده‌تری مانند kNN نیاز دارد.

در این بخش، با تفاوت طبقه‌بندی‌های خطی و غیرخطی آشنا می‌شویم. **طبقه‌بند خطی** طبقه‌بندی را با مقایسه یک ترکیب خطی از ویژگی‌ها با یک آستانه (threshold) انجام می‌دهد. در فضای دو بعدی، این طبقه‌بند یک خط است و در فضاهای با ابعاد بالاتر، یک هایپرپلن.

📐 شکل 14.8: بی‌نهایت خط جدا کردن دو کلاس

اگر دو کلاس کاملاً از هم جدا باشند (linearly separable)، بی‌نهایت خط (یا هایپرپلن) می‌تواند آن‌ها را از هم جدا کند. چالش اصلی، انتخاب بهترین خط است که روی داده‌های جدید بهترین عملکرد را داشته باشد.



► Figure 14.8 There are an infinite number of hyperplanes that separate two linearly separable classes.

الگوریتم یک طبقه‌بند خطی بسیار ساده است: یک امتیاز (score) بر اساس مجموع وزن‌دار ویژگی‌ها محاسبه شده و با آستانه مقایسه می‌شود.

🌸 شکل 14.9: الگوریتم طبقه‌بندی خطی

این الگوریتم نشان می‌دهد که چگونه یک بردار ویژگی ($\vec{x}$) با استفاده از بردار وزن‌ها ($\vec{w}$) و آستانه (b) طبقه‌بندی می‌شود.

```

APPLYLINEARCLASSIFIER( $\vec{w}, b, \vec{x}$ )
1   $score \leftarrow \sum_{i=1}^M w_i x_i$ 
2  if  $score > b$ 
3    then return 1
4    else return 0

```

► Figure 14.9 Linear classification algorithm.

اگر مجموع وزن‌دار ویژگی‌ها (**score**) از آستانه **b** بیشتر باشد، سند به کلاس 1 و در غیر این صورت به کلاس 0 تعلق می‌گیرد. چالش واقعی در آموزش این طبقه‌بند است، یعنی پیدا کردن مقادیر بهینه برای $\vec{w}$ و b .

شاید تعجب‌آور باشد، اما الگوریتم‌هایی که تاکنون بررسی کردیم، یعنی **Rocchio** و **Naive Bayes**، هر دو نمونه‌هایی از طبقه‌بندهای خطی هستند.

- **Rocchio**: مرز تصمیم در Rocchio، مجموعه‌ای از نقاط است که فاصله آن‌ها از دو مرکز (centroid) کلاس‌ها برابر است. می‌توان ثابت کرد که این مرز یک هایپرپلن است.
- **Naive Bayes**: قانون تصمیم Naive Bayes بر اساس بیشترین مقدار $P(c|d)$ است. اگر از لگاریتم نسبت احتمالات (log odds) استفاده کنیم، معادله به صورت یک ترکیب خطی درمی‌آید:

$$\log P(c|d) / P(c^-|d) = \log P(c) / P(c^-) + \sum \log P(t_k|c) / P(t_k|c^-)$$

این معادله دقیقاً همان ساختار یک طبقه‌بند خطی ($\vec{w}^T \vec{x} > b$) را دارد.

📊 جدول 14.4: یک طبقه‌بند خطی برای دسته‌بندی اخبار اقتصادی

این جدول یک طبقه‌بند خطی واقعی برای دسته "interest" (نرخ بهره) در مجموعه داده Reuters-21578 را نشان می‌دهد. هر کلمه یک وزن (w_i) دارد. کلماتی مانند "rate" و "discount" وزن مثبت دارند (نشانه‌ای از دسته interest)، در حالی که کلماتی مانند "world" و "dlrs" وزن منفی دارند (احتمالاً نشانه‌ای از دسته‌های دیگر مانند "currency").

t_i	w_i	d_{1i}	d_{2i}	t_i	w_i	d_{1i}	d_{2i}
prime	0.70	0	1	dlrs	-0.71	1	1
rate	0.67	1	0	world	-0.35	1	0
interest	0.63	0	0	sees	-0.33	0	0
rates	0.60	0	0	year	-0.25	0	0
discount	0.46	1	0	group	-0.24	0	0
bundesbank	0.43	0	0	dlr	-0.24	0	0

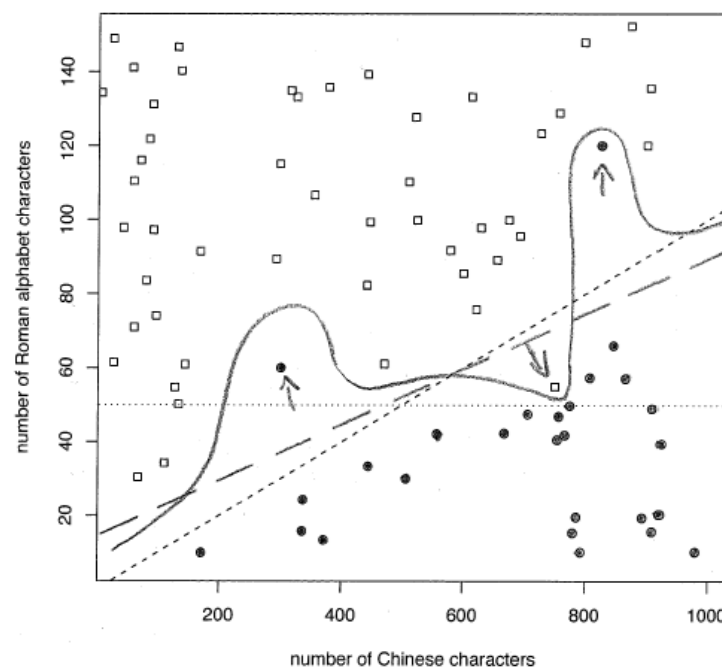
► **Table 14.4** A linear classifier. The dimensions t_i and parameters w_i of a linear classifier for the class *interest* (as in interest rate) in Reuters-21578. The threshold is $b = 0$. Terms like dlr and world have negative weights because they are indicators for the competing class *currency*.

برای طبقه‌بندی یک سند جدید، کفایت وزن کلمات موجود در سند را با هم جمع کنیم. اگر حاصل از آستانه ($b=0$) بیشتر باشد، سند به دسته "interest" تعلق دارد.

چالش‌های طبقه‌بندی خطی: نویز و جدایی‌پذیری خطی

طبقه‌بندی خطی با چالش‌های خاصی روبرو هستند:

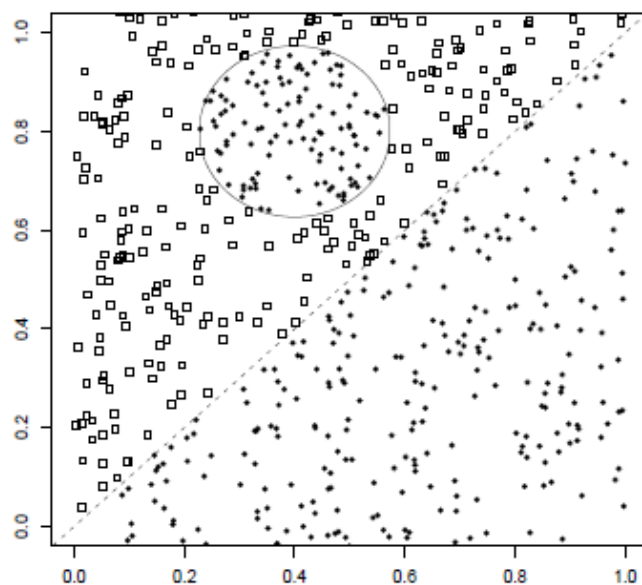
- **اسناد نویزی (Noise Documents):** اسنادی وجود دارند که در منطقه‌ای از فضا قرار دارند که اکثر اسناد متعلق به کلاس دیگری هستند. این اسناد می‌توانند الگوریتم آموزش را گمراه کنند. در شکل 14.10، سه سند با علامت پیکان، اسناد نویزی هستند.
- **جدایی‌پذیری خطی (Linear Separability):** اگر یک هایپرپلن بتواند دو کلاس را به طور کامل از هم جدا کند، می‌گوییم کلاس‌ها "خطی جدایی‌پذیر" هستند. در این صورت، بی‌نهایت هایپرپلن جداکننده وجود دارد (مانند شکل 14.8). چالش این است که بهترین هایپرپلن را انتخاب کنیم که روی داده‌های جدید بهترین عملکرد را داشته باشد.



► **Figure 14.10** A linear problem with noise. In this hypothetical web page classification scenario, Chinese-only web pages are solid circles and mixed Chinese-English web pages are squares. The two classes are separated by a linear class boundary (dashed line, short dashes), except for three noise documents (marked with arrows).

طبقه‌بندی غیرخطی: زمانی که یک خط کافی نیست

گاهی اوقات، مرز بین دو کلاس به سادگی یک خط یا هایپرپلن نیست. شکل 14.11 یک **مسئله غیرخطی** را نشان می‌دهد. یک طبقه‌بند خطی نمی‌تواند "claw" دایره‌ای شکل در گوشه بالا سمت چپ را به درستی طبقه‌بندی کند.



► **Figure 14.11** A nonlinear problem.

در این موارد، **طبقه‌بندهای غیرخطی** مانند kNN عملکرد بهتری دارند. مرز تصمیم kNN (خطوط دوتایی در شکل 14.6) از قطعات خطی محلی تشکیل شده است، اما به طور کلی شکلی پیچیده دارد که معادل یک خط یا هایپرپلن واحد نیست.

مثال واقعی: در سیستم تشخیص چهره فیس‌بوک، مرز بین "چهره شما" و "چهره افراد دیگر" بسیار پیچیده است. یک طبقه‌بند خطی ساده نمی‌تواند این تمایز را با دقت بالا ایجاد کند. در مقابل، kNN می‌تواند با مقایسه یک چهره جدید با نزدیک‌ترین چهره‌های موجود در پایگاه داده، این کار را با دقت بالایی انجام دهد.

در نهایت، انتخاب بین طبقه‌بند خطی و غیرخطی به ماهیت مسئله بستگی دارد. اگر مسئله خطی است، یک طبقه‌بند خطی ساده و کارآمد بهترین انتخاب است. اما اگر مرزهای کلاس‌ها پیچیده و غیرخطی هستند، طبقه‌بندهای غیرخطی مانند kNN دقت بالاتری خواهند داشت.

```
In [11]: # پیاده‌سازی و مقایسه طبقه‌بندهای خطی و غیرخطی

import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# تولید داده‌های خطی جداشد با نویز
def generate_linear_data(n_samples=200, noise_level=0.1):
    """
    تولید داده‌های خطی جداشد با نویز برای شبیه‌سازی چالش‌های دنیای واقعی
    """
    np.random.seed(42)

    # ایجاد دو کلاس خطی جداشد
    X_class1 = np.random.randn(n_samples, 2) + np.array([2, 2])
    X_class2 = np.random.randn(n_samples, 2) + np.array([-2, -2])

    # ایجاد برچسب‌ها
    y_class1 = np.ones(n_samples)
    y_class2 = np.zeros(n_samples)

    # ترکیب داده‌ها
    X = np.vstack([X_class1, X_class2])
    y = np.hstack([y_class1, y_class2])

    # افزودن نویز به درصدی از داده‌ها (اسناد نویزی)
    n_noise = int(noise_level * n_samples)
    noise_indices = np.random.choice(len(X), n_noise, replace=False)

    # معکوس کردن برچسب برخی از نقاط نویز
    y[noise_indices] = 1 - y[noise_indices]

    return X, y

# تولید داده‌های غیرخطی
def generate_nonlinear_data(n_samples=200):
    """
    تولید داده‌های واقعاً غیرخطی برای نمایش محدودیت‌های طبقه‌بندهای خطی
    """
    np.random.seed(42)
    # ایجاد داده‌های شکل ماه (غیرخطی)
    n = n_samples // 2
    X1 = np.zeros((n, 2))
    X2 = np.zeros((n, 2))
```

```
# کلاس اول: نیمه بالایی ماه
for i in range(n):
    angle = np.pi * i / n
    X1[i, 0] = np.cos(angle)
    X1[i, 1] = np.sin(angle)

# کلاس دوم: نیمه پایینی ماه (بدون شیب رو به پایین)
for i in range(n):
    angle = np.pi + np.pi * i / n
    X2[i, 0] = np.cos(angle)
    X2[i, 1] = np.sin(angle) # حذف شیب رو به پایین

# ترکیب داده‌ها
X = np.vstack([X1, X2])
y = np.hstack([np.zeros(n), np.ones(n)])

return X, y
```

Rocchio پیاده‌سازی طبقه‌بند خطی

```
class RocchioClassifier:
    """
    که یک طبقه‌بند خطی است Rocchio پیاده‌سازی ساده طبقه‌بند
    """

    def __init__(self):
        self.centroids = {}

    def fit(self, X, y):
        """
        برای هر کلاس centroids محاسبه مرکز
        """
        # دریافت کلاس‌های یکتا
        classes = np.unique(y)

        # محاسبه مرکز برای هر کلاس
        for c in classes:
            # انتخاب تمام نقاط داده برای کلاس c
            X_c = X[y == c]
            # محاسبه میانگین (مرکز)
            self.centroids[c] = np.mean(X_c, axis=0)

    def predict(self, X):
        """
        پیش‌بینی کلاس برای هر نقطه داده بر اساس نزدیکترین مرکز
        """
        y_pred = []

        # برای هر نقطه داده
        for x in X:
            # مقداردهی اولیه با فاصله بزرگ
            min_dist = float('inf')
            best_class = None

            # برای هر مرکز کلاس
            for c, centroid in self.centroids.items():
                # محاسبه فاصله تا مرکز
                dist = np.linalg.norm(x - centroid)

                # اگر این مرکز نزدیک‌تر است، به‌روزرسانی کن
                if dist < min_dist:
                    min_dist = dist
                    best_class = c

            # افزودن بهترین کلاس به پیش‌بینی‌ها
            y_pred.append(best_class)

        return np.array(y_pred)
```

Naive Bayes پیاده‌سازی طبقه‌بند خطی

```
class NaiveBayesClassifier:
    """
    که یک طبقه‌بند خطی است Naive Bayes پیاده‌سازی ساده طبقه‌بند
    """

    def __init__(self):
        self.class_priors = {}
        self.class_feature_means = {}
        self.class_feature_vars = {}

    def fit(self, X, y):
        """
        محاسبه احتمالات اولیه کلاس و توزیع ویژگی‌ها
        """
        # دریافت کلاس‌های یکتا
        classes = np.unique(y)

        # محاسبه احتمال اولیه برای هر کلاس
        for c in classes:
            # مجموع_تعداد / تعداد(c): P(c) = احتمال اولیه
            self.class_priors[c] = np.mean(y == c)

            # محاسبه میانگین و واریانس برای هر ویژگی
            X_c = X[y == c]
            self.class_feature_means[c] = np.mean(X_c, axis=0)
            self.class_feature_vars[c] = np.var(X_c, axis=0) + 1e-9 # افزودن مقدار کوچک برای جلوگیری از تقسیم بر صفر

    def predict(self, X):
        """
        Naive Bayes پیش‌بینی کلاس برای هر نقطه داده با استفاده از
        """
        y_pred = []

        # برای هر نقطه داده
        for x in X:
```

```
مقداردهی اولیه با مقدار بزرگ منفی (فضای لگاریتمی)
#
max_log_prob = -float('inf')
best_class = None

# برای هر کلاس
for c in self.class_priors:
    # شروع با لگاریتم احتمال اولیه
    log_prob = np.log(self.class_priors[c])

    # افزودن لگاریتم احتمال برای هر ویژگی
    for i in range(len(x)):
        # محاسبه احتمال مقدار ویژگی با توجه به کلاس
        # با استفاده از توزیع گوسی
        mean = self.class_feature_means[c][i]
        var = self.class_feature_vars[c][i]

        # لگاریتم احتمال مقدار ویژگی
        log_prob += -0.5 * np.log(2 * np.pi * var) - 0.5 * ((x[i] - mean) ** 2) / var

    # اگر این کلاس احتمال بالاتری دارد، به‌روزرسانی کن
    if log_prob > max_log_prob:
        max_log_prob = log_prob
        best_class = c

# افزودن بهترین کلاس به پیش‌بینی‌ها
y_pred.append(best_class)

return np.array(y_pred)
```

kNN پیاده‌سازی طبقه‌بند غیرخطی

```
class KNNClassifier:
    """
    نزدیکترین همسایه که یک طبقه‌بند غیرخطی است-k پیاده‌سازی ساده طبقه‌بند
    """

    def __init__(self, k=5):
        self.k = k
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        """
        (یک یادگیرنده تنبل است kNN) ذخیره داده‌های آموزشی
        """
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        """
        kNN پیش‌بینی کلاس برای هر نقطه داده با استفاده از
        """
        y_pred = []

        # برای هر نقطه داده
        for x in X:
            # محاسبه فاصله تا تمام نقاط آموزشی
            distances = np.array([np.linalg.norm(x - x_train) for x_train in self.X_train])

            # نزدیک‌ترین همسایه‌ها k دریافت شاخص‌های
            k_nearest_indices = np.argsort(distances)[:self.k]

            # نزدیک‌ترین همسایه‌ها k دریافت برچسب‌های
            k_nearest_labels = self.y_train[k_nearest_indices]

            # پیش‌بینی با رأی اکثریت
            counts = np.bincount(k_nearest_labels.astype(int))
            y_pred.append(np.argmax(counts))

        return np.array(y_pred)
```

تابع برای رسم مرز تصمیم

```
def plot_decision_boundary(classifier, X, y, title, ax=None):
    """
    رسم مرز تصمیم یک طبقه‌بند
    """

    if ax is None:
        ax = plt.gca()

    # ایجاد یک شبکه مش
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                          np.arange(y_min, y_max, 0.1))

    # پیش‌بینی برای هر نقطه در شبکه مش
    Z = classifier.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # نمایش مرز تصمیم و مناطق
    ax.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Paired)
    ax.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k')

    ax.set_title(title)
    ax.set_xlabel('Feature 1')
    ax.set_ylabel('Feature 2')
```

تابع برای مقایسه طبقه‌بندها روی داده‌های خطی

```
def compare_linear_classifiers():
    """
    مقایسه طبقه‌بندهای خطی و غیرخطی روی داده‌های خطی جداً با نویز
    """

    # تولید داده‌های خطی با نویز
    X, y = generate_linear_data(200, noise_level=0.1)
```

```
# تقسیم داده‌ها
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# ایجاد طبقه‌بندها
rocchio = RocchioClassifier()
naive_bayes = NaiveBayesClassifier()
knn = KNNClassifier(k=5)

# آموزش طبقه‌بندها
rocchio.fit(X_train, y_train)
naive_bayes.fit(X_train, y_train)
knn.fit(X_train, y_train)

# ارزیابی طبقه‌بندها
rocchio_acc = accuracy_score(y_test, rocchio.predict(X_test))
naive_bayes_acc = accuracy_score(y_test, naive_bayes.predict(X_test))
knn_acc = accuracy_score(y_test, knn.predict(X_test))

print(f"Rocchio Accuracy: {rocchio_acc:.3f}")
print(f"Naive Bayes Accuracy: {naive_bayes_acc:.3f}")
print(f"kNN Accuracy: {knn_acc:.3f}")

# نمایش مرزهای تصمیم
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

plot_decision_boundary(rocchio, X, y, 'Rocchio (Linear)', axes[0])
plot_decision_boundary(naive_bayes, X, y, 'Naive Bayes (Linear)', axes[1])
plot_decision_boundary(knn, X, y, 'kNN (Nonlinear)', axes[2])

plt.tight_layout()
plt.show()

# تابع برای مقایسه طبقه‌بندها روی داده‌های غیرخطی
def compare_nonlinear_classifiers():
    """
    مقایسه طبقه‌بندهای خطی و غیرخطی روی داده‌های غیرخطی جادش
    """
    # تولید داده‌های غیرخطی
    X, y = generate_nonlinear_data(200)

    # تقسیم داده‌ها
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    # ایجاد طبقه‌بندها
    rocchio = RocchioClassifier()
    naive_bayes = NaiveBayesClassifier()
    knn = KNNClassifier(k=5)

    # آموزش طبقه‌بندها
    rocchio.fit(X_train, y_train)
    naive_bayes.fit(X_train, y_train)
    knn.fit(X_train, y_train)

    # ارزیابی طبقه‌بندها
    rocchio_acc = accuracy_score(y_test, rocchio.predict(X_test))
    naive_bayes_acc = accuracy_score(y_test, naive_bayes.predict(X_test))
    knn_acc = accuracy_score(y_test, knn.predict(X_test))

    print(f"Rocchio Accuracy: {rocchio_acc:.3f}")
    print(f"Naive Bayes Accuracy: {naive_bayes_acc:.3f}")
    print(f"kNN Accuracy: {knn_acc:.3f}")

    # نمایش مرزهای تصمیم
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    plot_decision_boundary(rocchio, X, y, 'Rocchio (Linear)', axes[0])
    plot_decision_boundary(naive_bayes, X, y, 'Naive Bayes (Linear)', axes[1])
    plot_decision_boundary(knn, X, y, 'kNN (Nonlinear)', axes[2])

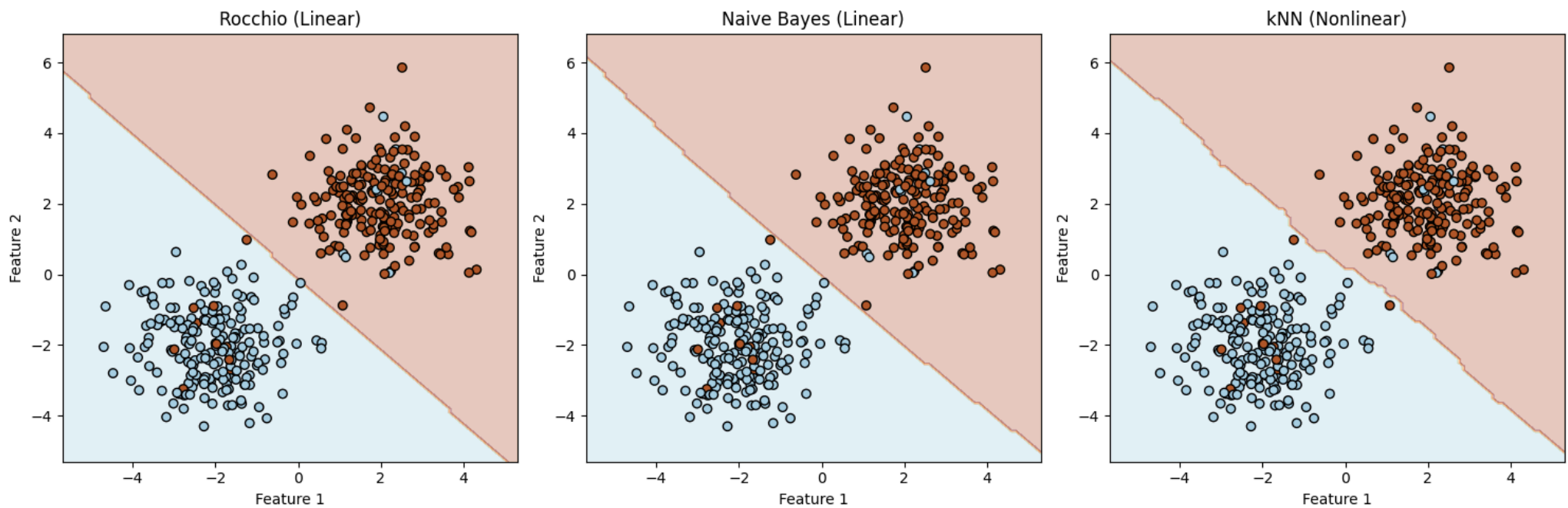
    plt.tight_layout()
    plt.show()

# اجرای مقایسه روی داده‌های خطی
print("Comparison of Classifiers on Linear Data with Noise:")
print("="*50)
compare_linear_classifiers()

# اجرای مقایسه روی داده‌های غیرخطی
print("\nComparison of Classifiers on Non-linear Data:")
print("="*50)
compare_nonlinear_classifiers()
```

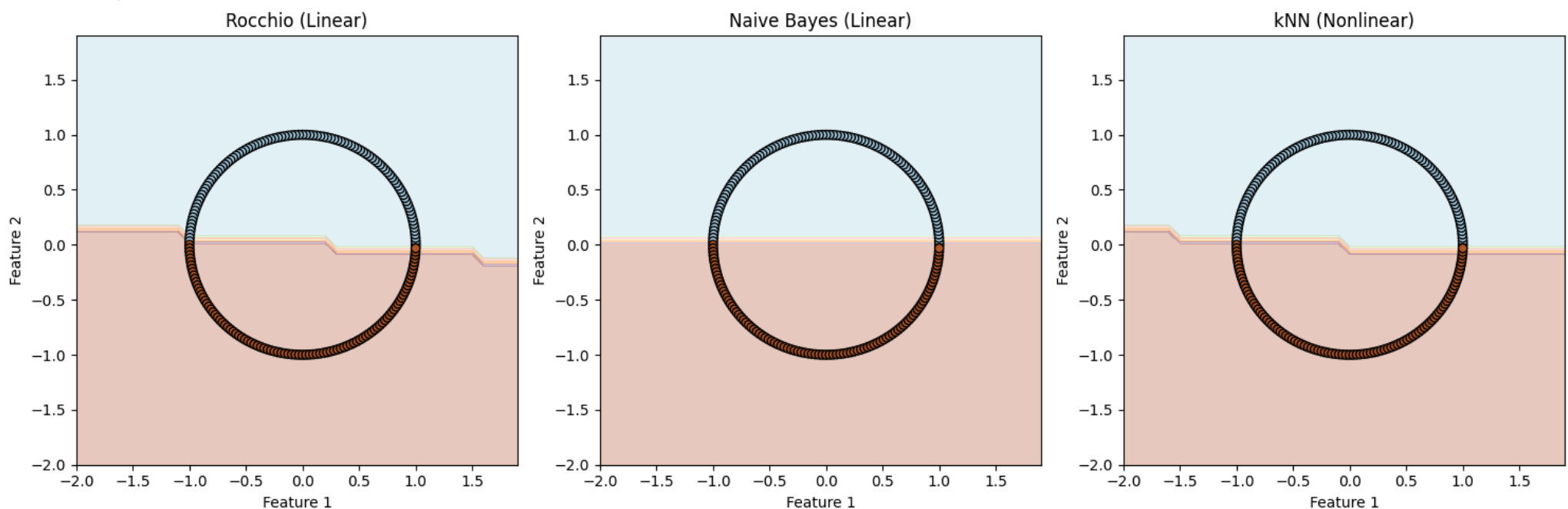
Comparison of Classifiers on Linear Data with Noise:
=====

Rocchio Accuracy:	0.933
Naive Bayes Accuracy:	0.933
kNN Accuracy:	0.925



Comparison of Classifiers on Non-linear Data:
=====

Rocchio Accuracy:	0.983
Naive Bayes Accuracy:	1.000
kNN Accuracy:	1.000



14.5 طبقه‌بندی با بیش از دو کلاس: از اخبار چندبعدی تا توصیه‌های شخصی‌سازی شده 🌐

🌐 چالش طبقه‌بندی خبر در گوگل نیوز: یک خبر می‌تواند چند برچسب داشته باشد؟

تصور کنید در تیم طبقه‌بندی محتوای گوگل نیوز کار می‌کنید و باید سیستمی طراحی کنید که اخبار را به دسته‌های مختلفی مانند "سیاست"، "اقتصاد"، "فناوری" و "ورزشی" تقسیم کند. آیا یک خبر می‌تواند همزمان به چند دسته تعلق داشته باشد؟

مثلاً، خبری درباره "قرارداد چند میلیارد دلاری شرکت اپل با دولت چین برای تولید آیفون" می‌تواند همزمان به دسته‌های "فناوری"، "اقتصاد" و "سیاست" تعلق داشته باشد. این یک **مسئله طبقه‌بندی هر-از-چند** است.

حالا تصور کنید وظیفه شما تشخیص زبان یک سند است. یک سند می‌تواند همزمان به چند زبان نوشته شده باشد؟ به ندرت. این یک **مسئله طبقه‌بندی یک-از-چند** است.

در مسائل طبقه‌بندی با بیش از دو کلاس، دو نوع اصلی وظیفه وجود دارد که بسته به ماهیت کلاس‌ها انتخاب می‌شود:

🏷️ **طبقه‌بندی هر-از-چند (Any-of Classification)**

در طبقه‌بندی هر-از-چند (که به آن چندبرچسبی یا چندمقداری نیز گفته می‌شود)، کلاس‌ها انحصاری متقابل نیستند. یک سند می‌تواند همزمان به چندین کلاس تعلق داشته باشد، یا به هیچ‌کدام از کلاس‌ها تعلق نداشته باشد.

مثال واقعی: در سیستم توصیه فیلم آمازون پرایم، یک فیلم می‌تواند همزمان برچسب‌های "اکشن"، "علمی-تخیلی" و "ماجراجویی" را داشته باشد. تصمیم برای یک برچسب، تصمیم برای دیگر را محدود نمی‌کند.

در طبقه‌بندی هر-از-چند، برای هر کلاس یک طبقه‌بند جداگانه ساخته می‌شود. برای هر کلاس، مجموعه آموزشی شامل اسناد متعلق به آن کلاس (برچسب‌های مثبت) و مکمل آن‌ها (برچسب‌های منفی) است. هنگام طبقه‌بندی یک سند جدید، هر طبقه‌بند به طور جداگانه اعمال می‌شود و تصمیم یک طبقه‌بند تأثیری بر تصمیمات طبقه‌بندهای دیگر ندارد.

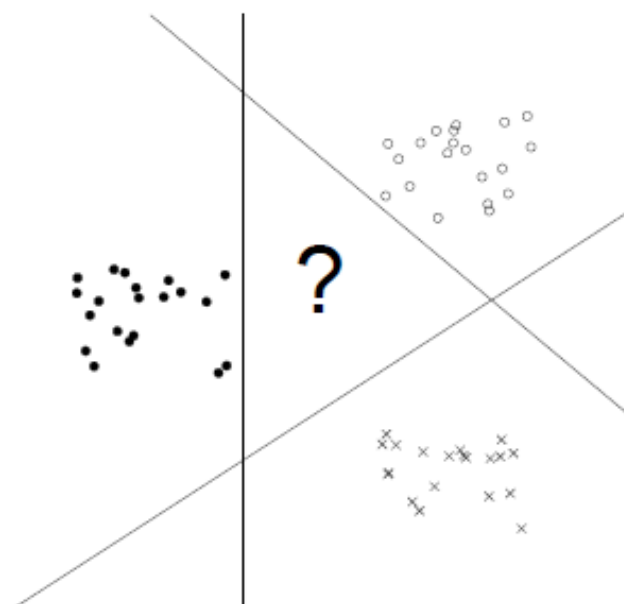
🔗 طبقه‌بندی یک-از-چند (One-of Classification)

در طبقه‌بندی یک-از-چند (که به آن چندوجهی، چندکلاسی یا تک‌برچسبی نیز گفته می‌شود)، کلاس‌ها انحصاری متقابل هستند. هر سند باید دقیقاً به یکی از کلاس‌ها تعلق داشته باشد.

مثال واقعی: در سیستم تشخیص زبان گوگل ترنسلیت، یک سند باید دقیقاً به یک زبان تعلق داشته باشد: انگلیسی، فرانسوی، اسپانیایی، و غیره. در این حالت، فرض یک-از-چند یک تقریب خوب است، زیرا اکثر متون فقط به یک زبان نوشته می‌شوند. مسائل واقعی یک-از-چند در طبقه‌بندی متن کمتر از مسائل هر-از-چند رایج هستند. با کلاس‌هایی مانند "بریتانیا"، "چین"، "طیور" یا "قهوه"، یک سند می‌تواند همزمان به چند موضوع مرتبط باشد - مانند وقتی که نخست‌وزیر بریتانیا برای صحبت درباره تجارت قهوه و طیور به چین سفر می‌کند.

📐 شکل 14.12: چالش تقسیم فضا با چند هایپرپلن

چالش اصلی در طبقه‌بندی یک-از-چند با استفاده از طبقه‌بندهای خطی این است که J هایپرپلن فضای برداری را به J ناحیه مجزا تقسیم نمی‌کنند. شکل 14.12 این مشکل را نشان می‌دهد: هایپرپلن‌ها یکدیگر را قطع می‌کنند و یک سند ممکن است همزمان به چندین ناحیه تعلق داشته باشد.



► **Figure 14.12** J hyperplanes do not divide space into J disjoint regions.

برای حل این مشکل، از روش ترکیبی استفاده می‌شود: ابتدا کلاس‌ها بر اساس امتیاز رتبه‌بندی می‌شوند و سپس کلاس برتر انتخاب می‌شود. از نظر هندسی، رتبه‌بندی می‌تواند بر اساس فاصله از J جداکننده خطی باشد. اسنادی که نزدیک به جداکننده یک کلاس هستند، احتمالاً اشتباه طبقه‌بندی می‌شوند، بنابراین هرچه فاصله از جداکننده بیشتر باشد، احتمال صحت تصمیم‌گیری مثبت بیشتر است.

📊 جدول 14.5: ماتریس سردرگمی برای تحلیل عملکرد طبقه‌بند

ماتریس سردرگمی ابزار مهمی برای تحلیل عملکرد طبقه‌بندهای چندکلاسی است. این ماتریس برای هر جفت کلاس (c_1, c_2) نشان می‌دهد که چند سند از کلاس c_1 به اشتباه به کلاس c_2 طبقه‌بندی شده‌اند.

جدول 14.5 یک ماتریس سردرگمی را برای طبقه‌بندی شش کلاس در مجموعه داده Reuters-21578 نشان می‌دهد. طبقه‌بند موفق می‌شود سه کلاس مالی (money-fx, trade, interest) را از سه کلاس کشاورزی (wheat, corn, grain) تشخیص دهد، اما در داخل هر یک از این دو گروه اشتباهات زیادی می‌کند.

assigned class	<i>money-fx</i>	<i>trade</i>	<i>interest</i>	<i>wheat</i>	<i>corn</i>	<i>grain</i>
true class						
<i>money-fx</i>	95	0	10	0	0	0
<i>trade</i>	1	1	90	0	1	0
<i>interest</i>	13	0	0	0	0	0
<i>wheat</i>	0	0	1	34	3	7
<i>corn</i>	1	0	2	13	26	5
<i>grain</i>	0	0	2	14	5	10

► **Table 14.5** A confusion matrix for Reuters-21578. For example, 14 documents from *grain* were incorrectly assigned to *wheat*. Adapted from Picca et al. (2006).

مثال واقعی: در سیستم طبقه‌بندی ایمیل گوگل، ماتریس سردرگمی می‌تواند نشان دهد که چرا برخی ایمیل‌های "تبلیغاتی" به اشتباه به دسته "اسپم" ارسال شده‌اند. با تحلیل این ماتریس، می‌توان ویژگی‌های جدیدی معرفی کرد که این دو دسته را بهتر از هم تمایز دهد.

ماتریس سردرگمی می‌تواند به شناسایی فرصت‌هایی برای بهبود دقت سیستم کمک کند. برای مثال، برای رسیدگی به دومین خطای بزرگ در جدول 14.5 (14 در سطر grain)، می‌توان تلاش کرد ویژگی‌هایی را معرفی کند که اسناد wheat را از اسناد grain متمایز کند.

در نهایت، انتخاب بین طبقه‌بندی هر-از-چند و یک-از-چند به ماهیت مسئله بستگی دارد. اگر کلاس‌ها انحصاری متقابل هستند، طبقه‌بندی یک-از-چند می‌تواند با اعمال یک محدودیت، دقت را افزایش دهد. اما اگر کلاس‌ها می‌توانند همپوشانی داشته باشند، طبقه‌بندی هر-از-چند انتخاب مناسب‌تری است.

```
In [12]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# مرحله ۱: تولید داده‌های مصنوعی برای ۳ کلاس
# هر کلاس در یک منطقه خاص از فضا متمرکز شده است تا مسئله طبقه‌بندی قابل مشاهده باشد
def generate_multi_class_data():
    """
    داده‌هایی برای سه کلاس تولید می‌کند.
    کلاس ۰: بالا-چپ (مثلاً "ورزشی")
    کلاس ۱: بالا-راست (مثلاً "سیاسی")
    کلاس ۲: پایین (مثلاً "فناوری")
    """
    np.random.seed(42)
    # کلاس ۰ (ورزشی)
    X0 = np.random.randn(50, 2) + np.array([-2, 2])
    y0 = np.zeros(50)

    # کلاس ۱ (سیاسی)
    X1 = np.random.randn(50, 2) + np.array([2, 2])
    y1 = np.ones(50)

    # کلاس ۲ (فناوری)
    X2 = np.random.randn(50, 2) + np.array([0, -2])
    y2 = np.full(50, 2)

    X = np.vstack([X0, X1, X2])
    y = np.hstack([y0, y1, y2])
    return X, y

# مرحله ۲: پیاده‌سازی یک طبقه‌بند خطی ساده (از صفر)
# این طبقه‌بند فقط دو کلاس را از هم جدا می‌کند (یکی در مقابل بقیه)
class SimpleLinearClassifier:
    def __init__(self, learning_rate=0.01, n_iterations=1000):
```

```
self.learning_rate = learning_rate
self.n_iterations = n_iterations
self.weights = None
self.bias = None

def fit(self, X, y_binary):
    """
    مدل را با داده‌های آموزشی آموزش می‌دهد.
    y_binary باید برچسب‌های 0 و 1 باشد.
    """

    n_samples, n_features = X.shape
    #مقداردهی اولیه وزن‌ها و بایاس
    self.weights = np.zeros(n_features)
    self.bias = 0

    # الگوریتم بهینه‌سازی ساده (گرادیان کاهشی)
    for _ in range(self.n_iterations):
        # مدل خطی:  $z = w \cdot x + b$ 
        linear_model = np.dot(X, self.weights) + self.bias
        # تابع فعال‌سازی ساده (پله‌ای)
        y_predicted = self._step_function(linear_model)

        # به‌روزرسانی وزن‌ها و بایاس
        dw = (1 / n_samples) * np.dot(X.T, (y_predicted - y_binary))
        db = (1 / n_samples) * np.sum(y_predicted - y_binary)
        self.weights -= self.learning_rate * dw
        self.bias -= self.learning_rate * db

def predict(self, X):
    """
    برچسب کلاس (0 یا 1) را برای داده‌های جدید پیش‌بینی می‌کند.
    """

    linear_model = np.dot(X, self.weights) + self.bias
    y_predicted = self._step_function(linear_model)
    return y_predicted

def decision_function(self, X):
    """
    امتیاز (فاصله از مرز تصمیم) را برای هر نقطه محاسبه می‌کند.
    این امتیاز برای مقایسه بین طبقه‌بندها در استراتژی یک-در-برابر-بقیه حیاتی است.
    """

    return np.dot(X, self.weights) + self.bias

def _step_function(self, x):
    """
    تابع پله‌ای ساده: اگر ورودی بزرگتر از 0 بود، 1 برمی‌گرداند، در غیر این صورت 0.
    """

    return np.where(x >= 0, 1, 0)

# مرحله ۳: ساخت طبقه‌بند چندکلاسی با استراتژی "یک-در-برابر-بقیه"
class OneVsRestClassifier:
    def __init__(self):
        self.classifiers = {}
        self.classes = None

    def fit(self, X, y):
        """
        برای هر کلاس، یک طبقه‌بند دوتایی آموزش می‌دهد.
        این پیاده‌سازی استراتژی "یک-از-چند" است که در مارکدون توضیح داده شد.
        """

        self.classes = np.unique(y)
        for cls in self.classes:
            print(f"Training classifier for class {int(cls)} vs rest...")
            # تبدیل برچسب‌ها به حالت دوتایی: 1 برای کلاس فعلی، 0 برای بقیه
            y_binary = np.where(y == cls, 1, 0)
            classifier = SimpleLinearClassifier()
            classifier.fit(X, y_binary)
            self.classifiers[cls] = classifier

    def predict(self, X):
        """
        برای هر نقطه داده، امتیاز آن را از تمام طبقه‌بندها دریافت کرده
        و کلاسی را انتخاب می‌کند که بالاترین امتیاز را داشته باشد.
        این روش رتبه‌بندی و انتخاب برتر، راحل مشکل تقسیم فضا توسط چند هاینرپلن است.
        """

        predictions = []
        for x in X:
            scores = {}
            for cls, classifier in self.classifiers.items():
                scores[cls] = classifier.decision_function(x)

            # کلاس با بالاترین امتیاز انتخاب می‌شود
            predicted_class = max(scores, key=scores.get)
            predictions.append(predicted_class)
        return np.array(predictions)

# مرحله ۴: اجرای کل فرآیند و نمایش نتایج

# تولید داده‌ها
X, y = generate_multi_class_data()

# آموزش طبقه‌بند چندکلاسی
ovr_classifier = OneVsRestClassifier()
ovr_classifier.fit(X, y)

# پیش‌بینی روی کل مجموعه داده برای تحلیل
y_pred = ovr_classifier.predict(X)

# مرحله ۵: محاسبه و نمایش ماتریس سردرگمی (از صفر)
def calculate_confusion_matrix(y_true, y_pred, classes):
    n_classes = len(classes)
    matrix = np.zeros((n_classes, n_classes), dtype=int)
    # ایجاد نگاشت از کلاس به اندیس ماتریس
```

```

class_to_index = {cls: i for i, cls in enumerate(classes)}

for true_label, pred_label in zip(y_true, y_pred):
    i = class_to_index[true_label]
    j = class_to_index[pred_label]
    matrix[i, j] += 1
return matrix

conf_matrix = calculate_confusion_matrix(y, y_pred, ovr_classifier.classes)

# چاپ نتایج
print("\n--- Confusion Matrix ---")
print("Rows: Actual Class, Columns: Predicted Class")
print(conf_matrix)

# نمایش ماتریس سردرگمی با نمودار حرارتی
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=[f"Pred {int(c)}" for c in ovr_classifier.classes],
            yticklabels=[f"Actual {int(c)}" for c in ovr_classifier.classes])
plt.title("Confusion Matrix (from scratch)")
plt.ylabel("Actual Class")
plt.xlabel("Predicted Class")
plt.show()

# نمایش مرزهای تصمیم
def plot_decision_boundaries_ovr(X, y, classifier):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                        np.arange(y_min, y_max, 0.1))

    # پیش‌بینی برای تمام نقاط در شبکه
    Z = classifier.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.4, cmap=plt.cm.Paired)
    scatter = plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k')
    plt.title("Decision Boundaries (One-vs-Rest from scratch)")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    # ایجاد لیجند (راهنما) برای کلاس‌ها
    legend1 = plt.legend(*scatter.legend_elements(),
                        title="Classes")
    plt.gca().add_artist(legend1)
    plt.show()

print("\n--- Plotting Decision Boundaries ---")
plot_decision_boundaries_ovr(X, y, ovr_classifier)

```

Training classifier for class 0 vs rest...

Training classifier for class 1 vs rest...

Training classifier for class 2 vs rest...

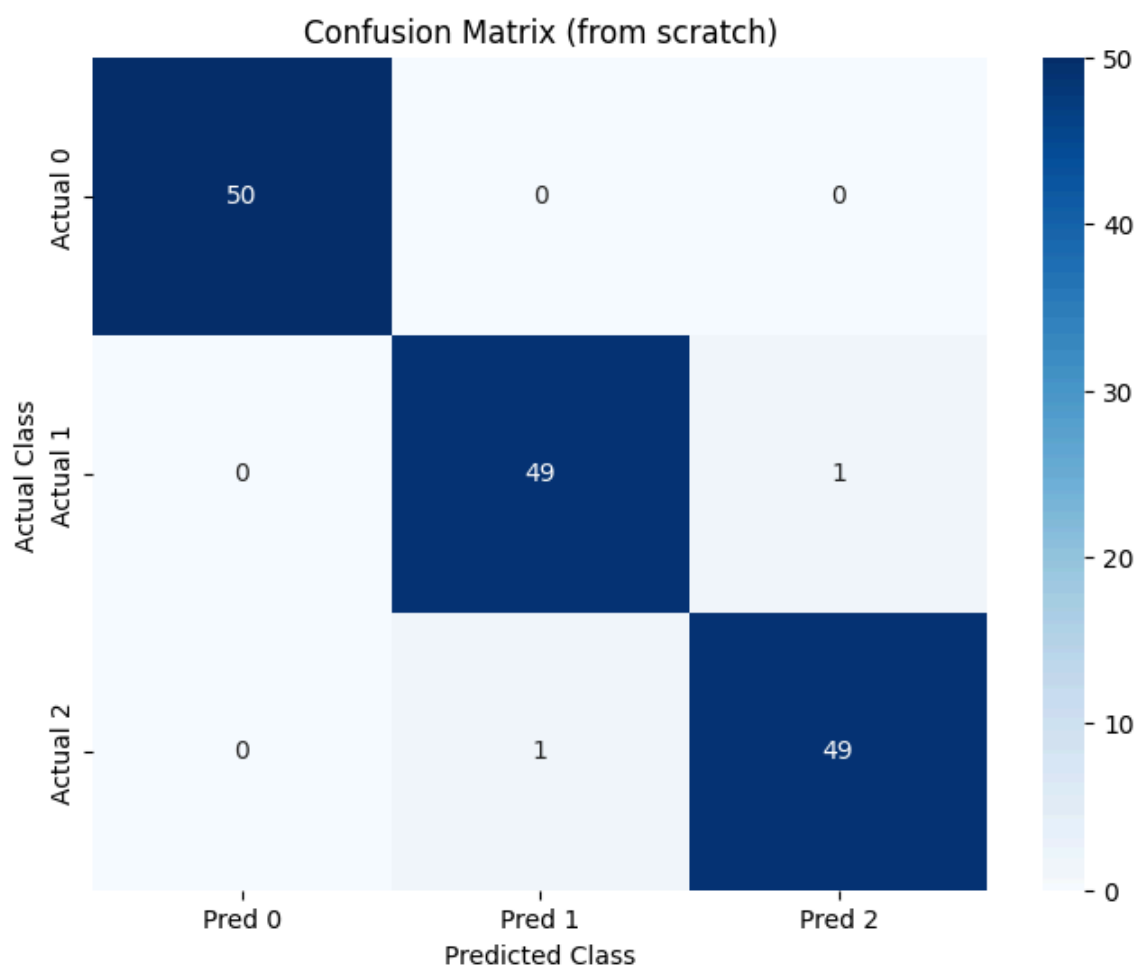
--- Confusion Matrix ---

Rows: Actual Class, Columns: Predicted Class

```

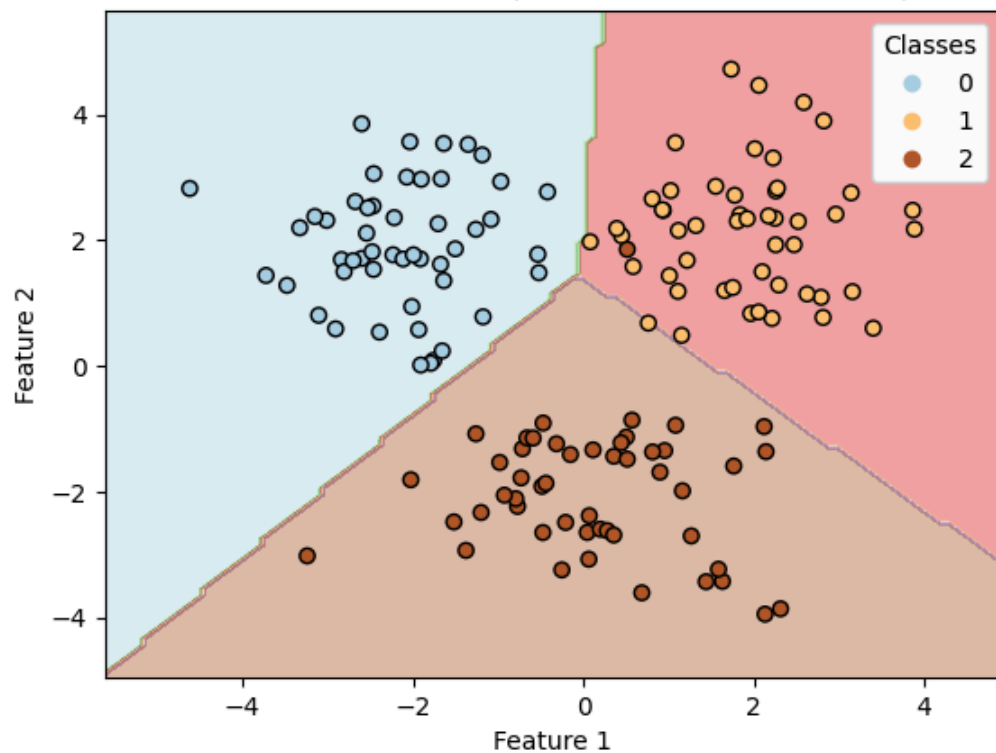
[[50  0  0]
 [ 0 49  1]
 [ 0  1 49]]

```



--- Plotting Decision Boundaries ---

Decision Boundaries (One-vs-Rest from scratch)



14.6 مبادله بایاس-واریانس: چرا هیچ الگوریتمی کامل نیست؟ 🏛️

🏠 چالش تشخیص پزشکی: آیا یک مدل ساده بهتر است یا یک مدل پیچیده؟

تصور کنید در حال ساخت یک سیستم هوشمند برای تشخیص یک بیماری نادر بر اساس علائم بیماران هستید. دو رویکرد مختلف وجود دارد:

- **مدل ساده (بایاس بالا، واریانس پایین):** یک مدل بسیار ساده که می‌گوید "هیچ‌کس بیمار نیست" چون بیماری نادر است. این مدل واریانس پایینی دارد (پیش‌بینی‌اش هرگز تغییر نمی‌کند) اما بایاس بسیار بالایی دارد (برای بیماران واقعی به طور مداوم اشتباه می‌کند). این مدل به دلیل سادگی بیش از حد، ناتوان است در الگوهای واقعی بیماری را تشخیص دهد.
- **مدل پیچیده (بایاس پایین، واریانس بالا):** یک مدل بسیار پیچیده که داده‌های آموزشی را به طور کامل به خاطر می‌سپارد، حتی شامل خطاهای اندازه‌گیری. این مدل بایاس پایینی روی داده‌های آموزشی دارد اما واریانس بالایی است. وقتی بیمار جدیدی با علائم کمی متفاوت وارد می‌شود، پیش‌بینی ممکن است به شدت اشتباه باشد. این مدل به دلیل پیچیدگی بیش از حد، به نویز در داده‌های آموزشی حساس است.

هدف در یادگیری ماشین، یافتن تعادل بین این دو رویکرد است: مدلی که به اندازه کافی پیچیده است تا الگوهای واقعی را تشخیص دهد، اما آنقدر پیچیده نیست که توسط نویز در داده‌های آموزشی گمراه شود.

در یادگیری ماشین، مفهوم **مبادله بایاس-واریانس** نشان می‌دهد که هیچ روش یادگیری به طور جهانی بهینه نیست. این مبادله توضیح می‌دهد که افزایش پیچیدگی یک طبقه‌بند معمولاً باعث کاهش بایاس (افزایش دقت) و افزایش واریانس (کاهش پایداری) می‌شود و برعکس.

📊 اندازه‌گیری خطا: خطای میانگین مربع (MSE)

در طبقه‌بندی، هدف ما بهینه‌سازی خطای طبقه‌بندی بر روی داده‌های جدید است، نه صرفاً به حداقل رساندن خطای روی داده‌های آموزشی. خطای طبقه‌بندی را با معیار **خطای میانگین مربع (MSE)** تعریف می‌کنیم:

$$\text{MSE}(\gamma) = \mathbb{E}_{\mathbf{d}}[\gamma(\mathbf{d}) - P(c|\mathbf{d})]^2$$

که در آن γ یک طبقه‌بند است، $\mathbf{d}$ یک سند، و $P(c|\mathbf{d})$ احتمال تعلق سند $\mathbf{d}$ به کلاس c است. MSE میانگین خطای طبقه‌بند را بر روی تمام اسناد اندازه‌گیری می‌کند و هم خطای ناشی از بایاس و هم خطای ناشی از واریانس را در نظر می‌گیرد.

مبادله بایاس-واریانس را می‌توان با مقایسه طبقه‌بندهای خطی و غیرخطی بهتر درک کرد:

- **طبقه‌بندهای خطی (مانند Rocchio و Naive Bayes):** این طبقه‌بندها **بایاس بالایی** دارند زیرا فقط می‌توانند یک نوع مرز کلاس (هایپرپلن) را مدل کنند. اگر مرز واقعی بین کلاس‌ها پیچیده باشد (مانند شکل 14.11)، این طبقه‌بندها به طور مداوم اشتباه طبقه‌بندی می‌کنند. با این حال، این طبقه‌بندها **واریانس پایینی** دارند زیرا مرز تصمیم آن‌ها پایدار است و با تغییرات جزئی در مجموعه آموزشی، به شدت تغییر نمی‌کند.
- **طبقه‌بندهای غیرخطی (مانند kNN):** این طبقه‌بندها **بایاس پایینی** دارند زیرا می‌توانند مرزهای تصمیم پیچیده‌ای ایجاد کنند که داده‌های آموزشی را به خوبی مدل می‌کنند. با این حال، این طبقه‌بندها **واریانس بالایی** دارند زیرا به اسناد نویزی حساس هستند. یک سند نویزی در مجموعه آموزشی می‌تواند مرز تصمیم را در ناحیه اطراف خود به شدت تغییر دهد.

مثال واقعی: در سیستم فیلتر اسپم جیمیل، یک طبقه‌بند خطی ممکن است به طور مداوم ایمیل‌های تبلیغاتی خاصی را اشتباه طبقه‌بندی کند (بایاس بالا)، اما پیش‌بینی‌های آن برای کاربران مختلف پایدار است (واریانس پایین). در مقابل، یک طبقه‌بند غیرخطی ممکن است بتواند اسپم‌های جدید و پیچیده‌تر را بهتر تشخیص دهد، اما ممکن است برای کاربران مختلف، رفتار متفاوتی از خود نشان دهد (واریانس بالا).

😬 چرا بسیاری از الگوریتم‌های برتر طبقه‌بندی متن، خطی هستند؟

شاید تعجب‌آور باشد که بسیاری از بهترین الگوریتم‌های طبقه‌بندی متن، خطی هستند. مبادله بایاس-واریانس به موفقیت آن‌ها کمک می‌کند:

- در طبقه‌بندی متن، معمولاً با ابعاد بسیار بالایی سروکار داریم (هزاران یا میلیون‌ها ویژگی، یکی برای هر کلمه).
- در فضاهای با ابعاد بالا، احتمال جدایی خطی به سرعت افزایش می‌یابد. این یعنی یک هایپرپلن ساده اغلب می‌تواند کلاس‌ها را به خوبی از هم جدا کند.
- بنابراین، "بایاس بالا" مدل‌های خطی در عمل در مسائل طبقه‌بندی متن مشکلی جدی ایجاد نمی‌کند.
- مزیت بزرگ مدل‌های خطی، "واریانس پایین" آن‌هاست که باعث می‌شود در برابر نویز مقاوم‌تر باشند و کمتر دچار بیش‌برازش (overfitting) شوند.

مثال واقعی: در سیستم طبقه‌بندی اسناد حقوقی شرکت‌هایی مانند Google یا Microsoft، با وجود پیچیدگی ظاهری زبان حقوقی، یک طبقه‌بند خطی ساده اغلب بهترین عملکرد را دارد. زیرا با هزاران ویژگی (کلمات)، یک مرز خطی می‌تواند اسناد مربوط به قراردادهای مختلف را به خوبی از هم جدا کند، در حالی که به نویز و تغییرات جزئی در زبان حساس نیست.

در نهایت، انتخاب بین یک روش یادگیری ساده (بایاس بالا، واریانس پایین) و یک روش پیچیده (بایاس پایین، واریانس بالا) به ماهیت مسئله، حجم داده‌ها و سطح نویز بستگی دارد. مبادله بایاس-واریانس به ما کمک می‌کند تا این تصمیم مهم را آگاهانه بگیریم و بهترین روش را برای مسئله خاص خود انتخاب کنیم.

In [14]: # نمایش مبادله بایاس-واریانس در طبقه‌بندی

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def generate_synthetic_data(n_samples=1000, noise_level=0.2):
```

```

"""
تولید داده‌های مصنوعی برای نمایش مبادله بایاس-واریانس
"""

np.random.seed(42)

# ایجاد دو کلاس با توزیع‌های متفاوت
X_class1 = np.random.randn(n_samples//2, 2) + np.array([2, 2])
X_class2 = np.random.randn(n_samples//2, 2) + np.array([-2, -2])

y = np.hstack([np.ones(n_samples//2), np.zeros(n_samples//2)])

# ترکیب داده‌ها
X = np.vstack([X_class1, X_class2])

# افزودن نویز به درصدی از داده‌ها
n_noise = int(noise_level * n_samples)
noise_indices = np.random.choice(len(X), n_noise, replace=False)

# معکوس کردن برچسب برخی از نقاط نویز
y[noise_indices] = 1 - y[noise_indices]

return X, y

def fit_classifiers(X_train, y_train):
    """
    آموزش چندین طبقه‌بند با پیچیدگی‌های متفاوت
    """
    # طبقه‌بند خطی (زیاد بایاس، کم واریانس)
    linear = LogisticRegression(max_iter=1000)
    linear.fit(X_train, y_train)

    # طبقه‌بند kNN (زیاد واریانس، کم بایاس)
    knn = KNeighborsClassifier(n_neighbors=5)
    knn.fit(X_train, y_train)

    # طبقه‌بند SVM (کم بایاس، زیاد واریانس)
    svm = SVC(kernel='rbf', C=1.0, gamma='scale')
    svm.fit(X_train, y_train)

    return {'linear': linear, 'knn': knn, 'svm': svm}

def evaluate_classifiers(classifiers, X_test, y_test):
    """
    ارزیابی طبقه‌بندها با محاسبه دقت
    """
    results = {}

    for name, clf in classifiers.items():
        y_pred = clf.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)
        results[name] = accuracy

    return results

def plot_decision_boundary(classifier, X, y, title, ax=None):
    """
    نمایش مرز تصمیم برای یک طبقه‌بند
    """
    if ax is None:
        ax = plt.gca()

    # ایجاد شبکه برای نمایش مرز
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                          np.arange(y_min, y_max, 0.1))

    # پیش‌بینی تمام نقاط شبکه
    Z = classifier.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # نمایش مرز تصمیم و مناطق
    ax.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Paired)
    ax.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k')

    ax.set_title(title)
    ax.set_xlabel('Feature 1')
    ax.set_ylabel('Feature 2')

    return ax

def calculate_bias_variance(classifiers, X_train, y_train):
    """
    محاسبه بایاس و واریانس برای طبقه‌بندها
    """
    results = {}

    for name, clf in classifiers.items():
        # استفاده از اعتبارسنجی متقابل برای تخمین بایاس و واریانس
        from sklearn.model_selection import cross_val_score

        # محاسبه بایاس به عنوان 1 - میانگین دقت در اعتبارسنجی متقابل
        scores = cross_val_score(clf, X_train, y_train, cv=5)
        bias = 1 - np.mean(scores)

        # محاسبه واریانس به عنوان واریانس دقت در اعتبارسنجی متقابل
        variance = np.var(scores)

        results[name] = {'bias': bias, 'variance': variance}

    return results

# مثال کاربردی

```

```
print("Generating synthetic data to show bias-variance tradeoff:")
print("="*60)

# تولید داده‌های آموزشی و آزمون‌ی
X, y = generate_synthetic_data(n_samples=1000, noise_level=0.2)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# آموزش طبقه‌بندها
classifiers = fit_classifiers(X_train, y_train)

# محاسبه بایاس و واریانس
bias_variance = calculate_bias_variance(classifiers, X_train, y_train)
print("\nBias and Variance of classifiers:")
for name, metrics in bias_variance.items():
    print(f"{name}: Bias={metrics['bias']:.4f}, Variance={metrics['variance']:.4f}")

# ارزیابی روی داده‌های آزمون
results = evaluate_classifiers(classifiers, X_test, y_test)
print("\nAccuracy of classifiers on test data:")
for name, accuracy in results.items():
    print(f"{name}: {accuracy:.4f}")

# نمایش مرزهای تصمیم
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

plot_decision_boundary(classifiers['linear'], X_test, y_test, 'Linear Classifier (High Bias, Low Variance)', axes[0])
plot_decision_boundary(classifiers['knn'], X_test, y_test, 'kNN Classifier (Low Bias, High Variance)', axes[1])
plot_decision_boundary(classifiers['svm'], X_test, y_test, 'SVM Classifier (Low Bias, High Variance)', axes[2])

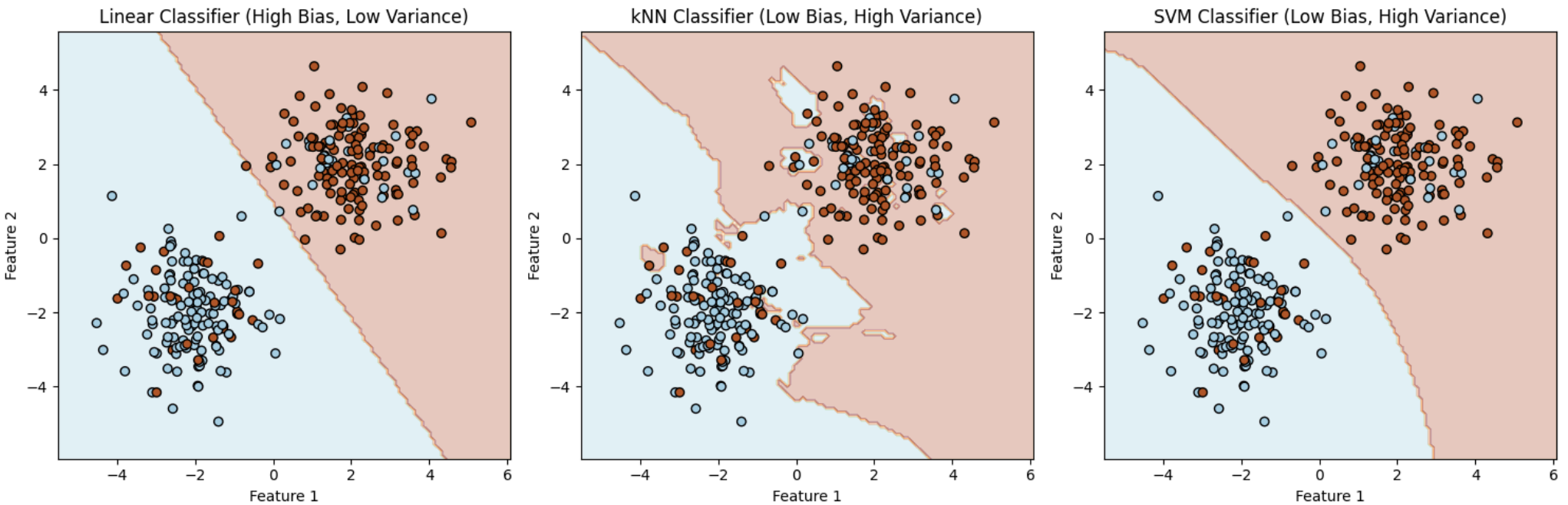
plt.tight_layout()
plt.show()

# تحلیل اصلاح‌شده بر اساس نتایج واقعی
print("\nتحلیل:")
print("طبقه‌بند خطی: بایاس بالا اما واریانس کم، دقت قابل قبول -")
print("بایاس و واریانس متوسط، دقت کمی پایین‌تر: kNN طبقه‌بند -")
print("بایاس و واریانس متوسط، بالاترین دقت: SVM طبقه‌بند -")
print("با وجود پیچیدگی بیشتر، بهترین عملکرد را در این مجموعه داده دارد SVM این مثال نشان می‌دهد که طبقه‌بند")
```

Generating synthetic data to show bias-variance tradeoff:
=====

Bias and Variance of classifiers:
linear: Bias=0.2071, Variance=0.0002
knn: Bias=0.2543, Variance=0.0004
svm: Bias=0.2043, Variance=0.0002

Accuracy of classifiers on test data:
linear: 0.8033
knn: 0.7800
svm: 0.8067



تحلیل:
- طبقه‌بند خطی: بایاس بالا اما واریانس کم، دقت قابل قبول
- بایاس و واریانس متوسط، دقت کمی پایین‌تر: kNN طبقه‌بند
- بایاس و واریانس متوسط، بالاترین دقت: SVM طبقه‌بند

با وجود پیچیدگی بیشتر، بهترین عملکرد را در این مجموعه داده دارد SVM این مثال نشان می‌دهد که طبقه‌بند

جمع‌بندی فصل ۱۴: طبقه‌بندی فضای برداری

چگونه می‌توانیم اسناد را بر اساس شباهتشان در یک فضای چندبعدی دسته‌بندی کنیم؟ فصل ۱۴ ما را به دنیای **طبقه‌بندی فضای برداری** می‌برد؛ رویکردی که اسناد را به صورت بردارهایی در یک فضای چندبعدی نمایش می‌دهد و بر اساس فاصله یا شباهتشان تصمیم‌گیری می‌کند. این فصل نشان می‌دهد که چگونه می‌توان با استفاده از مفاهیم هندسی و جبری، مرزهای تصمیم‌گیری را بین کلاس‌های مختلف ترسیم کرد و اسناد جدید را به درستی طبقه‌بندی نمود.

- فرض **تداوم (Contiguity Hypothesis)** را به عنوان اساس طبقه‌بندی فضای برداری معرفی کردیم. این فرض می‌گوید اسناد مربوط به یک کلاس، ناحیه‌های پیوسته‌ای را در فضای برداری تشکیل می‌دهند که با ناحیه‌های کلاس‌های دیگر همپوشانی ندارند.

- الگوریتم ****روچیو (Rocchio)**** را به عنوان روشی برای طبقه‌بندی بر اساس ****مراکز (Centroids)**** بررسی کردیم. در این روش، هر کلاس با مرکز جرمی اسنادش نمایش داده می‌شود و مرزهای تصمیم‌گیری به صورت ابرصفحه‌هایی بین این مراکز ترسیم می‌گردند.
- الگوریتم **k**** نزدیک‌ترین همسایه (**kNN**) را به عنوان روشی برای طبقه‌بندی محلی بررسی کردیم. در این روش، هر سند جدید بر اساس اکثریت کلاس **k** همسایه نزدیک‌ترش طبقه‌بندی می‌شود و مرزهای تصمیم‌گیری به صورت قطعه‌هایی از ****تسلیم‌بندی ورونویی (Voronoi Tessellation)**** شکل می‌گیرند.
- تفاوت بین ****طبقه‌بندهای خطی و غیرخطی**** را بررسی کردیم. طبقه‌بندهای خطی مانند روچیو و بیز ساده، مرزهای تصمیم‌گیری خطی (ابرصفحه‌ها) ایجاد می‌کنند، در حالی که روش‌هایی مانند **kNN** می‌توانند مرزهای تصمیم‌گیری پیچیده‌تر و غیرخطی ایجاد کنند.
- روش‌های طبقه‌بندی برای مسائل با ****بیش از دو کلاس**** را بررسی کردیم: مسائل ****یکی-از (One-of)**** که در آن هر سند باید دقیقاً به یک کلاس تعلق گیرد، و مسائل ****هر-از (Any-of)**** که در آن یک سند می‌تواند به چندین کلاس تعلق گیرد.
- مفهوم ****مبادله-واریانس (Bias-Variance Tradeoff)**** را به عنوان یکی از بنیادی‌ترین مفاهیم در یادگیری ماشین معرفی کردیم. این مفهوم توضیح می‌دهد که چرا هیچ روش طبقه‌بندی به طور جهانی بر دیگری برتری ندارد و چگونه باید بین پیچیدگی مدل و حساسیت به داده‌های آموزشی تعادل برقرار کرد.

بیاید این مفاهیم را با چند مثال واقعی و کاربردی زنده کنیم:

- ♦ **دسته‌بندی اخبار در خبرگزاری‌ها:** تصور کنید در یک خبرگزاری بزرگ مانند Reuters، می‌خواهیم اخبار را به دسته‌هایی مانند «سیاسی»، «ورزشی»، «اقتصادی» و «فناوری» تقسیم کنیم. با استفاده از مدل فضای برداری، هر خبر به صورت یک بردار با وزن‌های **tf-idf** نمایش داده می‌شود. اخبار مربوط به «سیاسی» معمولاً کلماتی مانند «پارلمان»، «دیپلماسی»، «انتخابات» را دارند، در حالی که اخبار «ورزشی» حاوی کلماتی مانند «فوتبال»، «تیم»، «بازی» هستند. این اسناد در فضای برداری خوشه‌های مجزایی را تشکیل می‌دهند که می‌توان با ابرصفحه‌هایی بین آن‌ها مرزهای تصمیم‌گیری ترسیم کرد.
- ♦ **فیلتر کردن نظرات مشتریان در سایت‌های تجارت الکترونیک:** سایت‌هایی مانند Amazon از طبقه‌بندی فضای برداری برای تحلیل نظرات مشتریان استفاده می‌کنند. با استفاده از الگوریتم روچیو، می‌توان مرکز هر دسته از نظرات (مثبت، منفی، خنثی) را محاسبه کرد. برای مثال، نظرات مثبت معمولاً حاوی کلماتی مانند «عالی»، «پیشنهاد می‌کنم»، «کیفیت بالا» هستند، در حالی که نظرات منفی حاوی کلماتی مانند «بد»، «نامناسب»، «خراب» هستند. وقتی نظر جدیدی ثبت می‌شود، سیستم فاصله آن را از مراکز محاسبه شده محاسبه کرده و به نزدیک‌ترین کلاس اختصاص می‌دهد.
- ♦ **تشخیص زبان سند با استفاده از kNN:** در یک سازمان بین‌المللی مانند اتحادیه اروپا، هزاران سند به زبان‌های مختلف دریافت می‌شود. برای تشخیص خودکار زبان هر سند، می‌توان از الگوریتم **kNN** استفاده کرد. هر سند به صورت یک بردار از فراوانی کلمات نمایش داده می‌شود. وقتی سند جدیدی می‌رسد، سیستم **k** همسایه نزدیک‌ترین آن را در مجموعه آموزشی پیدا کرده و زبان اکثریت آن همسایه‌ها را به عنوان زبان سند جدید تشخیص می‌دهد. این روش به خصوص برای زبان‌هایی که شباهت زیادی دارند (مانند هلندی و نروژی) بسیار مؤثر است.
- ♦ **دسته‌بندی چندبرچسبی در شبکه‌های اجتماعی:** در پلتفرم‌هایی مانند Instagram یا Twitter، یک پست می‌تواند به چندین برچسب (هشتگ) تعلق گیرد. این یک مسئله «هر-از» است که در آن یک سند (پست) می‌تواند به چندین کلاس (برچسب) تعلق گیرد. برای مثال، یک عکس از غذا در رستوران می‌تواند همزمان به برچسب‌های «#غذا»، «#رستوران»، «#زیبا» و «#سفر» تعلق گیرد. در این موارد، برای هر برچسب یک طبقه‌بند جداگانه آموزش داده می‌شود و در زمان تشخیص، هر طبقه‌بند به طور مستقل تصمیم‌گیری می‌کند.
- ♦ **مبادله-واریانس در سیستم‌های تشخیص پزشکی:** در سیستم‌های تشخیص پزشکی، باید بین پیچیدگی مدل و حساسیت به داده‌های آموزشی تعادل برقرار کرد. یک مدل با واریانس بالا (مانند **kNN** با **k** کوچک) ممکن است داده‌های آموزشی را به

خوبی به خاطر بسپارد، اما در تشخیص بیماران جدید عملکرد ضعیفی داشته باشد. از سوی دیگر، یک مدل با مبادله بالا (مانند یک طبقه‌بند خطی ساده) ممکن است نتواند الگوهای پیچیده بیماری را تشخیص دهد. در عمل، پزشکان و محققان باید با توجه به مقدار داده‌های در دسترس و پیچیدگی مسئله، مدل مناسبی را انتخاب کنند که مبادله و واریانس متعادل داشته باشد.

این فصل نشان داد که چگونه می‌توان با استفاده از نمایش فضای برداری، اسناد را به صورت مؤثری طبقه‌بندی کرد. روش‌هایی مانند روجیو و kNN، با وجود تفاوت‌هایشان، هر دو در کاربردهای واقعی بسیار مفید هستند. درک مفهوم مبادله-واریانس به ما کمک می‌کند تا بدانیم چرا هیچ الگوریتمی به طور جهانی بر دیگری برتری ندارد و انتخاب روش مناسب به ویژگی‌های مسئله خاص بستگی دارد. در فصل‌های بعدی، با روش‌های پیشرفته‌تری مانند ماشین‌های بردار پشتیبان آشنا خواهیم شد که این مفاهیم را به سطح بالاتری ارتقا می‌دهند.

End of Chapter Exercises

مسئله 14.1: نمایش برداری اسناد (Vector Representation of Documents)

توضیح دهید که چگونه یک سند متنی خام به یک بردار در فضای n -بعدی تبدیل می‌شود؟ نقش واژه‌نامه (Vocabulary) در تعیین ابعاد این فضا چیست؟

مسئله 14.2: محاسبه شباهت کسینوسی (Cosine Similarity Calculation)

فرض کنید دو سند متنی پس از پیش‌پردازش به بردارهای $d_1 = (1, 0, 2, 1)$ و $d_2 = (0, 1, 1, 2)$ در یک فضای 4 بعدی تبدیل شده‌اند. شباهت کسینوسی (Cosine Similarity) بین این دو سند را به صورت دستی محاسبه کنید.

مسئله 14.3: محاسبه مرکز ثقل (Centroid Calculation in Rocchio)

در الگوریتم Rocchio، فرمول ریاضی محاسبه بردار مرکز ثقل (Centroid) یک کلاس را بنویسید و به زبان ساده توضیح دهید که این بردار از نظر هندسی چه نقطه‌ای را در فضای بردارهای اسناد نشان می‌دهد.

مسئله 14.4: مرز تصمیم‌گیری روکیو (Rocchio Decision Boundary)

مرز تصمیم‌گیری (Decision Boundary) بین دو کلاس در الگوریتم Rocchio چگونه رسم می‌شود؟ توضیح دهید که چرا این مرز همیشه یک Hyperplane (ابرصفحه) است.

مسئله 14.5: مفهوم پایه همسایه‌های نزدیک (Basic Concept of kNN)

الگوریتم k Nearest Neighbor برای دسته‌بندی یک سند جدید دقیقاً چه مرحله‌ای را طی می‌کند؟ نقش پارامتر k در این الگوریتم چیست؟

مسئله 14.6: شکستن گره در دسته‌بندی (Tie-Breaking in kNN)

اگر در الگوریتم **kNN** مقدار **k=4** باشد و دو همسایه متعلق به کلاس **A** و دو همسایه متعلق به کلاس **B** باشند، سیستم با یک گره (**Tie**) مواجه می‌شود. دو روش رایج برای حل این مشکل در بازیابی اطلاعات را نام ببرید.

مسئله 14.7: تعریف طبقه‌بند خطی (Definition of Linear Classifier)

یک طبقه‌بند خطی (**Linear Classifier**) در یادگیری ماشین و بازیابی اطلاعات به چه معناست؟ آیا الگوریتم‌های **kNN** و **Rocchio** جزو طبقه‌بندهای خطی محسوب می‌شوند یا غیرخطی؟

مسئله 14.8: استراتژی‌های چندکلاسه (Multiclass Strategies)

تفاوت اساسی بین دو روش دسته‌بندی **One-of** (تک‌برچسبی) و **Any-of** (چندبرچسبی) چیست؟ برای یک سیستم خبری که هر خبر می‌تواند همزمان برچسب‌های «ورزشی» و «اقتصادی» داشته باشد، کدام رویکرد مناسب‌تر است؟

مسئله 14.9: تعریف بایاس و واریانس (Definition of Bias and Variance)

در زمینه دسته‌بندی متون، مفاهیم **Bias** (بایاس) و **Variance** (واریانس) را تعریف کنید. خطای سیستم (**System Error**) چگونه از ترکیب این دو به وجود می‌آید؟

مسئله 14.10: وزن‌دهی کلمات (Term Weighting and TF-IDF)

چرا در ایجاد فضای برداری اسناد (**Vector Space Model**) برای دسته‌بندی، استفاده از وزن‌دهی **TF-IDF** به جای شمارش ساده کلمات (**Term Frequency**) باعث بهبود چشمگیر عملکرد مدل‌های **Rocchio** و **kNN** می‌شود؟

مسئله 14.11: چالش کلاس‌های غیرمحدب (The Challenge of Non-convex Classes)

شما در حال دسته‌بندی اخبار هستید. کلاس «اخبار حوادث» شامل خبرهای تصادفات رانندگی، آتش‌سوزی و سرقت مسلحانه است که دایره‌ها کاملاً متفاوتی دارند. تحلیل کنید که چرا الگوریتم **Rocchio** در دسته‌بندی چنین کلاس‌های **Non-convex** (غیرمحدب) دچار مشکل جدی می‌شود؟

مسئله 14.12: تحلیل پیچیدگی زمانی (Time Complexity Analysis)

اگر مجموعه داده آموزشی ما شامل **|D|** سند و واژه‌نامه ما شامل **|V|** کلمه باشد، پیچیدگی زمانی فاز آموزش (**Training**) و فاز آزمایش (**Testing**) را برای دو الگوریتم **Rocchio** و **kNN** مقایسه کنید. چرا می‌گوییم **kNN** یک الگوریتم تنبلی (**Lazy**) است؟

مسئله 14.13: تفکیک‌پذیری خطی داده‌ها (Linear Separability of Data)

استدلال کنید که چرا در فضاهای با ابعاد بسیار بالا (مانند فضای بردارهای متنی که ابعاد آن برابر تعداد کلمات واژه‌نامه است)، احتمال اینکه داده‌ها به صورت **Linearly Separable** (تفکیک‌پذیر خطی) باشند، بسیار بیشتر از فضاهای دو یا سه بعدی است؟

مسئله 14.14: دسته‌بندی انحصاری در برابر چندگانه (Any-of vs One-of Classification)

می‌خواهیم 5 کلاس مختلف داشته باشیم. اگر از روش **One-vs-All** برای ساختن طبقه‌بند استفاده کنیم، دقیقاً به چند طبقه‌بند باینری (دوکلاسه) نیاز داریم؟ نحوه ترکیب نتایج این طبقه‌بندها برای رسیدن به یک تصمیم نهایی در حالت **One-of** چگونه است؟

مسئله 14.15: تاثیر پارامتر k بر واریانس (Impact of k on Variance)

نمودار **Trade-off** بین **Bias** و **Variance** را برای الگوریتم **kNN** رسم کنید و نشان دهید که کاهش مقدار **k** (مثلاً از 15 به 1) چگونه باعث افزایش **واریانس** و احتمال **Overfitting** می‌شود.

مسئله 14.16: بروزرسانی پویای مرکز ثقل (Dynamic Centroid Updating)

در یک موتور جستجو، روزانه هزاران سند جدید به کلاس «تکنولوژی» اضافه می‌شود. آیا برای بروزرسانی بردار **Centroid** در الگوریتم **Rocchio** نیاز است هر روز تمام اسناد قبلی را دوباره پردازش کنیم؟ پاسخ منفی است. با نگهداری تعداد اسناد پیشین (N) و بردار مرکز ثقل فعلی ($\vec{C}_{old}$)، فرمول بازگشتی برای محاسبه مرکز ثقل جدید ($\vec{C}_{new}$) پس از اضافه شدن M سند جدید به صورت زیر است:

$$\vec{C}_{new} = \frac{N \cdot \vec{C}_{old} + \sum_{i=1}^M \vec{d}_i}{N + M}$$

مسئله 14.17: همسایگی وزن‌دار کسینوسی (Cosine-Weighted kNN)

در مدل استاندارد **kNN**، رای تمام **k** همسایه وزن برابری دارد. اگر بخواهیم یک سیستم همسایگی وزن‌دار (**Weighted kNN**) طراحی کنیم که در آن همسایه‌های نزدیک‌تر تاثیر بیشتری داشته باشند، چگونه می‌توان از **شباهت کسینوسی** (**Cosine Similarity**) به عنوان وزن آرا استفاده کرد؟ فرمول تصمیم‌گیری آن را بنویسید.

مسئله 14.18: مقایسه مرزهای تصمیم (Comparing Decision Boundaries)

تفاوت ظاهری مرزهای تصمیم در **kNN** و **Rocchio** چیست؟ توضیح دهید که چرا مرز تصمیم در **kNN** (**Local**) (محلی) و در **Rocchio** (**Global**) (سراسری) است.

مسئله 14.19: مشکل کم‌برازش در روکیو (Underfitting in Rocchio)

الگوریتم **Rocchio** به دلیل تکیه بر یک «مرز تصمیم خطی» و استفاده از تنها یک نقطه مرکزی (**Centroid**) برای هر کلاس، دارای **بایاس بالا** (**High Bias**) است.

مثال: تصور کنید در یک فضای دوبعدی، داده‌های کلاس **A** به شکل یک «هلال» (C-shape) پراکنده شده‌اند و کلاس **B** در فضای خالی میان هلال قرار دارد. در این حالت، مرکز ثقل کلاس **A** در فضای خالی (جایی که کلاس **B** هست) قرار می‌گیرد. در نتیجه، **Rocchio** نمی‌تواند این ساختار پیچیده (غیرمحدب) را به درستی تفکیک کند و به دلیل سادگی بیش از حد مدل، دچار **کم‌برازش** (**Underfitting**) می‌شود.

مسئله 14.20: مقدمه‌ای بر نفرین ابعاد (Intro to Curse of Dimensionality)

در الگوریتم **kNN**، با افزایش شدید ابعاد فضای متنی (مثلاً یک میلیون کلمه یکتا)، پدیده‌ای به نام **نفرین ابعاد** رخ می‌دهد که در آن فاصله بین نزدیک‌ترین و دورترین اسناد به یکدیگر نزدیک می‌شود. توضیح دهید که چگونه این همگرایی فواصل باعث کاهش قدرت تفکیک و کارایی معیارهای شباهت در فضاها با ابعاد بالا شده و چه تأثیری بر دقت دسته‌بندی **kNN** می‌گذارد.

مسئله 14.21: بهینه‌سازی جستجو با نمایه وارونه (Search Optimization with Inverted Index) ★

سیستم **kNN** شما در یک دیتابیس با 10 میلیون سند اجرا می‌شود. مقایسه بردار کوئری با تک تک اسناد $O(|D|)$ زمان می‌برد که فاجعه‌بار است. یک شبه‌کد کامل بنویسید که نشان دهد چگونه با استفاده از ساختار **نمایه وارونه** (**Inverted Index**)، می‌توان تنها اسنادی را برای محاسبه شباهت بازیابی کرد که حداقل یک کلمه مشترک با سند ورودی (کوئری) دارند. پیچیدگی زمانی الگوریتم شما چقدر کاهش می‌یابد؟

مسئله 14.22: معماری سیستم و مبادله بایاس-واریانس (System Architecture and Bias-Variance Tradeoff) ★

سیستم تحلیل احساسات (Sentiment Analysis) دیجی‌کالا روی نظرات کوتاه کاربران پیاده‌سازی شده است. داده‌های آموزشی به شدت نویزی هستند (کاربران از کلمات طعنه‌آمیز یا غلط املایی زیاد استفاده می‌کنند).
الف) اگر از **1-NN** استفاده کنید، سیستم چه رفتاری در برابر این نویزها نشان می‌دهد (از منظر **Bias/Variance**)؟
ب) به عنوان معمار سیستم، استفاده از **Rocchio** را در این شرایط نویزی پیشنهاد می‌کنید یا یک **kNN** با پارامتر **k** بزرگ؟ چرا؟

مسئله 14.23: روکیو در توزیع‌های چندوجهی (Rocchio in Multimodal Distributions) ★

فرض کنید اسناد کلاس «سیاسی» در دو کلاستر کاملاً جداگانه در فضای برداری قرار گرفته‌اند (مثلاً سیاست داخلی و سیاست خارجی) و بین این دو کلاستر، اسناد کلاس «اقتصادی» قرار دارند. با رسم یک نمودار هندسی اثبات کنید که الگوریتم **Rocchio** حتماً در دسته‌بندی اسنادی که در مرکز این دو کلاستر قرار دارند، دچار خطای فاحش می‌شود و آن‌ها را به اشتباه «اقتصادی» دسته‌بندی می‌کند.

مسئله 14.24: تحلیل هندسی موزائیک‌بندی ورونوی (Geometrical Analysis of Voronoi Tessellation) ★

در فضای دو بعدی، 3 سند از کلاس **A** و 3 سند از کلاس **B** داریم. مرز تصمیم **1-NN** را با رسم دقیق موزائیک‌بندی ورونوی (**Voronoi Tessellation**) به دست آورید. سپس نشان دهید که اگر **k** را به 3 افزایش دهیم، این مرزهای چندضلعی چگونه نرم (Smooth) شده و تغییر ساختار می‌دهند.

مسئله 14.25: طبقه‌بندی سلسله‌مراتبی در مقیاس بالا (Large-Scale Hierarchical Classification) ★

وبسایت ویکی‌پدیا می‌خواهد تمام مقالات خود را در یک ساختار درختی با 1000 دسته و زیردسته مختلف طبقه‌بندی کند. یک مدل **kNN** ساده در این مقیاس به شدت کند است. طراحی یک سیستم "طبقه‌بند سلسله‌مراتبی" (**Hierarchical**)

Classifier) با استفاده از ترکیبی از **Rocchio** (برای دسته‌های اصلی در بالای درخت) و **kNN** (برای زیردسته‌های دقیق در پایین درخت) را پیشنهاد دهید و منطق بهینگی آن را اثبات کنید.

مسئله 14.26: رفتار طبقه‌بندها در فضای خلوت ابعاد بالا (Classifiers Behavior in High-Dimensional Sparse Space) ★

در بردارهای متنی با 100 هزار بعد، بیش از 99% درایه‌های بردار صفر هستند (**Sparsity**). تحلیل کنید که این خلوت بودن فضا، چه مزیت محاسباتی پنهانی برای الگوریتم‌های مبتنی بر ضرب داخلی (**Dot Product**) مانند محاسبه شباهت کسینوسی ایجاد می‌کند؟ فرمول ریاضی را بنویسید که نشان دهد درایه‌های صفر چگونه از چرخه محاسبات حذف می‌شوند.

مسئله 14.27: تقریب همسایگی برای کلان‌داده‌ها (Neighborhood Approximation for Big Data) ★

در سیستم‌های واقعی با صدها میلیون رکورد، حتی **Inverted Index** هم برای **kNN** کند است. به عنوان یک مهندس جستجو، نحوه استفاده از الگوریتم (**Locality Sensitive Hashing**) **LSH** را برای پیاده‌سازی یک نسخه تقریبی اما فوق‌سریع از **kNN** شرح دهید. در این سیستم معماری چگونه تغییر می‌کند؟

مسئله 14.28: تزریق نویز و مقاومت مدل (Noise Injection and Model Robustness) ★

فرض کنید به 5% از داده‌های آموزشی کلاس «ورزشی»، به صورت عمدی و مخرب کلماتی از دایره لغات کلاس «علمی» تزریق شده است (**Data Poisoning**). الگوریتم‌های **Rocchio** و **kNN** (با k کوچک) هر کدام چه واکنشی به این نویز مخرب نشان می‌دهند؟ با استدلال بر اساس مفاهیم **Centroid** و **Local Boundaries** پاسخ دهید.

مسئله 14.29: مسیریابی نظرات در تجارت الکترونیک (Review Routing in E-commerce) ★

آمازون سیستمی دارد که نظرات کاربران را می‌خواند و اگر کاربر مشکل «تاخیر در ارسال» داشت به دپارتمان لجستیک، و اگر مشکل «کیفیت کالا» داشت به دپارتمان فروش می‌فرستد. کاربر می‌نویسد: "کالا عالی بود اما خیلی دیر رسید". سیستم **Rocchio** با معماری **One-of** این نظر را مسدود کرده و خطا می‌دهد. سیستم را با معماری **Any-of** بازطراحی کنید و منطق تصمیم‌گیری (**Thresholding**) آن را روی کاغذ مدل کنید.

مسئله 14.30: انتخاب پویای پارامتر k در زمان اجرا (Dynamic k Selection at Runtime) ★

در حالت استاندارد **kNN**، مقدار k یک عدد ثابت است (مثلا 5). یک الگوریتم خلاقانه طراحی کنید (همراه با شبه‌کد) که در آن سیستم بر اساس "تراکم همسایگی" هر سند ورودی، مقدار k را به صورت پویا (**Dynamic**) تنظیم کند. یعنی اگر اسناد اطراف کوئری بسیار به آن نزدیک بودند k کوچک باشد و اگر فضا خلوت بود k افزایش یابد.

مسئله 14.31: تاثیر کاهش ابعاد بر خطی بودن (Impact of Dimensionality Reduction on Linearity) ★

فرض کنید بردارهای 100 هزار بعدی متنی را با استفاده از تکنیک‌هایی مانند **SVD** (که در فصل **LSI** خوانده‌اید) به بردارهای 200 بعدی متراکم تبدیل کنیم. این کار چه تاثیری بر "تفکیک‌پذیری خطی" داده‌ها می‌گذارد؟ آیا پس از کاهش ابعاد، استفاده از یک طبقه‌بند خطی مانند **Rocchio** همچنان انتخاب منطقی است یا باید به سمت طبقه‌بندهای غیرخطی پیچیده‌تر حرکت کنیم؟

مسئله 14.32: طراحی جامع سیستم تشخیص بات (Comprehensive Bot Detection System Design) ★

یک شبکه اجتماعی 50 میلیون کاربر دارد. کاربران واقعی متونی به شدت پراکنده می‌نویسند (**Variance** بالا) در حالی که بات‌های اسپمر متونی بسیار شبیه به هم با کلمات تکراری تولید می‌کنند (تراکم بالا در نقاط خاصی از فضا). می‌خواهیم یک سیستم ترکیبی بسازیم. طراحی سیستمی را ارائه دهید که در فاز اول (فیلتر سریع) از یک مکانیزم مبتنی بر **Rocchio** برای غربالگری اولیه استفاده کند و در فاز دوم، موارد مشکوک را با یک سیستم **kNN** بهینه‌شده با ساختار نمایه وارونه (**Inverted Index**) بررسی دقیق کند. نقشه‌های جریان داده (**Data Flow**) و توجیه ریاضی این ترکیب را بر اساس مدیریت **Bias/Variance** شرح دهید.

پاسخنامه مسئله 14.1: نمایش برداری اسناد (Vector Representation)

تحلیل اولیه و هدف مسئله

در سیستم‌های بازیابی اطلاعات، کامپیوترها قادر به درک معنایی متون خام (مانند جملات و پاراگراف‌ها) نیستند. الگوریتم‌های محاسباتی تنها با اعداد و ساختارهای ریاضی کار می‌کنند. هدف از این مسئله، درک فرآیندی است که طی آن یک متن بدون ساختار، به یک موجودیت ریاضی به نام «بردار» (**Vector**) در مدلی به نام **مدل فضای برداری (Vector Space Model)** تبدیل می‌شود تا امکان مقایسه و محاسبه شباهت میان اسناد فراهم گردد.

بخش اول: چگونگی تبدیل سند متنی خام به بردار

تبدیل یک سند متنی به بردار طی یک فرآیند سه‌مرحله‌ای و ساختاریافته انجام می‌شود:

1. پیش‌پردازش متن (Text Preprocessing):

در ابتدا، سند خام باید پاک‌سازی و استاندارد شود. این مرحله شامل خرد کردن متن به کلمات مجزا (**Tokenization**)، حذف کلمات توقف و بی‌اثر مانند حروف اضافه و ربط (**Stop-word Removal**) و در نهایت ریشه‌یابی کلمات (**Stemming/Lemmatization**) است. خروجی این مرحله، مجموعه‌ای از کلمات کلیدی و خالص است.

2. استخراج واژه‌نامه (Vocabulary Construction):

سیستم تمام اسناد موجود در مجموعه (**Corpus**) را بررسی کرده و لیستی از تمامی کلمات یکتای باقی‌مانده پس از پیش‌پردازش تهیه می‌کند. این لیست جامع، واژه‌نامه سیستم را تشکیل می‌دهد.

3. وزن‌دهی و تشکیل بردار (Vectorization & Term Weighting):

در این مرحله، سند به یک بردار ریاضی تبدیل می‌شود. اگر واژه‌نامه ما دارای **n** کلمه یکتا باشد، هر سند به صورت یک بردار با **n** مؤلفه نمایش داده می‌شود:

$$\vec{d} = (w_1, w_2, \dots, w_n)$$

مقدار **w_i** نشان‌دهنده «وزن» یا اهمیت کلمه **i**ام واژه‌نامه، در سند **d** است. این وزن می‌تواند به ساده‌ترین شکل، صفر و 1 (حضور یا عدم حضور کلمه)، تعداد تکرار کلمه (**Term Frequency**) و یا از طریق فرمول‌های پیشرفته‌تری مانند **TF-IDF** محاسبه شود.

بخش دوم: نقش واژه‌نامه (Vocabulary) در تعیین ابعاد فضا

در مدل فضای برداری، نقش واژه‌نامه تعیین‌کننده **ابعاد فضا (Dimensionality)** است.

به بیان دقیق‌تر، هر کلمه یکتا در واژه‌نامه، نشان‌دهنده یک «بُعد» یا یک «محور مختصات» مستقل و عمود بر هم در یک فضای چندبعدي است. اگر واژه‌نامه کل مجموعه اسناد دارای **n** کلمه متمایز باشد (به عنوان مثال، یک میلیون کلمه یکتا)، فضای برداری ما یک فضای **n**-بعدي (یک میلیون بعدي) خواهد بود.

بنابراین:

$$V = \{t_1, t_2, \dots, t_n\} = \text{(Vocabulary)}$$

$$|V| = n = \text{تعداد ابعاد فضا}$$

در این فضای **n**-بعدي، هر سند تنها یک «نقطه» یا «بردار» است که مختصات آن نقطه روی هر محور، برابر با وزن آن کلمه در آن سند خاص می‌باشد.

مثال آموزشی و کاربردی

برای درک بهتر، فرض کنید کل مجموعه داده ما تنها شامل دو سند زیر است (پس از پیش‌پردازش):

• **سند 1:** «دانشجو»، «کتاب»، «خواند»

• **سند 2:** «دانشجو»، «کد»، «نوشت»

گام اول: تشکیل واژه‌نامه و تعیین ابعاد

کلمات یکتای کل مجموعه عبارتند از: {دانشجو، کتاب، خواند، کد، نوشت}.

تعداد کلمات یکتا 5 عدد است، بنابراین فضای برداری ما **5 بعدی** خواهد بود (**n=5**).

گام دوم: نمایش برداری (با فرض وزن‌دهی بر اساس تعداد تکرار)

هر سند اکنون به یک بردار 5 مؤلفه‌ای تبدیل می‌شود.

نام سند	دانشجو (بُعد 1)	کتاب (بُعد 2)	خواند (بُعد 3)	کد (بُعد 4)	نوشت (بُعد 5)
بردار سند 1	1	1	1	0	0
بردار سند 2	1	0	0	1	1

نتیجه‌گیری: همان‌طور که مشاهده می‌کنید، سند متنی خام کاملاً به یک ساختار عددی تبدیل شد. اکنون ماشین می‌تواند با استفاده از روابط ریاضی (مانند محاسبه کسینوس زاویه بین دو بردار)، میزان شباهت معنایی سند 1 و سند 2 را محاسبه کند. کلماتی که در سند وجود ندارند، روی محور مربوط به خود مقدار 0 می‌گیرند که باعث می‌شود بردارهای اسناد متنی معمولاً بسیار خلوت (**Sparse**) باشند.

پاسخنامه مسئله 14.2: محاسبه شباهت کسینوسی (Cosine Similarity Calculation)

تحلیل اولیه و هدف مسئله

در مدل فضای برداری، پس از آنکه اسناد به بردار تبدیل شدند، نیازمند یک معیار ریاضی برای سنجش میزان نزدیکی یا شباهت آن‌ها هستیم. هدف از این مسئله، آموزش نحوه محاسبه معیار «شباهت کسینوسی» است. این معیار زاویه بین دو بردار را در فضای چندبعدی اندازه‌گیری می‌کند. مزیت کلیدی شباهت کسینوسی نسبت به سایر معیارها (مانند فاصله اقلیدسی) این است که به طول اسناد حساس نیست؛ به این معنا که یک سند طولانی و یک سند کوتاه، در صورتی که توزیع کلمات مشابهی داشته باشند، زاویه کمی با هم خواهند داشت و مشابه در نظر گرفته می‌شوند.

مفاهیم و پیش‌نیازهای تئوری

برای محاسبه شباهت کسینوسی بین دو بردار $\vec{d}_1$ و $\vec{d}_2$ ، از رابطه ریاضی زیر استفاده می‌شود که در آن صورت کسر نشان‌دهنده «ضرب داخلی» بردارها و مخرج کسر برابر با حاصل‌ضرب «طول (اندازه)» دو بردار است:

$$\text{Cosine Similarity} = \frac{\vec{d}_1 \cdot \vec{d}_2}{|\vec{d}_1| \times |\vec{d}_2|}$$

حل گام‌به‌گام و تحلیلی

بردارهای داده شده در صورت مسئله به شرح زیر هستند:

$$\vec{d}_1 = (1, 0, 2, 1)$$

$$\vec{d}_2 = (0, 1, 1, 2)$$

برای رسیدن به پاسخ نهایی، محاسبات را در 3 گام مجزا انجام می‌دهیم:

گام اول: محاسبه ضرب داخلی (Dot Product) برای صورت کسر

برای محاسبه ضرب داخلی دو بردار، باید مؤلفه‌های متناظر (هم‌مکان) آن‌ها را در یکدیگر ضرب کرده و سپس حاصل‌ضرب‌ها را با هم جمع کنیم:

$$\vec{d}_1 \cdot \vec{d}_2 = (1 \times 0) + (0 \times 1) + (2 \times 1) + (1 \times 2) = 0 + 0 + 2 + 2 = 4$$

گام دوم: محاسبه طول یا اندازه (Magnitude) هر بردار برای مخرج کسر

اندازه هر بردار برابر است با جذر مجموع مربعات مؤلفه‌های آن بردار. ابتدا طول بردار اول ($\vec{d}_1$) را محاسبه می‌کنیم:

$$|\vec{d}_1| = \sqrt{1^2 + 0^2 + 2^2 + 1^2} = \sqrt{1 + 0 + 4 + 1} = \sqrt{6}$$

سپس طول بردار دوم ($\vec{d}_2$) را به دست می‌آوریم:

$$|\vec{d}_2| = \sqrt{0^2 + 1^2 + 1^2 + 2^2} = \sqrt{0 + 1 + 1 + 4} = \sqrt{6}$$

گام سوم: جایگذاری در فرمول اصلی و محاسبه نتیجه نهایی

اکنون مقادیر به دست آمده از گام‌های قبل را در فرمول شباهت کسینوسی قرار می‌دهیم:

$$\text{Cosine Similarity} = \frac{4}{\sqrt{6} \times \sqrt{6}} = \frac{4}{6} \approx 0.667$$

جمع‌بندی و نکات تکمیلی

نتیجه‌گیری: مقدار محاسبه شده برای شباهت این دو سند، تقریباً برابر با 0.667 است. از آنجا که وزن‌ها (فراوانی کلمات) مقادیری نامنفی هستند، مقدار کسینوس زاویه بین بردارهای اسناد همواره بازه‌ای بین 0 (عدم شباهت مطلق و بردارهای متعامد) تا 1 (تطابق کامل و بردارهای هم‌راستا) خواهد داشت. مقدار به دست آمده نشان می‌دهد که این دو سند دارای هم‌پوشانی و شباهت نسبتاً بالایی در محتوا هستند. در یک موتور جستجو، اسناد بازیابی شده بر اساس این عدد به صورت نزولی (از بیشترین شباهت به کمترین) مرتب شده و به کاربر نمایش داده می‌شوند.

پاسخنامه مسئله 14.3: محاسبه مرکز ثقل (Centroid Calculation in Rocchio)

تحلیل اولیه و هدف مسئله

در الگوریتم‌های دسته‌بندی متن (مانند الگوریتم **Rocchio**) و همچنین در خوشه‌بندی، نیاز داریم تا مجموعه‌ای از اسناد مرتبط یا متعلق به یک کلاس خاص را با یک «نماینده واحد» نشان دهیم. هدف این مسئله، معرفی این نماینده که همان «مرکز ثقل» یا **Centroid** است، و درک مفهوم هندسی آن در مدل فضای برداری است.

بخش اول: فرمول ریاضی مرکز ثقل

برای یک کلاس (یا دسته) مشخص مانند c که شامل مجموعه‌ای از اسناد D_c است، بردار مرکز ثقل (که معمولاً با نماد $\vec{\mu}(c)$ نشان داده می‌شود) برابر با میانگین حسابی بردارهای تمامی اسناد موجود در آن کلاس است. فرمول ریاضی آن به شکل زیر تعریف می‌شود:

$$\vec{\mu}(c) = \frac{1}{|D_c|} \sum_{d \in D_c} \vec{v}(d)$$

- $\vec{\mu}(c)$: بردار مرکز ثقل برای کلاس c .
- D_c : مجموعه تمام اسنادی که متعلق به کلاس c هستند.
- $|D_c|$: تعداد کل اسناد موجود در کلاس c .
- $\vec{v}(d)$: بردار نمایش‌دهنده یک سند منفرد در فضای n -بعدی.
- $\sum$: نماد جمع برداری؛ به این معنا که تمامی مؤلفه‌های متناظر بردارها با یکدیگر جمع می‌شوند.

بخش دوم: تفسیر هندسی به زبان ساده

از دیدگاه هندسی، اگر هر سند متنی را به عنوان یک «نقطه» در فضای n -بعدی در نظر بگیریم، بردار مرکز ثقل، دقیقاً «مرکز جرم» (Center of Mass) یا «نقطه میانگین» آن مجموعه از نقاط است.

برای درک بهتر، یک فضای 2 بعدی (فقط با 2 کلمه در واژه‌نامه) را تصور کنید که در آن اسناد یک کلاس مانند نقاطی روی یک صفحه پراکنده‌اند. اگر مختصات طول و عرض همه این نقاط را با هم جمع کرده و بر تعدادشان تقسیم کنیم، به نقطه‌ای در وسط این تجمع می‌رسیم. این نقطه میانی همان مرکز ثقل است.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: مرکز ثقل به ما کمک می‌کند تا یک مجموعه بزرگ از اسناد را تنها در قالب **یک بردار واحد (پروتوتایپ)** خلاصه کنیم. در الگوریتم **Rocchio**، وقتی یک سند جدید و ناشناخته وارد سیستم می‌شود، به جای اینکه ماشین آن را با صدها سند مقایسه کند، تنها کافیت فاصله (یا شباهت) آن را با مرکز ثقل هر کلاس بسنجد. سند جدید به کلاسی اختصاص می‌یابد که به مرکز ثقل آن نزدیک‌تر باشد. این کار محاسبات سیستم‌های بازیابی اطلاعات را به شدت کاهش داده و سرعت پردازش را بالا می‌برد.

پاسخنامه مسئله 14.4: مرز تصمیم‌گیری روکیو (Rocchio Decision Boundary)

تحلیل اولیه و هدف مسئله

در الگوریتم دسته‌بندی روکیو (**Rocchio**)، تخصیص یک سند جدید به یک کلاس، بر اساس میزان نزدیکی آن سند به مرکز ثقل (**Centroid**) کلاس‌ها انجام می‌شود. هدف از این مسئله، درک مفهوم «مرز تصمیم‌گیری» است؛ یعنی دقیقاً آن مرزی در فضای برداری که اگر سندی دقیقاً روی آن قرار گیرد، سیستم در تخصیص آن به کلاس اول یا دوم دچار تردید (تساوی فاصله) می‌شود. این مسئله یکی از پایه‌ای‌ترین مفاهیم برای شناخت عملکرد «دسته‌بندهای خطی» (**Linear Classifiers**) است.

بخش اول: نحوه رسم مرز تصمیم‌گیری از دیدگاه هندسی

از لحاظ هندسی، مرز تصمیم‌گیری بین دو کلاس در الگوریتم روکیو، مجموعه‌ای از تمام نقاطی است که فاصله آن‌ها تا مرکز ثقل کلاس اول ($\vec{\mu}_1$) دقیقاً برابر با فاصله‌شان تا مرکز ثقل کلاس دوم ($\vec{\mu}_2$) باشد.

برای رسم این مرز (به عنوان مثال در یک فضای 2 بعدی)، سیستم دقیقاً مشابه رسم یک «عمود منصف» در هندسه کلاسیک عمل می‌کند:

1. یک پاره‌خط فرضی بین مرکز ثقل کلاس اول و مرکز ثقل کلاس دوم رسم می‌شود.

2. نقطه وسط (Midpoint) این پاره‌خط مشخص می‌گردد.

3. خطی دقیقاً از این نقطه وسط و با زاویه 90 درجه (عمود) بر پاره‌خط رسم می‌شود.

این خط عمود منصف، همان مرز تصمیم‌گیری است. در یک طرف این مرز، اسناد به کلاس اول و در طرف دیگر، به کلاس دوم اختصاص می‌یابند.

بخش دوم: اثبات ریاضی چرا مرز تصمیم‌گیری همیشه یک ابرصفحه است؟

«اب‌صفحه» (Hyperplane) در واقع تعمیم‌یافته‌ی مفهوم خط (در فضای 2 بعدی) و صفحه (در فضای 3 بعدی) برای فضاهاى چندبعدی (n-بعدی) است. از نظر ریاضی، یک معادله زمانی یک ابرصفحه را تعریف می‌کند که نسبت به متغیر ورودی (در اینجا بردار سند $\vec{x}$) یک **معادله خطی** (درجه 1) باشد و هیچ توانی بالاتر از 1 نداشته باشد.

برای اثبات اینکه مرز روکیو همیشه یک ابرصفحه است، باید نشان دهیم که شرط «تساوی فواصل» همواره منجر به یک معادله خطی می‌شود. فرض می‌کنیم از معیار فاصله اقلیدسی استفاده می‌کنیم:

گام 1: برابری فواصل از دو مرکز ثقل

سند $\vec{x}$ دقیقاً روی مرز قرار دارد اگر و تنها اگر فاصله آن از هر دو مرکز ثقل برابر باشد:

$$|\vec{x} - \vec{\mu}_1| = |\vec{x} - \vec{\mu}_2|$$

گام 2: به توان 2 رساندن طرفین برای حذف رادیکال

برای تسهیل محاسبات، هر دو طرف معادله را به توان 2 می‌رسانیم (چون فاصله مقداری نامنفی است، این کار تغییری در معادله ایجاد نمی‌کند):

$$|\vec{x} - \vec{\mu}_1|^2 = |\vec{x} - \vec{\mu}_2|^2$$

گام 3: بسط دادن مجذور فواصل بر اساس ضرب داخلی

بر اساس قوانین جبر برداری، مجذور فاصله دو بردار به شکل $(\vec{A} - \vec{B})^2$ برابر است با $\|\vec{A}\|^2 - 2\vec{A} \cdot \vec{B} + \|\vec{B}\|^2$. این قانون را برای هر دو طرف معادله پیاده‌سازی می‌کنیم:

$$|\vec{x}|^2 - 2\vec{x} \cdot \vec{\mu}_1 + |\vec{\mu}_1|^2 = |\vec{x}|^2 - 2\vec{x} \cdot \vec{\mu}_2 + |\vec{\mu}_2|^2$$

گام 4: ساده‌سازی و حذف عبارات غیرخطی (نقطه کلیدی اثبات)

همان‌طور که مشاهده می‌کنید، عبارت $\|\vec{x}\|^2$ که نشان‌دهنده اندازه (طول) بردار متغیر ماست و یک عبارت درجه 2 (غیرخطی) محسوب می‌شود، در هر دو سمت تساوی با علامت مثبت وجود دارد. بنابراین این دو عبارت یکدیگر را به طور کامل خنثی کرده و حذف می‌شوند:

$$-2\vec{x} \cdot \vec{\mu}_1 + |\vec{\mu}_1|^2 = -2\vec{x} \cdot \vec{\mu}_2 + |\vec{\mu}_2|^2$$

گام 5: مرتب‌سازی به فرم استاندارد معادله خطی

اکنون تمام عبارات دارای متغیر $\vec{x}$ را به یک سمت و مقادیر ثابت را به سمت دیگر می‌بریم و فاکتورگیری می‌کنیم:

$$2\vec{x} \cdot (\vec{\mu}_1 - \vec{\mu}_2) - (|\vec{\mu}_1|^2 - |\vec{\mu}_2|^2) = 0$$

اگر کل معادله را بر 2 تقسیم کنیم و بردار ثابت $\vec{w} = \vec{\mu}_1 - \vec{\mu}_2$ (که همان بردار نرمال یا عمود بر مرز است) و مقدار ثابت $b = \frac{1}{2}(|\vec{\mu}_2|^2 - |\vec{\mu}_1|^2)$ را تعریف کنیم، معادله به شناخته‌شده‌ترین فرم استاندارد یک ابرصفحه در هندسه تحلیلی تبدیل می‌شود:

$$\vec{w} \cdot \vec{x} + b = 0$$

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: از آنجا که در فرآیند بسط دادن فرمول، عبارت توان‌دار و غیرخطی $\|\vec{x}\|^2$ از هر دو طرف خط خورد و حذف شد، معادله نهایی $(\vec{w} \cdot \vec{x} + b = 0)$ هیچ متغیری با توان 2 یا بالاتر ندارد. این اثبات ریاضی به‌وضوح نشان می‌دهد که چرا الگوریتم **Rocchio** ذاتاً یک الگوریتم **طبقه‌بندی خطی (Linear Classifier)** است. این ویژگی باعث می‌شود که محاسبات روکیو در فضای ابعاد بالا (مانند متن) بسیار سریع باشد، اما در عین حال نقطه ضعف آن این است که نمی‌تواند کلاس‌هایی با مرزهای پیچیده، منحنی‌شکل یا تو در تو را به درستی از یکدیگر تفکیک کند.

پاسخنامه مسئله 14.5: مفهوم پایه همسایه‌های نزدیک (Basic Concept of kNN)

تحلیل اولیه و هدف مسئله

برخلاف الگوریتم‌هایی مانند **Rocchio** که برای هر کلاس یک «نماینده کلی» (مرکز ثقل) می‌سازند، الگوریتم «**k** همسایه نزدیک» (**k-Nearest Neighbors**) یا به اختصار **kNN** رویکردی کاملاً متفاوت دارد. این الگوریتم در دسته روش‌های «یادگیری تنبل» (**Lazy Learning**) قرار می‌گیرد؛ به این معنا که هیچ مدل ریاضی مشخصی در مرحله آموزش (**Training**) از پیش نمی‌سازد و تمام بار محاسباتی و تصمیم‌گیری‌ها را به لحظه ورود یک سند جدید موکول می‌کند. هدف از این مسئله، درک دقیق مراحل اجرای این الگوریتم و تحلیل تأثیر یکی از مهم‌ترین پارامترهای آن (یعنی **k**) بر رفتار و عملکرد سیستم است.

بخش اول: مراحل گام‌به‌گام دسته‌بندی یک سند جدید

هنگامی که یک سند جدید و ناشناخته جهت دسته‌بندی وارد سیستم می‌شود، الگوریتم **kNN** دقیقاً 3 مرحله ساختاریافته زیر را طی می‌کند:

گام 1: محاسبه فاصله یا شباهت (Distance/Similarity Calculation)

سیستم سند جدید را به عنوان یک بردار $\vec{d}_{new}$ در فضای n -بعدی در نظر می‌گیرد. سپس، ماشین فاصله (مانند فاصله اقلیدسی) یا میزان شباهت (مانند شباهت کسینوسی) این بردار را با تک‌تک بردارهای اسناد موجود در مجموعه داده آموزشی (Training Set) محاسبه می‌کند. فرمول ریاضی محاسبه فاصله اقلیدسی برای درک بهتر به شرح زیر است:

$$Distance(\vec{d}_{new}, \vec{d}_i) = \sqrt{\sum_{j=1}^n (w_{new,j} - w_{i,j})^2}$$

گام 2: مرتب‌سازی و انتخاب همسایه‌ها (Finding the Neighborhood)

پس از محاسبه شباهت‌ها، تمام اسناد آموزشی بر اساس میزان نزدیکی به سند جدید مرتب می‌شوند (نزولی برای شباهت و صعودی برای فاصله). در این مرحله، الگوریتم دقیقاً k سندی که بالاترین شباهت را به سند جدید دارند، انتخاب می‌کند. این k نقطه در فضای برداری، اصطلاحاً «همسایه‌های نزدیک» نامیده می‌شوند.

گام 3: رأی‌گیری اکثریت (Majority Voting)

در مرحله نهایی، الگوریتم برچسب (کلاس) این k همسایه را بررسی کرده و از مکانیزم «رأی‌گیری اکثریت» استفاده می‌کند. هر کلاس به تعداد اسنادی که در میان این k همسایه دارد، یک رأی دریافت می‌کند. در نهایت، سند جدید به کلاسی اختصاص می‌یابد که بیشترین رأی را کسب کرده باشد. فرمول ریاضی این تصمیم‌گیری به شکل زیر نشان داده می‌شود:

$$c^* = \operatorname{argmax}_{c \in C} \sum_{\vec{d}_i \in N_k(\vec{d}_{new})} I(y_i = c)$$

(نکته مهم: در برخی پیاده‌سازی‌ها، رأی‌گیری به صورت «وزن‌دار» انجام می‌شود؛ یعنی رأی همسایه‌ای که به سند جدید نزدیک‌تر است، ارزش بیشتری نسبت به همسایه دورتر دارد.)

بخش دوم: نقش پارامتر k و تأثیر آن بر مرزهای تصمیم‌گیری

پارامتر k در این الگوریتم یک ابرپارامتر (Hyperparameter) است که نشان‌دهنده «اندازه همسایگی» می‌باشد. این عدد تعیین می‌کند که چند سند باید در رأی‌گیری نهایی مشارکت کنند. انتخاب مقدار مناسب برای k کاملاً حیاتی است و رفتار مدل را به شدت تغییر می‌دهد:

اگر k بسیار کوچک باشد (مثلاً $k=1$):

در این حالت، مدل صرفاً به نزدیک‌ترین سند نگاه می‌کند. مرزهای تصمیم‌گیری در فضای برداری بسیار پیچیده، دندان‌دار و نامنظم خواهند شد. در این شرایط الگوریتم به شدت تحت تأثیر نویزها و داده‌های پرت (Outliers) قرار می‌گیرد و مدل اصطلاحاً دچار پدیده «بیش‌برازش» (Overfitting) می‌شود.

اگر k بسیار بزرگ باشد:

با افزایش k ، مرزهای تصمیم‌گیری نرم‌تر، کلی‌تر و هموارتر می‌شوند و مدل در برابر نویز داده‌های محلی مقاوم می‌گردد. اما اگر k بیش از حد بزرگ انتخاب شود (مثلاً نزدیک به کل تعداد اسناد آموزشی)، کلاس‌های پرجمعیت‌تر در رأی‌گیری

همیشه غالب می‌شوند و مدل الگوهای ظریف را نادیده می‌گیرد؛ پدیده‌ای که به آن «کم‌برازش» (Underfitting) می‌گویند.

قاعده فرد بودن مقدار k :

در مسائل دسته‌بندی دوکلاسه (Binary Classification)، همواره توصیه می‌شود مقدار k یک عدد فرد (مانند 3، 5 یا 7) انتخاب شود. این کار از بروز حالت تساوی آرا (Tie) در زمان رأی‌گیری جلوگیری می‌کند.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: نقطه قوت اصلی kNN سادگی فوق‌العاده در فهم و توانایی بالای آن در شکل‌دهی به مرزهای تصمیم‌گیری بسیار پیچیده و غیرخطی است. با این حال، همواره باید به بزرگ‌ترین چالش محاسباتی این الگوریتم در سیستم‌های بازیابی اطلاعات دقت کنید: از آنجا که مدل فاقد فاز آموزش است، هزینه محاسباتی در فاز تست (Testing Phase) به شدت بالاست. برای دسته‌بندی تنها یک سند متنی جدید، سیستم مجبور است فاصله آن را با ده‌ها هزار یا میلیون‌ها سند آموزشی مقایسه کند که نیازمند زمان پردازش و حافظه بسیار بالایی خواهد بود.

پاسخنامه مسئله 14.6: شکستن گره در دسته‌بندی (Tie-Breaking in kNN)

تحلیل اولیه و هدف مسئله

در الگوریتم k -Nearest Neighbors، تصمیم‌گیری بر اساس مکانیزم «رأی‌گیری اکثریت» انجام می‌شود. یکی از چالش‌های ساختاری این روش، احتمال وقوع «گره» یا تساوی آرا (Tie) است. این وضعیت معمولاً زمانی رخ می‌دهد که پارامتر k یک عدد زوج انتخاب شود (مانند $k=4$ در مسئله حاضر) و یا زمانی که با یک مسئله چندکلاسه (Multi-class) مواجه باشیم که در آن آرا بین چندین کلاس پخش می‌شود. هدف این مسئله، معرفی تکنیک‌های استاندارد و علمی برای خروج از این بن‌بست تصمیم‌گیری در سیستم‌های بازیابی اطلاعات است.

بخش اول: روش‌های رایج شکستن گره (Tie-Breaking)

برای رفع مشکل تساوی آرا در الگوریتم kNN ، سیستم‌های هوشمند عموماً از یکی از 2 رویکرد استاندارد زیر استفاده می‌کنند:

1. روش اول: وزن‌دهی آرا بر اساس شباهت/فاصله (Similarity-Weighted Voting)

این روش که علمی‌ترین و رایج‌ترین تکنیک در بازیابی اطلاعات محسوب می‌شود، قانون «هر همسایه، یک رأی برابر» را کنار می‌گذارد. در این استراتژی، ارزش رأی هر همسایه با میزان نزدیکی آن به سند جدید متناسب است.

نحوه عملکرد: سیستم میزان شباهت (مثلاً شباهت کسینوسی) تک‌تک همسایه‌ها را محاسبه می‌کند. سپس برای

کلاس **A**، شباهت‌های 2 همسایه متعلق به این کلاس را با هم جمع می‌کند و همین کار را برای کلاس **B** نیز انجام

می‌دهد. کلاسی که مجموع شباهت‌های بالاتری (یا مجموع فواصل کمتری) داشته باشد، برنده قطعی رأی‌گیری خواهد بود.

مزیت: این روش بسیار دقیق است زیرا همسایه‌ای که به سند جدید بسیار نزدیک‌تر چسبیده است، تأثیر بسیار بیشتری در تعیین کلاس آن خواهد داشت.

2. روش دوم: تغییر موقت شعاع همسایگی (Dynamic k Adjustment / Fallback)

در این رویکرد، سیستم به جای تغییر وزن آرا، ساختار همسایگی را به صورت موقت و تنها برای همان سند خاص دستکاری می‌کند تا تساوی را برهم بزند. این کار معمولاً به 2 شکل انجام می‌شود:

کاهش k (بازگشت به $k-1$): سیستم دورترین همسایه در میان آن 4 همسایه را حذف می‌کند تا مسئله به یک رأی‌گیری با 3 همسایه ($k=3$) تبدیل شود. با فرد شدن تعداد، گره شکسته می‌شود.

اتکا به نزدیک‌ترین همسایه ($1-NN$): به عنوان یک راهکار بسیار سریع، سیستم تمام رأی‌گیری‌ها را نادیده گرفته و سند را مستقیماً به کلاسی تخصیص می‌دهد که **نزدیک‌ترین همسایه مطلق (رتبه اول در شباهت)** به آن تعلق دارد.

نکته تکمیلی (روش سوم در سناریوهای خاص)

در مواقعی که حتی با 2 روش بالا نیز تساوی برقرار بماند (مثلاً همسایه‌ها دقیقاً فواصل یکسانی داشته باشند)، سیستم‌ها از **احتمال پیشین کلاس‌ها (Class Prior)** استفاده می‌کنند؛ به این معنا که سیستم بررسی می‌کند در کل مجموعه داده‌های آموزشی، کلاس **A** اسناد بیشتری دارد یا کلاس **B**. سپس سند جدید را به کلاسی که بزرگ‌تر و پرجمعیت‌تر است اختصاص می‌دهد، زیرا از نظر آماری احتمال تعلق به کلاسی رایج‌تر، همواره بیشتر است.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: وقوع گره در الگوریتم **kNN** نشان‌دهنده ابهام مدل در مرز تصمیم‌گیری است. ساده‌ترین و پیشگیرانه‌ترین راهکار برای جلوگیری از این مشکل در مسائل دوکلاسه، **انتخاب همیشگی یک عدد فرد برای k** (مانند 3، 5 یا 7) در فاز تنظیم پارامترهاست. با این حال، تعبیه الگوریتم‌های شکستن گره (به‌ویژه روش رأی‌گیری وزن‌دار) در کدنویسی سیستم‌های هوشمند الزامی است، زیرا در مسائل دارای بیش از 2 کلاس، حتی فرد بودن **k** نیز تضمینی برای عدم وقوع تساوی آرا نخواهد بود.

پاسخنامه مسئله 14.7: تعریف طبقه‌بند خطی (Definition of Linear Classifier)

تحلیل اولیه و هدف مسئله

در دنیای یادگیری ماشین و بازیابی اطلاعات، الگوریتم‌های طبقه‌بندی (**Classification**) بر اساس «شکل هندسی مرز تصمیم‌گیری» آن‌ها به 2 دسته بزرگ خطی و غیرخطی تقسیم می‌شوند. هدف این مسئله، درک دقیق این تفاوت ساختاری و تعیین جایگاه 2 الگوریتم پایه یعنی **Rocchio** و **kNN** در این دسته‌بندی است. شناخت این موضوع به طراح سیستم کمک می‌کند تا الگوریتم مناسب را بر اساس پیچیدگی توزیع داده‌ها انتخاب کند.

بخش اول: طبقه‌بند خطی چیست؟

یک **طبقه‌بند خطی (Linear Classifier)** الگوریتمی است که تصمیم‌گیری و تفکیک کلاس‌ها را صرفاً بر اساس یک «ترکیب خطی» از ویژگی‌های ورودی (مؤلفه‌های بردار) انجام می‌دهد.

از دیدگاه هندسی، مرز تصمیم‌گیری در یک طبقه‌بند خطی همواره یک **اب‌صفحه (Hyperplane)** است.

- در یک فضای 2 بعدی، این مرز یک **خط راست** است.
- در فضای 3 بعدی، یک **صفحه صاف** است.
- در فضاهای **n**-بعدی (مانند فضای برداری متون)، یک سطح کاملاً هموار و بدون انحنا است.

معادله ریاضی این مرز همواره به فرم استاندارد زیر است که در آن هیچ متغیری توان بالاتر از 1، ضرب متغیرها در یکدیگر، یا توابع غیرخطی (مانند سینوس یا لگاریتم) ندارد:

$$\vec{w} \cdot \vec{x} + b = 0$$

که در آن $\vec{w}$ بردار وزن‌ها (تعیین‌کننده شیب مرز) و b مقدار انحراف یا عرض از مبدأ (**Bias**) است.

بخش دوم: جایگاه الگوریتم Rocchio

الگوریتم Rocchio یک طبقه‌بند کاملاً خطی است.

همان‌طور که در مسئله 14.4 به صورت ریاضی اثبات شد، مرز تصمیم‌گیری بین 2 کلاس در این الگوریتم، عمود منصف پاره‌خط متصل‌کننده مراکز ثقل آن 2 کلاس است. معادله این عمود منصف به صورت یک ترکیب خطی ساده به دست آمد. به دلیل همین ماهیت خطی، الگوریتم Rocchio بسیار سریع است، اما نمی‌تواند کلاس‌هایی را که در فضای برداری به صورت تو در تو یا با مرزهای منحنی توزیع شده‌اند، با دقت بالایی از هم تفکیک کند.

بخش سوم: جایگاه الگوریتم kNN

الگوریتم **k-Nearest Neighbors (kNN)** یک طبقه‌بند غیرخطی (**Non-linear Classifier**) محسوب می‌شود.

برخلاف روکیو، **kNN** سعی نمی‌کند کل فضا را با یک خط صاف به 2 نیمه تقسیم کند. از آنجا که تصمیم‌گیری به صورت کاملاً «محلی» و بر اساس همسایگان نزدیک انجام می‌شود، مرزهای تصمیم‌گیری می‌توانند به شدت پیچیده، دندان‌دار و دارای انحنا باشند.

در حالتی که **k=1** باشد، مرز تصمیم‌گیری **kNN** شکلی هندسی به نام **تسل‌بندی ورونوی (Voronoi Tessellation)** ایجاد می‌کند که مجموعه‌ای از چندضلعی‌های پیچیده حول هر نقطه داده است. با افزایش **k**، این مرزها کمی هموارتر می‌شوند، اما هرگز به یک خط صاف (ابرفضحه) محدود نمی‌شوند. این ویژگی به **kNN** اجازه می‌دهد الگوهای بسیار پیچیده و غیرخطی را یاد بگیرد.

نکته کلیدی آموزشی: در انتخاب الگوریتم، یک بده‌بستان (**Trade-off**) همیشگی وجود دارد: طبقه‌بندهای خطی (مانند **Rocchio**) سریع‌تر هستند و کمتر دچار بیش‌برازش (**Overfitting**) می‌شوند، در حالی که طبقه‌بندهای غیرخطی (مانند **kNN**) انعطاف‌پذیری بیشتری برای حل مسائل پیچیده دارند اما به داده و زمان پردازش بسیار بیشتری نیازمندند.

پاسخنامه مسئله 14.8: استراتژی‌های چندکلاسه (Multiclass Strategies)

تحلیل اولیه و هدف مسئله

در بسیاری از سیستم‌های بازیابی اطلاعات و دسته‌بندی متون، با مسائلی مواجه هستیم که تعداد کلاس‌ها (برچسب‌ها) بیشتر از 2 مورد است. در این شرایط، نحوه برخورد سیستم با تخصیص کلاس‌ها به اسناد بسیار تعیین‌کننده است. هدف از این مسئله، شناخت 2 استراتژی بنیادی **One-of** و **Any-of** در مواجهه با مجموعه داده‌های چندکلاسه و توانایی انتخاب معماری صحیح بر اساس منطق کسب‌وکار یا نیاز سیستم است.

بخش اول: بررسی تفاوت‌های ساختاری

2 رویکرد اصلی برای تخصیص کلاس‌ها به یک سند وجود دارد که تفاوت آن‌ها در «قانون انحصار» است:

1. استراتژی One-of (دسته‌بندی تک‌برچسبی یا انحصاری):

در این استراتژی، کلاس‌ها نسبت به یکدیگر **مانعه‌الجمع (Mutually Exclusive)** هستند. این بدان معناست که هر سند متنی باید و تنها باید به **1 و فقط 1 کلاس** تخصیص یابد.

در این روش، سیستم امتیاز یا احتمال تعلق سند را برای تمام کلاس‌ها محاسبه کرده و در نهایت کلاسی را که بالاترین امتیاز را دارد (**Max Score**)، به عنوان تنها برنده اعلام می‌کند.

مثال کاربردی: دسته‌بندی زبان یک متن. یک مقاله یا به زبان فارسی است یا انگلیسی؛ نمی‌تواند همزمان هر 2 زبان غالب را داشته باشد.

2. استراتژی Any-of (دسته‌بندی چندبرچسبی یا مستقل):

در این استراتژی، کلاس‌ها کاملاً از یکدیگر **مستقل (Independent)** فرض می‌شوند. یک سند می‌تواند همزمان 0، 1، یا چندین برچسب را به خود اختصاص دهد.

در این روش، سیستم به جای پیدا کردن یک برنده قطعی، برای هر کلاس یک «حد آستانه» (**Threshold**) در نظر می‌گیرد. اگر امتیاز سند در هر کلاسی بالاتر از آن آستانه باشد، آن برچسب به سند داده می‌شود.

مثال کاربردی: برچسب‌گذاری فیلم‌ها (ژانر). یک فیلم می‌تواند همزمان اکشن، علمی‌تخیلی و کمدی باشد.

مقایسه سریع برای دانشجویان

ویژگی	استراتژی One-of (تک‌برچسبی)	استراتژی Any-of (چندبرچسبی)
تعداد برچسب‌های خروجی	دقیقاً 1 برچسب	از 0 تا تمامی برچسب‌های ممکن
رابطه بین کلاس‌ها	مانعه‌الجمع و رقابتی	کاملاً مستقل از یکدیگر
معیار تصمیم‌گیری	انتخاب کلاس دارای حداکثر امتیاز	عبور از حد آستانه مستقل برای هر کلاس

بخش دوم: تحلیل سناریوی سیستم خبری

صورت مسئله یک سیستم خبری را مطرح می‌کند که در آن، یک خبر می‌تواند همزمان برچسب‌های «ورزشی» و «اقتصادی» را دریافت کند (مثلاً خبری درباره واگذاری سهام یک باشگاه بزرگ فوتبال در بورس).

با توجه به این شرط، انتخاب استراتژی **One-of** کاملاً **اشتباه** است، زیرا سیستم را مجبور می‌کند که بین ورزش و اقتصاد تنها یکی را انتخاب کند و اطلاعات ارزشمند دیگر را دور بریزد.

بنابراین، **استراتژی Any-of (چندبرچسبی) مناسب‌ترین و تنها انتخاب منطقی برای این سیستم است.**

در این روش، تعلق خبر به کلاس «ورزشی» هیچ مانعی برای تعلق همزمان آن به کلاس «اقتصادی» ایجاد نمی‌کند و سیستم می‌تواند محتوای چندوجهی خبر را به درستی منعکس کند.

نکته در پیاده‌سازی: در پروژه‌های عملی و برنامه‌نویسی، استراتژی **Any-of** معمولاً با استفاده از تکنیک **One-vs-All** پیاده‌سازی می‌شود. به این صورت که برای سیستم خبری مذکور، سیستم عملاً چند دسته‌بند مستقل می‌سازد: یک دسته‌بند باینری که فقط

می‌پرسد «آیا خبر ورزشی است یا خیر؟» و یک دسته‌بند دیگر که می‌پرسد «آیا خبر اقتصادی است یا خیر؟» و نتایج این 2 کاملاً مستقل از هم بررسی می‌شوند.

پاسخنامه مسئله 14.9: تعریف بایاس و واریانس (Bias and Variance)

تحلیل اولیه و هدف مسئله

در یادگیری ماشین، عملکرد یک مدل دسته‌بندی تنها به الگوریتم انتخابی بستگی ندارد؛ بلکه به میزان «خطای پیش‌بینی» مدل نیز وابسته است. برای درک اینکه چرا یک مدل ممکن است در تست‌های آموزشی عالی عمل کند اما روی داده‌های واقعی شکست بخورد، باید با مفاهیم **بایاس (Bias)** و **واریانس (Variance)** آشنا باشیم. این مسئله به بررسی منشأ خطاها و چگونگی ایجاد تعادل میان این 2 پارامتر می‌پردازد.

بخش اول: تعریف مفاهیم پایه

برای درک این مفاهیم، تصور کنید هدف ما آموزش مدلی است که مرز تصمیم‌گیری بهینه را پیدا کند.

1. بایاس (Bias):

بایاس به معنای «خطای ناشی از فرضیات ساده‌انگارانه» است. اگر مدل ما بیش از حد ساده باشد (مانند یک طبقه‌بند خطی برای داده‌هایی که توزیع بسیار پیچیده و منحنی دارند)، مدل نمی‌تواند پیچیدگی داده‌ها را درک کند.

بایاس بالا: به معنای **کم‌برازش (Underfitting)** است؛ یعنی مدل حتی روی داده‌های آموزشی هم دقت خوبی ندارد، زیرا فرض‌های آن برای مسئله بسیار ساده است.

2. واریانس (Variance):

واریانس به معنای «حساسیت بیش از حد مدل به تغییرات کوچک در داده‌های آموزشی» است. اگر مدل ما بیش از حد پیچیده باشد (مانند یک **kNN** با **k=1**)، سعی می‌کند تمام جزئیات و نویزهای داده‌های آموزشی را حفظ کند.

واریانس بالا: به معنای **بیش‌برازش (Overfitting)** است؛ یعنی مدل روی داده‌های آموزشی عالی عمل می‌کند (خطای 0)، اما به محض دیدن داده‌های جدید و ندیده، به شدت شکست می‌خورد، زیرا مدل به جای یادگیری «الگوی کلی»، «حافظه» داده‌های آموزشی شده است.

بخش دوم: ترکیب خطا (Bias-Variance Trade-off)

خطای کلی سیستم (**Total Error**) حاصل جمع این 2 عامل به همراه یک خطای غیرقابل اجتناب (نویز ذاتی داده‌ها) است. رابطه آن‌ها را می‌توان به صورت زیر فرموله کرد:

$$Total Error = Bias^2 + Variance + Irreducible Error$$

تفسیر این رابطه:

اگر بایاس را کاهش دهیم (مدل پیچیده‌تر شود): واریانس مدل به صورت خودکار افزایش می‌یابد.

اگر واریانس را کاهش دهیم (مدل ساده‌تر شود): بایاس مدل افزایش می‌یابد.

این یک «بده‌بستان» (Trade-off) است. ما به دنبال نقطه‌ای طلایی هستیم که در آن مجموع این 2 به حداقل برسد.

مثال کاربردی در دسته‌بندی متون

برای روشن‌تر شدن موضوع، 2 الگوریتمی که تا اینجا بررسی کردیم را در این چارچوب تحلیل می‌کنیم:

Rocchio (مدل ساده): معمولاً دارای **بایاس بالا و واریانس پایین** است. چون مرز آن همیشه خطی و ساده است، به داده‌های آموزشی وابسته نیست (واریانس کم)، اما اگر داده‌ها غیرخطی باشند، دقت آن محدود می‌ماند (بایاس زیاد).

kNN (مدل پیچیده): معمولاً دارای **بایاس پایین و واریانس بالا** است. چون مرز آن بسیار منعطف است، می‌تواند هر الگویی را یاد بگیرد (بایاس کم)، اما اگر داده‌های آموزشی تغییر کنند یا نویز داشته باشند، پیش‌بینی‌های مدل به شدت تغییر می‌کند (واریانس زیاد).

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: هدف ما در طراحی یک سیستم بازیابی اطلاعات، رسیدن به مدل «متعادل» است. اگر سیستم شما روی داده‌های آموزشی 100% دقت دارد اما در عمل ضعیف است، مشکل شما **واریانس بالا** است و باید با افزایش داده‌های آموزشی یا افزایش **k** (در **kNN**) مدل را ساده‌تر کنید. اگر سیستم شما روی داده‌های آموزشی هم دقت کمی دارد، مشکل **بایاس بالا** است و باید به سمت الگوریتم‌های پیچیده‌تر (مانند **SVM** یا مدل‌های غیرخطی) حرکت کنید.

پاسخنامه مسئله 14.10: وزن‌دهی کلمات (Term Weighting and TF-IDF)

تحلیل اولیه و هدف مسئله

در ساخت «مدل فضای برداری» (**Vector Space Model**)، قدم نهایی پیش از اجرای الگوریتم‌های دسته‌بندی (مانند **kNN** و **Rocchio**)، تعیین مقدار عددی هر مؤلفه در بردارهاست. این مقدار نشان‌دهنده میزان اهمیت یک کلمه در یک سند است. هدف از این مسئله، درک چرایی ناکارآمدی روش «شمارش ساده کلمات» (**Term Frequency**) و تحلیل اثرات ریاضی و منطقی روش پیشرفته‌تر **TF-IDF** در افزایش دقت دسته‌بندهاست.

بخش اول: ضعف بنیادین در شمارش ساده (TF)

اگر وزن هر کلمه در بردار سند را صرفاً برابر با تعداد دفعات تکرار آن (**Term Frequency**) در نظر بگیریم، با یک مشکل ساختاری به نام «فقدان قدرت تفکیک‌پذیری» (**Lack of Discriminative Power**) مواجه می‌شویم.

در هر زبان یا دامنه تخصصی، کلماتی وجود دارند که به شدت پرتکرار هستند اما هیچ بار معنایی خاصی برای دسته‌بندی ندارند. این کلمات شامل حروف ربط و اضافه (مانند «در»، «است»، «که») و یا کلمات عمومی یک حوزه (مانند کلمه «کامپیوتر» در مجموعه مقالات علوم رایانه) می‌شوند.

در روش شمارش ساده، این کلمات بی‌ارزش، بزرگ‌ترین اعداد را در بردار سند به خود اختصاص می‌دهند و در نتیجه، به اصلی‌ترین بُعد فضا و بازیگر اصلی در محاسبات ریاضی تبدیل می‌شوند.

معیار **TF-IDF** این مشکل را با معرفی پارامتری به نام «**فراوانی معکوس سند**» (**Inverse Document Frequency - IDF**) حل می‌کند. فرمول پایه آن به شکل زیر است:

$$\text{TF-IDF}_{t,d} = \text{TF}_{t,d} \times \log\left(\frac{N}{\text{DF}_t}\right)$$

در این رابطه، N کل تعداد اسناد و DF_t تعداد اسنادی است که کلمه t حداقل 1 بار در آن‌ها آمده است. نقش مکانیزم لگاریتمی در این فرمول بسیار هوشمندانه است: اگر کلمه‌ای در تمام اسناد تکرار شده باشد (مثل یک حرف اضافه)، مقدار N و DF_t برابر شده و کسر برابر با 1 می‌شود. لگاریتم 1 برابر با 0 است؛ بنابراین وزن آن کلمه در تمام فضا 0 می‌شود. در مقابل، کلماتی که در یک سند خاص تکرار بالایی دارند (**TF** بالا) اما در کل مجموعه اسناد نادر هستند (**IDF** بالا)، بیشترین وزن را دریافت می‌کنند.

بخش سوم: تأثیر TF-IDF بر بهبود الگوریتم‌های دسته‌بندی

استفاده از این روش وزن‌دهی، معماری فضای برداری را اصلاح کرده و عملکرد الگوریتم‌ها را به 2 دلیل زیر متحول می‌کند:

1. بهبود چشمگیر در الگوریتم Rocchio (اصلاح مراکز ثقل):

در الگوریتم روکیو، مرکز ثقل ($\vec{\mu}$) هر کلاس با میانگین‌گیری از بردارهای آن کلاس محاسبه می‌شود. اگر از شمارش ساده استفاده کنیم، وزن بالای کلمات عمومی باعث می‌شود که بردارهای تمام کلاس‌ها در ابعاد مربوط به این کلمات عمومی، مقادیر بسیار بزرگی داشته باشند. این امر به صورت هندسی باعث می‌شود تا مراکز ثقل تمام کلاس‌ها به سمت یک نقطه مشترک (مرکز کلمات عمومی) کشیده شوند و در هم تنیده گردند.

اثر TF-IDF: با 0 کردن یا کاهش شدید وزن کلمات عمومی، این «نیروی گرانش کاذب» از بین می‌رود و مراکز ثقل کلاس‌ها در فضای برداری به خوبی از یکدیگر فاصله می‌گیرند (افزایش فاصله بین‌کلاسی)، که این امر رسم مرز تصمیم‌گیری را بسیار دقیق‌تر می‌کند.

2. بهبود چشمگیر در الگوریتم kNN (اصلاح محاسبه شباهت):

الگوریتم **kNN** کاملاً وابسته به محاسبه فاصله یا شباهت (مانند شباهت کسینوسی) بین 2 بردار است. در روش شمارش ساده، 2 سند کاملاً نامرتبب ممکن است صرفاً به دلیل داشتن تعداد زیادی از کلمات مشترک عمومی (مثل حروف ربط)، در فضای برداری به یکدیگر نزدیک ارزیابی شوند. این اتفاق باعث می‌شود همسایگان واقعی یک سند به درستی شناسایی نشوند.

اثر TF-IDF: این روش تضمین می‌کند که زاویه و شباهت بین 2 سند، تنها بر اساس اشتراک آن‌ها در **کلمات کلیدی و تخصصی** سنجیده شود. بنابراین، همسایگی در فضای برداری معنایی واقعی و مفهومی پیدا می‌کند.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: فضای برداری حاصل از **TF** ساده را می‌توان به اتاقی تشبیه کرد که در آن صدای نویز پس‌زمینه (کلمات عمومی) آن‌قدر بلند است که صدای اصلی (کلمات تخصصی) شنیده نمی‌شود. الگوریتم‌های **kNN** و **Rocchio** کورکورانه به این نویزها واکنش نشان می‌دهند و دچار خطای دسته‌بندی می‌شوند. **TF-IDF** در واقع یک «فیلتر حذف نویز معنایی» است که ابعاد زائد و گمراه‌کننده فضا

را کوچک کرده و به الگوریتم‌ها اجازه می‌دهد تا روی ویژگی‌های متمایزکننده واقعی تمرکز کنند. این مرحله در پیش‌پردازش تمام سیستم‌های بازیابی اطلاعات مدرن غیرقابل چشم‌پوشی است.

پاسخنامه مسئله 14.11: چالش کلاس‌های غیرمحدب

(The Challenge of Non-convex Classes)

تحلیل اولیه و هدف مسئله

در دنیای واقعی، بسیاری از دسته‌ها یا کلاس‌های اطلاعاتی ماهیتی «تک‌موضوعی» ندارند. کلاسی مانند «اخبار حوادث» 1 کلاس **غیرمحدب (Non-convex)** یا **چندنمایی (Multimodal)** است؛ به این معنا که از چندین زیرمجموعه کاملاً مجزا (تصادف، آتش‌سوزی، سرقت) تشکیل شده است که هر کدام واژگان تخصصی خود را دارند. هدف این مسئله، تحلیل نقطه ضعف بنیادین الگوریتم **Rocchio** (و سایر روش‌های مبتنی بر مرکز ثقل) در مواجهه با توزیع‌های پراکنده و ناهمگون است.

بخش اول: ریشه مشکل در میانگین‌گیری از مفاهیم ناهمگون

همان‌طور که در مسائل پیشین آموختیم، الگوریتم **Rocchio** کل اسناد 1 کلاس را در قالب 1 بردار واحد به نام **مرکز ثقل (Centroid)** خلاصه می‌کند. این بردار از طریق محاسبه میانگین ریاضی تمامی اسناد آن کلاس به دست می‌آید.

$$\vec{\mu}(c) = \frac{1}{|D_c|} \sum_{d \in D_c} \vec{v}(d)$$

در سناریوی اخبار حوادث، واژگان پرتکرار هر زیرمجموعه به این شکل هستند:

- تصادف:** خودرو، جاده، ترمز، مجروح.
- آتش‌سوزی:** شعله، دود، آتش‌نشان، سوختگی.
- سرقت:** سارق، اسلحه، نقاب، پلیس.

هنگامی که الگوریتم **Rocchio** میانگین بردارهای این 3 گروه کاملاً متفاوت را محاسبه می‌کند، مرکز ثقل به دست آمده شامل وزن‌های پایینی از تمامی این کلمات خواهد بود.

بخش دوم: تفسیر هندسی و تولد «سند فانتوم» (Phantom Document)

از دیدگاه هندسی در فضای برداری، این 3 زیرگروه مانند 3 جزیره کاملاً دور از هم قرار دارند. وقتی شما میانگین مختصات این 3 جزیره را محاسبه می‌کنید، نقطه مرکز ثقل دقیقاً در وسط اقیانوس (1 فضای خالی بین جزیره‌ها) می‌افتد.

این مرکز ثقل محاسبه‌شده، در واقع نماینده سندی است که **در دنیای واقعی وجود خارجی ندارد**. این 1 «سند فانتوم» است؛ خبری که به طور همزمان و با نسبتی مساوی درباره ماشین، دود، اسلحه، و ترمز صحبت می‌کند. از آنجا که این مرکز ثقل در نقطه‌ای خالی از فضای برداری قرار گرفته است، نمی‌تواند نماینده خوبی برای هیچ‌یک از آن 3 جزیره اصلی باشد.

بخش سوم: چرا دسته‌بندی با خطای جدی مواجه می‌شود؟

تصور کنید 1 خبر جدید و خالص درباره «آتش‌سوزی در جنگل» وارد سیستم می‌شود.

فاصله تا کلاس حوادث: این خبر دارای کلمات تخصصی آتش‌سوزی است، اما مرکز ثقل کلاس حوادث (به دلیل رقیق

شدن با کلمات سرقت و تصادف) شباهت زیادی با این خبر خالص نخواهد داشت.

فاصله تا کلاس‌های دیگر: از طرف دیگر، ممکن است این خبر به دلیل داشتن کلماتی مانند «جنگل» و «درخت»، به مرکز

ثقل کلاس منسجم‌تری مانند «محیط زیست» نزدیک‌تر ارزیابی شود.

در نتیجه، الگوریتم **Rocchio** خبر آتش‌سوزی را به اشتباه در کلاس محیط زیست قرار می‌دهد، زیرا مرزهای تصمیم‌گیری خطی

نمی‌توانند کلاس‌های چندپاره را در بر بگیرند.

نتیجه‌گیری: الگوریتم **Rocchio** (و مدل‌های خطی مشابه) تنها زمانی به خوبی کار می‌کنند که کلاس‌ها دارای ویژگی **پیوستگی و**

محدب بودن (Convexity) باشند؛ یعنی تمام اسناد 1 کلاس حول 1 موضوع محوری واحد تجمع کرده باشند.

برای دسته‌بندی اخبار حوادث با موضوعات پراکنده، ما به الگوریتمی مانند **kNN** نیاز داریم. از آنجا که **kNN** نیازی به محاسبه

میانگین و مرکز ثقل ندارد و صرفاً همسایگان محلی را بررسی می‌کند، به راحتی متوجه می‌شود که خبر جدید درباره آتش‌سوزی، در

همسایگی اخبار قدیمی آتش‌سوزی قرار دارد و آن را به درستی در کلاس «حوادث» دسته‌بندی می‌کند.

پاسخنامه مسئله 14.12: تحلیل پیچیدگی زمانی (Time Complexity Analysis)

تحلیل اولیه و هدف مسئله

در طراحی سیستم‌های بازیابی اطلاعات برای مقیاس‌های بزرگ (مانند وب)، دقت دسته‌بندی تنها یکی از معیارهای ارزیابی است.

معیار حیاتی دیگر، هزینه محاسباتی یا «پیچیدگی زمانی» (**Time Complexity**) الگوریتم‌هاست. هدف از این مسئله، مقایسه

هزینه پردازشی 2 الگوریتم **Rocchio** و **kNN** در 2 فاز کاملاً مجزا است: فاز آموزش (**Training**) که سیستم با داده‌های اولیه آشنا

می‌شود، و فاز آزمایش (**Testing**) که سیستم یک سند کاملاً جدید را دسته‌بندی می‌کند.

برای این تحلیل، علاوه بر تعداد اسناد آموزشی (**|D|**) و اندازه واژه‌نامه (**|V|**)، متغیر سومی به نام تعداد کلاس‌ها (**C**) را نیز در نظر

می‌گیریم.

بخش اول: تحلیل پیچیدگی زمانی الگوریتم Rocchio

1. فاز آموزش (Training):

در این فاز، الگوریتم باید مرکز ثقل (**Centroid**) هر کلاس را محاسبه کند. برای این کار، سیستم باید تمامی **|D|** سند موجود در

مجموعه آموزشی را پیمایش کرده و هر سند را که یک بردار با ابعاد **|V|** است، با بردارهای هم‌کلاس خود جمع کند.

پیچیدگی زمانی:

$$O(|D| \cdot |V|)$$

توضیح: این یک فرآیند زمان‌بر است، اما فقط **1 بار** و معمولاً در سرورهای پشتیبان (آفلاین) انجام می‌شود. پس از

محاسبه، مدل نهایی تنها شامل **C** بردار مرکز ثقل خواهد بود.

2. فاز آزمایش (برای یک سند جدید Testing):

وقتی یک سند جدید وارد سیستم می‌شود، **Rocchio** دیگر کاری با اسناد آموزشی ندارد. سیستم تنها باید فاصله این سند جدید (با ابعاد $|V|$) را با تک تک **C** مرکز ثقل محاسبه کند تا نزدیک‌ترین آن‌ها را بیابد.

پیچیدگی زمانی:

$$O(C \cdot |V|)$$

توضیح: از آنجا که تعداد کلاس‌ها (**C**) در مقایسه با تعداد کل اسناد ($|D|$) بسیار کوچک است، این فاز با سرعت فوق‌العاده بالایی (در لحظه) انجام می‌شود.

بخش دوم: تحلیل پیچیدگی زمانی الگوریتم kNN

1. فاز آموزش (Training):

الگوریتم **kNN** در این فاز عملاً هیچ کار محاسباتی خاصی (مانند یافتن میانگین یا رسم خط) انجام نمی‌دهد. تنها کاری که می‌کند، ذخیره کردن داده‌های آموزشی در حافظه است.

پیچیدگی زمانی:

$$O(1)$$

توضیح: مدل در کسری از ثانیه آموزش می‌بیند، زیرا هیچ مدل ریاضیاتی انتزاعی ساخته نمی‌شود.

2. فاز آزمایش (برای یک سند جدید Testing):

فاجعه محاسباتی **kNN** در این مرحله رخ می‌دهد. برای دسته‌بندی تنها یک سند جدید، سیستم مجبور است شباهت کسینوسی (یا فاصله) این سند (با ابعاد $|V|$) را با **تک تک اسناد موجود در مجموعه آموزشی** ($|D|$) محاسبه کند. سپس باید این فواصل را مرتب‌سازی کند تا **k** همسایه نزدیک را بیابد.

پیچیدگی زمانی:

$$O(|D| \cdot |V|)$$

توضیح: اگر مجموعه آموزشی ما 1 میلیون سند داشته باشد، برای هر جستجوی کاربر یا ورود خبر جدید، 1 میلیون بار ضرب داخلی بردارهای طولانی باید انجام شود که به شدت کند و پرهزینه است.

بخش سوم: چرا می‌گوییم kNN یک یادگیرنده «تنبل» (Lazy Learner) است؟

در ادبیات یادگیری ماشین، الگوریتم‌ها به 2 دسته **شتاب‌زده (Eager)** و **تنبل (Lazy)** تقسیم می‌شوند:

Rocchio یک یادگیرنده Eager است: زیرا در مرحله آموزش، تمام انرژی و زمان خود را صرف می‌کند تا یک «مدل کلی و انتزاعی» (مراکز ثقل) بسازد. او کار سخت را از پیش انجام می‌دهد تا در روز امتحان (تست)، سریع پاسخ دهد.

kNN یک یادگیرنده Lazy است: زیرا در فاز آموزش کاملاً «تنبل» است. هیچ مدلی نمی‌سازد، هیچ الگویی را تعمیم نمی‌دهد و صرفاً داده‌ها را به خاطر می‌سپارد (حفظ می‌کند). او تمام بار سنگین پردازشی را به لحظه آخر (زمان تست و ورود داده جدید) موکول می‌کند.

مقایسه کلان

الگوریتم	فاز آموزش (Training)	فاز آزمایش (Testing)	ماهیت یادگیری
Rocchio	$O(D \cdot V)$ (کند)	$O(C \cdot V)$ (بسیار سریع)	Eager (شتاب‌زده)
kNN	$O(1)$ (بسیار سریع)	$O(D \cdot V)$ (بسیار کند)	Lazy (تنبل)

نتیجه‌گیری: در سیستم‌های مدرن وب که روزانه با میلیون‌ها کاربر و درخواست بلادرنگ (**Real-time**) مواجه هستند، سرعتِ فازِ **Testing** بسیار مهم‌تر از سرعتِ فازِ **Training** است. به همین دلیل، با وجود دقت بالای الگوریتم تنبلی مانند **kNN** در داده‌های غیرخطی، استفاده خام از آن در ابعاد وب عملاً غیرممکن است (مگر با استفاده از تکنیک‌های تقریب‌زنی و ساختارهای درختی مانند **kd-trees**). در مقابل، **Rocchio** با وجود سادگی، در محیط‌های عملیاتی سرعت بسیار مطلوبی دارد.

پاسخنامه مسئله 14.13: تفکیک‌پذیری خطی داده‌ها (Linear Separability of Data)

تحلیل اولیه و هدف مسئله

در مسائل قبلی دیدیم که الگوریتم‌های مبتنی بر دسته‌بندی خطی (مانند **Rocchio**) در فضاهاى 2 بعدی روی داده‌های پیچیده و غیرمحدب شکست می‌خورند. با این وجود، در سیستم‌های واقعی پردازش متن، طبقه‌بندهای خطی (مانند **Naive Bayes**، **Logistic Regression** و **SVM** های خطی) به عنوان قدرتمندترین و پرکاربردترین روش‌ها شناخته می‌شوند.

هدف این مسئله، حل این تناقض ظاهری است. ما باید با استفاده از مفاهیم هندسه چندبعدی اثبات کنیم که چرا داده‌هایی که در ابعاد پایین در هم تنیده و غیرقابل تفکیک‌اند، در فضاهاى بسیار بزرگ (مانند فضای متون) تقریباً همیشه به راحتی با یک ابرصفحه (**Hyperplane**) جدا می‌شوند.

بخش اول: مفهوم «درجات آزادی» (Degrees of Freedom)

یک مرز تصمیم‌گیری خطی (ابرفضا) توسط معادله $\vec{w} \cdot \vec{x} + b = 0$ تعریف می‌شود. بردار وزن‌ها $(\vec{w})$ دارای n مؤلفه است (که n برابر با تعداد ابعاد فضا می‌باشد).

- در فضای 2 بعدی:** ابرصفحه یک «خط راست» است و تنها 2 درجه آزادی (شیب و عرض از مبدأ) برای چرخش و جابجایی دارد. اگر داده‌های 2 کلاس به شکل یک ضربدر (مسئله **XOR**) یا 2 دایره تو در تو قرار گیرند، این خط محدود هرگز نمی‌تواند آن‌ها را از هم جدا کند.

- **در فضای n بعدی (متن):** وقتی واژه‌نامه شما 50,000 کلمه دارد، فضای برداری شما 50,000 بعدی است. در اینجا، ابرصفحه دارای **50,000 درجه آزادی** است. این یعنی ابرصفحه می‌تواند در 50,000 جهت مختلف خم شود، بچرخد و زاویه بگیرد تا از میان هر روزنه و شکافی بین داده‌های کلاس **A** و **B** عبور کند. این انعطاف‌پذیری شگفت‌انگیز، شانس یافتن مرز جداکننده را به شدت افزایش می‌دهد.

بخش دوم: پراکندگی فضا و تعامد اسناد (Sparsity and Orthogonality)

دلیل دوم به ذات داده‌های متنی برمی‌گردد. در یک مجموعه داده با 50 هزار کلمه، یک سند معمولی (مثلاً یک خبر) نهایتاً 200 تا 300 کلمه یکتا دارد.

این یعنی در بردار آن سند، 49,700 مؤلفه برابر با **0** است. به این حالت «پراکندگی بسیار بالا» (**High Sparsity**) می‌گویند.

از نظر هندسی، وقتی بردارهای اسناد تا این حد خلوت هستند، اکثر آن‌ها روی محورهای کاملاً متفاوتی قرار می‌گیرند و نسبت به یکدیگر تقریباً **متعامد (Orthogonal)** یا زاویه‌دار (نزدیک به 90 درجه) می‌شوند. در فضایی که نقاط داده (اسناد) این‌قدر از هم دور هستند و فضای خالی بی‌نهایتی بین آن‌ها وجود دارد، عبور دادن یک ابرصفحه مسطح از بین آن‌ها بدون اینکه با هیچ‌کدام برخورد کند، کاری بسیار ساده است.

بخش سوم: قضیه کاور (Cover's Theorem)

از نظر ریاضیات محض، این پدیده توسط **قضیه کاور** اثبات شده است. این قضیه بیان می‌کند:

«یک مسئله طبقه‌بندی الگو که به صورت غیرخطی در یک فضای با ابعاد پایین در هم تنیده است، اگر به یک فضای با ابعاد به اندازه کافی بالا نگاشت شود، با احتمال بسیار بالایی به صورت خطی تفکیک‌پذیر خواهد بود.»

به بیان ساده‌تر، اگر تعداد ابعاد فضا (d) به طور قابل توجهی بزرگتر یا هم‌اندازه با تعداد داده‌های آموزشی (N) باشد، داده‌ها تقریباً به طور قطع تفکیک‌پذیر خطی هستند ($d \geq N$). در پردازش متن، اغلب اندازه واژه‌نامه از تعداد اسناد آموزشی بزرگ‌تر است.

نتیجه‌گیری: به دلیل همین پدیده هندسی است که در مسائل پردازش زبان طبیعی (**NLP**) و طبقه‌بندی متن، استفاده از مدل‌های به شدت پیچیده و غیرخطی (مانند شبکه‌های عصبی عمیق بدون **Regularization**) اغلب باعث بیش‌برازش (**Overfitting**) شدید می‌شود. در فضای ابعاد بالا، 1 مدل خطی ساده هم قدرت تفکیک کافی را دارد و هم از حفظ کردن نویزهای اضافی جلوگیری می‌کند.

پاسخنامه مسئله 14.14: دسته‌بندی انحصاری در برابر چندگانه

(Any-of vs One-of Classification)

تحلیل اولیه و هدف مسئله

یکی از روش‌های استاندارد برای تبدیل یک الگوریتم ذاتاً دوکلاسه (مانند **SVM** کلاسیک یا **Rocchio**) به یک سیستم چندکلاسه، استفاده از تکنیک **One-vs-All** (یا **One-vs-Rest**) است. هدف از این مسئله، درک معماری این تکنیک در سطح کلان و آشنایی با مکانیزم تصمیم‌گیری نهایی در سناریوهایی است که کلاس‌ها مانع‌الجمع (**Mutually Exclusive**) هستند و هر سند تنها باید 1 برچسب دریافت کند.

بخش اول: تعداد طبقه‌بندهای مورد نیاز در روش One-vs-All

در استراتژی **One-vs-All**، معماری سیستم بر اساس تجزیه یک مسئله بزرگ به چندین مسئله کوچکتر و مستقل بنا می‌شود.

قاعده کلی این است: **به ازای هر کلاس موجود در سیستم، دقیقاً به 1 طبقه‌بند باینری (دوکلاسه) مجزا نیاز داریم.**

بنابراین، برای یک سیستم دارای 5 کلاس متمایز (C_1 تا C_5)، ما دقیقاً به **5 طبقه‌بند باینری** نیاز خواهیم داشت.

منطق آموزش (Training) این 5 مدل به شکل زیر است:

طبقه‌بند 1: با داده‌های کلاس 1 به عنوان نمونه‌های مثبت (**Yes**) و داده‌های کلاس‌های 2، 3، 4 و 5 به عنوان

نمونه‌های منفی (**No**) آموزش می‌بیند.

طبقه‌بند 2: با داده‌های کلاس 2 به عنوان نمونه‌های مثبت، و داده‌های کلاس‌های 1، 3، 4 و 5 به عنوان نمونه‌های

منفی آموزش می‌بیند.

... و این روال تا طبقه‌بند 5ام ادامه می‌یابد.

بخش دوم: ترکیب نتایج در حالت One-of

هنگامی که یک سند کاملاً جدید و ناشناخته وارد سیستم می‌شود، چالش اصلی این است که چگونه خروجی این 5 مدل مستقل

را به یک تصمیم واحد و انحصاری تبدیل کنیم. در استراتژی **One-of**، این کار طی یک فرآیند ترتیبی و مشخص انجام می‌پذیرد:

1. اجرای مستقل و استخراج امتیاز

سند جدید d به طور همزمان به هر 5 طبقه‌بند ارسال می‌شود. هر طبقه‌بند به جای خروجی قطعی (بله/خیر)، یک خروجی

پیوسته تولید می‌کند که نشان‌دهنده «میزان اطمینان» (**Confidence Score**) یا احتمال تعلق سند به کلاس تخصیصی خود

است.

2. تشکیل بردار امتیازات

سیستم تمامی این 5 عدد را جمع‌آوری کرده و یک بردار امتیاز تشکیل می‌دهد. (مثلاً: $C_1 = 0.2$ ، $C_2 = 0.85$ ، $C_3 = 0.1$ ،

$C_4 = 0.6$ ، $C_5 = 0.4$).

3. اعمال قانون حداکثر (Argmax)

سیستم تمامی آستانه‌های محلی را نادیده گرفته و صرفاً به دنبال بالاترین عدد می‌گردد. کلاسی که بیشترین امتیاز اطمینان را

کسب کرده باشد، به عنوان تنها برنده و برچسب نهایی سند انتخاب می‌شود.

فرمول ریاضی این فرآیند تصمیم‌گیری بسیار ساده و به شکل زیر است:

$$C^* = \arg \max_{i=1}^5 \text{Score}_i(d)$$

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: شاید بپرسید چرا در مرحله آخر، صرفاً اولین طبقه‌بندی که پاسخ «بله» می‌دهد را انتخاب نمی‌کنیم؟ دلیل آن احتمال

وقوع تضاد است. در عمل، به دلیل پیچیدگی مرزهای تصمیم‌گیری، ممکن است همزمان 2 یا 3 طبقه‌بند ادعا کنند که سند متعلق

به آن‌هاست (مثلاً در مثال بالا هم C_2 و هم C_4 امتیاز بالایی داده‌اند). همچنین ممکن است هیچ‌یک از 5 طبقه‌بند پاسخ قطعی ندهند. استفاده از عملگر **Argmax** این ابهامات را برطرف کرده و همواره به صورت کاملاً رقابتی، محتمل‌ترین کلاس را در سیستم‌های **One-of** انتخاب می‌کند.

پاسخنامه مسئله 14.15: تأثیر پارامتر k بر واریانس (Impact of k on Variance)

تحلیل اولیه و هدف مسئله

در الگوریتم **k-Nearest Neighbors (kNN)**، پارامتر k تنها یک عدد ساده برای تعیین تعداد همسایگان نیست، بلکه **کلید تنظیم پیچیدگی مدل** است. درک رابطه معکوس بین مقدار k و پیچیدگی مدل، برای جلوگیری از خطاهای رایج در دسته‌بندی داده‌ها (به‌ویژه **Overfitting**) کاملاً ضروری است. هدف این مسئله، ترسیم و تحلیل دقیق تغییرات خطای سیستم با تغییر این پارامتر است.

بخش اول: رابطه معکوس k و پیچیدگی مدل

در اکثر الگوریتم‌های یادگیری ماشین، هرچه مدل پیچیده‌تر شود، واریانس بالا رفته و بایاس کاهش می‌یابد. اما در الگوریتم **kNN**، رابطه پارامتر اصلی با پیچیدگی، یک **رابطه معکوس** است:

پیچیدگی بالا ← مقدار k کوچک (مثلاً $k=1$)

پیچیدگی پایین ← مقدار k بزرگ (مثلاً $k=100$)

بنابراین در نمودارهای استاندارد **Bias-Variance** برای **kNN**، اگر محور افقی را مقدار k در نظر بگیریم، با حرکت از چپ (اعداد کوچک) به راست (اعداد بزرگ)، مدل در حال ساده‌تر شدن است.

بخش دوم: کاهش k از 15 به 1 (مسیر ورود به Overfitting)

بیا بیا فرض کنیم در حال دسته‌بندی اسناد با $k=15$ هستیم و تصمیم می‌گیریم این مقدار را به $k=1$ کاهش دهیم. از نظر ریاضی و هندسی اتفاقات زیر رخ می‌دهد:

حالت اول: $k=15$ (نقطه تعادل نسبی)

وقتی سیستم برای تصمیم‌گیری به 15 همسایه نزدیک نگاه می‌کند، در واقع در حال «میانگین‌گیری» یا «هموارسازی» فضا است. اگر 14 از اسناد آموزشی دارای برچسب اشتباه (نویز) باشد، رأی آن در میان 14 رأی دیگر گم می‌شود.

اثر: مرزهای تصمیم‌گیری در فضای برداری نرم و هموار هستند. سیستم نسبت به تغییرات کوچک یا نویزها مقاوم است (واریانس پایین) اما ممکن است الگوهای بسیار ظریف را نبیند (بایاس کمی بالاتر).

حالت دوم: $k=1$ (اوج پیچیدگی و واریانس)

با کاهش k به عدد 1، سیستم کل فرآیند میانگین‌گیری را کنار گذاشته و صرفاً به نزدیک‌ترین تک‌سند همسایه اعتماد می‌کند.

حفظ کردن داده‌ها (Memorization): مدل اکنون تمام داده‌های آموزشی را حفظ کرده است. خطای سیستم روی

داده‌های آموزشی (**Training Error**) دقیقاً به 0 می‌رسد.

افزایش شدید واریانس: مرزهای تصمیم‌گیری به شدت دنداندار و پیچیده می‌شوند تا دورِ تکتک نقاط داده بپیچند.

اگر حتی 1 سند آموزشیِ نويزدار وجود داشته باشد، سیستم 1 «جزیره» کوچکِ اشتباه به دور آن می‌کشد.

پدیده Overfitting (بیش‌برازش): به دلیل همین حساسیت افراطی، سیستم روی داده‌های جدید و ندیده

(Test Data) به شدت دچار خطا می‌شود. مدل به جای یادگیری «مفهوم کلی کلاس»، جزئیات بی‌ارزش و نویزهای

داده‌های آموزشی را یاد گرفته است.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: در دنیای واقعی و پروژه‌های عملی، هرگز نباید از **k=1** استفاده کنید، مگر آنکه داده‌های شما 100% خالص و بدون هیچ‌گونه نویزی باشند (که در واقعیت محال است). یافتن نقطه بهینه (کفِ منحنی **U** شکلِ خطای کل) معمولاً از طریق تکنیکی به نام **اعتبارسنجی متقاطع (Cross-Validation)** روی داده‌های آموزشی انجام می‌شود تا مدلی با تعادل ایده‌آل بین بایاس و واریانس انتخاب گردد.

پاسخنامه مسئله 14.16: بروزرسانی پویای مرکز ثقل (Dynamic Centroid Updating)

تحلیل اولیه و هدف مسئله

در محیط‌های واقعی و عملیاتی مانند موتورهای جستجو، داده‌ها ایستا (**Static**) نیستند؛ بلکه به صورت جریانی پیوسته (**Data Stream**) روزانه به سیستم تزریق می‌شوند. چالش مهندسی در اینجا است که محاسبه مجدد مدل‌ها از نقطه 0، به شدت زمان‌بر و از نظر منابع پردازشی توجیه‌ناپذیر است. هدف این مسئله، آموزش مفهوم «یادگیری برخط» (**Online/Incremental Learning**) در الگوریتم‌های مبتنی بر مرکز ثقل است و نشان می‌دهد که چگونه می‌توان با استفاده از ویژگی‌های جبری میانگین، مدل را با کمترین هزینه محاسباتی به‌روز نگه داشت.

بخش اول: آیا نیاز به پردازش مجدد تمام اسناد است؟

پاسخ قطعی: خیر.

در الگوریتم **Rocchio**، مرکز ثقل 1 کلاس چیزی جز «میانگین حسابی» بردارهای آن کلاس نیست. از نظر ریاضی، برای محاسبه میانگین 1 مجموعه جدید که با مجموعه‌ای قدیمی ترکیب شده است، نیازی به دسترسی به تکتک اعضای مجموعه قدیمی نداریم. تنها کافی است «میانگین قبلی» و «تعداد کل اعضای قبلی» را در حافظه داشته باشیم.

بنابراین، اسناد پردازش‌شده در روزهای گذشته می‌توانند با خیال راحت از حافظه اصلی (**RAM**) پاک یا بایگانی شوند و سیستم برای بروزرسانی کلاس «تکنولوژی»، نیازی به فراخوانی مجدد آن‌ها نخواهد داشت.

بخش دوم: استخراج فرمول بازگشتی (Incremental Update)

برای استخراج این فرمول، ابتدا متغیرهای سیستم را تعریف می‌کنیم:

- N_{old} : تعداد کل اسناد قبلی در کلاس تکنولوژی (که تا دیروز پردازش شده‌اند).
- $\vec{\mu}_{old}$: بردار مرکز ثقل فعلی کلاس تکنولوژی (که در پایگاه داده ذخیره شده است).
- D_{new} : مجموعه اسناد جدیدی که امروز به کلاس اضافه شده‌اند.

• N_{new} : تعداد اسناد جدید (یعنی $|D_{new}|$).

• $\vec{\mu}_{new}$: بردار مرکز ثقل جدید که می‌خواهیم آن را محاسبه کنیم.

گام 1: تعریف پایه مرکز ثقل جدید

مرکز ثقل جدید برابر است با مجموع بردارهای تمام اسناد (قدیم و جدید) تقسیم بر تعداد کل آن‌ها:

$$\vec{\mu}_{new} = \frac{\sum_{d \in D_{old}} \vec{d} + \sum_{d \in D_{new}} \vec{d}}{N_{old} + N_{new}}$$

گام 2: جایگذاری اطلاعات قدیمی

می‌دانیم که میانگین قدیمی طبق این فرمول محاسبه شده بود:

$$\vec{\mu}_{old} = \frac{1}{N_{old}} \sum_{d \in D_{old}} \vec{d}$$

اگر 2 طرف این معادله را در N_{old} ضرب کنیم، مجموع بردارهای قدیمی به دست می‌آید:

$$\sum_{d \in D_{old}} \vec{d} = N_{old} \cdot \vec{\mu}_{old}$$

گام 3: دستیابی به فرمول نهایی بازگشتی

اکنون کافی است عبارت به‌دست‌آمده در گام 2 را در فرمول گام 1 جایگذاری کنیم. فرمول نهایی برای بروزرسانی دسته‌ای

(Batch Update) به شکل زیر خواهد بود:

$$\vec{\mu}_{new} = \frac{N_{old} \cdot \vec{\mu}_{old} + \sum_{d \in D_{new}} \vec{d}}{N_{old} + N_{new}}$$

(نکته: اگر بخواهیم فقط 1 سند جدید $\vec{d}_{new}$ را اضافه کنیم، فرمول حتی ساده‌تر شده و به شکل زیر در می‌آید):

$$\vec{\mu}_{new} = \frac{N_{old} \cdot \vec{\mu}_{old} + \vec{d}_{new}}{N_{old} + 1}$$

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: این فرمول ساده، معماری سیستم‌های بازیابی اطلاعات را متحول می‌کند. از لحاظ مهندسی نرم‌افزار، این فرمول به ما

می‌گوید که برای هر کلاس (مثلاً کلاس تکنولوژی)، پایگاه داده ما تنها نیاز به ذخیره **1 بردار** ($\vec{\mu}_{old}$) و **1 عدد صحیح** (N_{old}) دارد.

از نظر هندسی و فیزیکی، این فرمول نشان‌دهنده 1 «میانگین وزن‌دار» (**Weighted Average**) است. هرچه N_{old} بزرگ‌تر باشد (یعنی کلاس تاریخیچه طولانی‌تری داشته باشد)، بردار $\vec{\mu}_{old}$ وزن و لنگر بیشتری دارد و اسناد جدید نمی‌توانند مرکز ثقل را به راحتی جابه‌جا کنند. این ویژگی باعث پایداری مدل (**Stability**) در برابر نوسانات روزانه و اخبار گذرا می‌شود.

پاسخنامه مسئله 14.17: همسایگی وزن‌دار کسینوسی (Cosine-Weighted kNN)

تحلیل اولیه و هدف مسئله

در نسخه استاندارد الگوریتم **kNN**، تصمیم‌گیری بر اساس «رأی‌گیری اکثریت مطلق» (**Majority Voting**) انجام می‌شود؛ به این معنا که تمامی **k** همسایه انتخاب‌شده، ارزش رأی کاملاً یکسانی (برابر با 1) دارند. این رویکرد 1 نقطه ضعف ساختاری دارد: همسایه‌ای که در لبه مرز همسایگی قرار گرفته و به زحمت جزو **k** سند برتر شده است، دقیقاً همان‌قدر در تعیین کلاس سند جدید نقش دارد که نزدیک‌ترین و شبیه‌ترین همسایه نقش دارد.

هدف این مسئله، ارتقای این الگوریتم به نسخه **Weighted kNN** است، جایی که «ارزش رأی هر سند، متناسب با میزان شباهت آن به سند جدید» تعریف می‌شود.

بخش اول: منطق استفاده از شباهت کسینوسی به عنوان وزن

در فضای برداری متون، شباهت کسینوسی (**Cosine Similarity**) عددی بین 0 (عدم شباهت مطلق) تا 1 (تطابق کامل زاویه‌ای) است. در استراتژی وزن‌دار، قانون «هر همسایه 1 رأی» به «هر همسایه به اندازه شباهت کسینوسی‌اش رأی می‌دهد» تغییر می‌کند.

اگر سند جدیدی با 1 همسایه از کلاس **A** شباهت کسینوسی 0.9 و با همسایه دیگری از کلاس **B** شباهت 0.2 داشته باشد، رأی همسایه اول به مراتب سنگین‌تر از رأی همسایه دوم خواهد بود و این تفاوت به صورت ریاضی در سیستم اعمال می‌شود.

بخش دوم: فرمول ریاضی تصمیم‌گیری

برای استخراج فرمول، متغیرهای زیر را تعریف می‌کنیم:

- $\vec{d}$: بردار سند جدید که قصد دسته‌بندی آن را داریم.
- $N_k(\vec{d})$: مجموعه **k** همسایه نزدیک به سند $\vec{d}$ در مجموعه آموزشی.
- $\vec{x}_i$: بردار i از اسناد همسایه در مجموعه $N_k(\vec{d})$.
- C : مجموعه تمام کلاس‌های موجود در سیستم.
- $I(\vec{x}_i \in c)$: 1 «تابع نشانگر» (**Indicator Function**) که اگر سند $\vec{x}_i$ متعلق به کلاس c باشد، مقدار 1 و در غیر این صورت مقدار 0 را برمی‌گرداند.

گام 1: محاسبه امتیاز (Score) برای هر کلاس

امتیاز کل برای 1 کلاس مشخص مانند c ، برابر است با مجموع شباهت‌های کسینوسی تمام همسایگانی که به آن کلاس تعلق دارند:

$$\text{Score}(c, \vec{d}) = \sum_{\vec{x}_i \in N_k(\vec{d})} \cos(\vec{d}, \vec{x}_i) \cdot I(\vec{x}_i \in c)$$

گام 2: انتخاب کلاس برنده (قانون تصمیم)

در نهایت، سیستم کلاسی را به عنوان برچسب نهایی (c^*) انتخاب می‌کند که بیشترین امتیاز (سنگین‌ترین مجموع آرا) را کسب کرده باشد. این بخش با عملگر آرگ‌ماکس (**Argmax**) نشان داده می‌شود:

$$c^* = \arg \max_{c \in C} \text{Score}(c, \vec{d})$$

بخش سوم: مثال عددی برای درک بهتر مکانیزم

فرض کنید $k=4$ است و می‌خواهیم سند جدیدی را دسته‌بندی کنیم. سیستم، همسایگان زیر را در فضای برداری پیدا کرده است:

همسایه 1 (کلاس A): شباهت کسینوسی = 0.85

همسایه 2 (کلاس B): شباهت کسینوسی = 0.30

همسایه 3 (کلاس B): شباهت کسینوسی = 0.25

همسایه 4 (کلاس B): شباهت کسینوسی = 0.20

در kNN استاندارد (اکثریت): کلاس B دارای 3 رأی و کلاس A دارای 1 رأی است. بنابراین **کلاس B** برنده می‌شود (با وجود اینکه همسایگان کلاس B شباهت بسیار کمی با سند اصلی دارند).

در kNN وزن‌دار کسینوسی:

امتیاز کلاس A = 0.85

امتیاز کلاس B = 0.20 + 0.25 + 0.30 = 0.75

در این حالت، **کلاس A** برنده می‌شود، زیرا تنها همسایه آن آن‌قدر به سند جدید نزدیک و شبیه است که به تنهایی ارزش آرای پراکنده کلاس B را خنثی می‌کند.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: ارتقای الگوریتم به نسخه وزن‌دار (**Weighted kNN**) نه تنها دقت دسته‌بندی را در مرزهای پیچیده و نامنظم به شدت افزایش می‌دهد، بلکه مشکل معروف «شکستن گره» (**Tie-Breaking**) را نیز به طور بنیادین حل می‌کند. از آنجا که احتمال اینکه مجموع وزن‌های اعشاری 2 کلاس دقیقاً با هم برابر شود بسیار نزدیک به 0 است، سیستم دیگر در حالت تساوی آرا گیر نمی‌افتد و نیازی به انتخاب اجباری مقادیر فرد برای پارامتر k نخواهد بود.

پاسخنامه مسئله 14.18: مقایسه مرزهای تصمیم (Comparing Decision Boundaries)

در طراحی سیستم‌های طبقه‌بندی (**Classification**)، شکل و رفتار «مرز تصمیم‌گیری» (**Decision Boundary**) نشان‌دهنده میزان انعطاف‌پذیری و قدرت یادگیری مدل است. هدف از این مسئله، مقایسه هندسی و مفهومی بین 2 الگوریتم پایه است تا دانشجویان درک کنند چرا 1 الگوریتم کل فضا را با 1 خط صاف به 2 نیمه تقسیم می‌کند، در حالی که الگوریتم دیگر مرزهایی پیچیده و جزیره‌ای می‌سازد. این تفاوت، ریشه در نگرش «سراسری» یا «محلی» این الگوریتم‌ها به داده‌ها دارد.

بخش اول: تفاوت ظاهری و هندسی مرزهای تصمیم

اگر بخواهیم مرز تصمیم‌گیری این 2 الگوریتم را در 1 فضای 2 بعدی ترسیم کنیم، تفاوت ظاهری آن‌ها کاملاً بنیادین است:

1. مرز تصمیم در الگوریتم Rocchio:

ظاهر هندسی: همیشه 1 خط کاملاً راست (در 2 بعدی) یا 1 سطح کاملاً صاف و هموار (ابرفضا در فضاهای چندبعدی) است.

ویژگی بارز: این مرز فاقد هرگونه انحنا، شکستگی یا دندان است. فضای برداری را با 1 برش تمیز به 2 (یا چند) ناحیه بزرگ و یکپارچه تقسیم می‌کند. امکان ندارد در این روش شاهد 1 «جزیره» از کلاس **A** در میان دریایی از کلاس **B** باشیم.

2. مرز تصمیم در الگوریتم kNN:

ظاهر هندسی: به شدت پیچیده، دندانه‌دار، نامنظم و پر از انحنا و شکستگی است. (در حالتی که $k=1$ باشد، این مرزها شبکه‌ای از چندضلعی‌های پیچیده به نام «تسل‌بندی ورونوی» می‌سازند).

ویژگی بارز: این مرزها می‌توانند اشکال کاملاً محصور (مانند دایره‌های تو در تو) یا مسیرهای مارپیچ ایجاد کنند. اگر چند داده از کلاس **A** به طور تصادفی در میان داده‌های کلاس **B** افتاده باشند، **kNN** مرز خود را خم می‌کند تا 1 حباب یا جزیره کوچک دور آن نقاط **A** بکشد.

بخش دوم: چرا Rocchio 1 الگوریتم سراسری (Global) است؟

Rocchio را 1 مدل **سراسری (Global)** می‌نامیم زیرا تصمیم‌گیری آن بر اساس 1 «نمای کلان» از تمام داده‌ها انجام می‌شود:

1. **محاسبه میانگین کلان:** برای یافتن مرکز ثقل، **Rocchio** مختصات تمامی اسناد آموزشی 1 کلاس را (چه آن‌هایی که در مرکز تجمع قرار دارند و چه داده‌های پرت در حاشیه) با هم جمع کرده و میانگین می‌گیرد.

2. **حذف هویت فردی اسناد:** پس از محاسبه مرکز ثقل، تمام اسناد آموزشی دور ریخته می‌شوند و مدل تنها 1 «نماینده کلان» برای هر کلاس نگه می‌دارد.

3. **تأثیرپذیری مرز:** مرز تصمیم‌گیری (که عمود منصف بین مراکز ثقل است)، کاملاً تحت تأثیر این نمای سراسری است. مدل به توزیع‌های محلی، خوشه‌های کوچک درون 1 کلاس یا فضاهای خالی نقطه‌ای هیچ اهمیتی نمی‌دهد و 1 قانون کلی و سراسری را بر کل فضا دیکته می‌کند.

بخش سوم: چرا kNN 1 الگوریتم محلی (Local) است؟

الگوریتم **kNN** را 1 مدل **محلّی (Local)** می‌نامیم زیرا هیچ درک کلی و سراسری از مجموعه داده‌ها ندارد و تصمیم‌گیری آن صرفاً بر اساس «همسایگی بلافصل» انجام می‌شود:

1. **فقدان مدل کلان: kNN** هیچ مرکز ثقل یا نماینده‌ای برای کل کلاس نمی‌سازد.
2. **تمرکز بر مقیاس خرد (Micro-scale):** وقتی 1 سند جدید وارد سیستم می‌شود، **kNN** میلیون‌ها سند دیگر را کاملاً نادیده می‌گیرد و تنها ذره‌بین خود را روی 1 ناحیه بسیار کوچک و محلّی (دقیقاً **k** همسایه نزدیک) متمرکز می‌کند.
3. **انعطاف‌پذیری محلّی مرز:** به دلیل همین نگاه نقطه‌ای، مرز تصمیم‌گیری در هر گوشه از فضای برداری می‌تواند متناسب با پراکندگی اسناد در همان ناحیه کوچک، تغییر شکل دهد. اگر در 1 گوشه از فضا تراکم کلاس **A** بیشتر باشد، مرز به نفع **A** خم می‌شود، بدون اینکه این خمیدگی تأثیری روی مرز در سمت دیگر فضا داشته باشد.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: تفاوت بین استراتژی سراسری و محلّی، دقیقاً تفاوت بین مدیریت متمرکز و غیرمتمرکز است. **Rocchio** مانند 1 شهردار است که 1 قانون سراسری (خط مستقیم) برای کل شهر وضع می‌کند که اجرای آن ساده و سریع است اما استثنائات محلّی را نادیده می‌گیرد. در مقابل، **kNN** مانند شورا‌های محله است؛ هر ناحیه کوچک از فضا، قانون خاص خود (مرزهای پیچیده) را بر اساس ساکنان همان محله وضع می‌کند. این ویژگی به **kNN** قدرت بی‌نظیری در یادگیری الگوهای پیچیده و غیرمحدب می‌دهد، اما همزمان آن را به شدت در برابر نویزهای محلّی (داده‌های پرت) آسیب‌پذیر می‌سازد.

پاسخنامه مسئله 14.19: مشکل کم‌برازش در روکیو (Underfitting in Rocchio)

تحلیل اولیه و هدف مسئله

الگوریتم‌های یادگیری ماشین بر اساس میزان پیش‌فرض‌ها و محدودیت‌هایی که برای شکل مرز تصمیم‌گیری قائل می‌شوند، دسته‌بندی می‌گردند. الگوریتم **Rocchio** 1 مدل با **بایاس بالا (High Bias)** است. هدف از این مسئله، درک هندسی این مفهوم است که چگونه پیش‌فرض‌های سخت‌گیرانه و ساده‌انگارانه 1 الگوریتم، باعث ناتوانی آن در یادگیری الگوهای پیچیده شده و سیستم را دچار پدیده **کم‌برازش (Underfitting)** می‌کند.

بخش اول: بایاس بالا به چه معناست؟

در یادگیری ماشین، «بایاس» به معنای تمایل ذاتی 1 مدل به 1 شکل خاص از مرز تصمیم‌گیری، مستقل از شکل واقعی داده‌هاست.

مدل **Rocchio** دارای بایاس بسیار بالاست، زیرا ذاتاً «متعصب» است و پیش‌فرض بسیار قدرتمندی دارد: «**مرز تصمیم‌گیری بین 2 کلاس، همواره باید 1 خط کاملاً راست (یا 1 ابرصفحه کاملاً مسطح) باشد.**»

هنگامی که داده‌های آموزشی در دنیای واقعی پیچیده، دارای انحنا یا تو در تو باشند، **Rocchio** نمی‌تواند خود را با این پیچیدگی تطبیق دهد. مدل به جای اینکه الگوی واقعی داده‌ها را یاد بگیرد، همچنان اصرار دارد که 1 خط راست را از میان آن‌ها عبور دهد. نتیجه این پافشاری ریاضی، خطای بالا هم روی داده‌های آموزشی و هم روی داده‌های تست است که به آن کم‌برازش می‌گویند.

بخش دوم: مثال توزیع داده‌ها در فضای دوبعدی

برای درک بهتر، 1 فضای 2 بعدی را تصور کنید که در آن داده‌های 2 کلاس به شکل **دایره‌های هم‌مرکز (Concentric Circles)** توزیع شده‌اند:

کلاس A (دایره داخلی): مجموعه‌ای از داده‌ها که به صورت متراکم در مرکز محور مختصات تجمع کرده‌اند.

کلاس B (حلقه بیرونی): مجموعه‌ای از داده‌ها که به صورت 1 حلقه کاملاً دورتادور کلاس A را احاطه کرده‌اند.

فرآیند شکست الگوریتم Rocchio در این سناریو:

1. محاسبه مراکز ثقل: مرکز ثقل دایره داخلی (کلاس A) دقیقاً در مرکز شکل می‌افتد. از طرف دیگر، به دلیل تقارن حلقه بیرونی (کلاس B)، میانگین مختصات تمام نقاط آن نیز دقیقاً در **همان نقطه مرکزی** می‌افتد.

2. رسم مرز تصمیم: الگوریتم تلاش می‌کند عمود منصفی بین این 2 مرکز ثقل رسم کند. اما چون هر 2 مرکز ثقل تقریباً روی 1 نقطه افتاده‌اند، مدل دچار سردرگمی ساختاری می‌شود.

3. نتیجه نهایی: حتی اگر الگوریتم 1 خط راست تصادفی از این فضا عبور دهد، این خط در بهترین حالت تنها می‌تواند نیمی از داده‌ها را درست دسته‌بندی کند (دقت حول و حوش 50%). مدل دچار کم‌برازش مطلق شده است زیرا ظرفیت مدل اجازه نمی‌دهد 1 مرز دایره‌ای دور کلاس A بکشد.

نتیجه‌گیری: کم‌برازش زمانی رخ می‌دهد که «ظرفیت و انعطاف‌پذیری مدل» بسیار کمتر از «پیچیدگی مسئله» باشد. در فضاهای با ابعاد پایین (مثل 2 یا 3 بُعد)، الگوریتم Rocchio به شدت مستعد کم‌برازش است. با این حال، به خاطر داشته باشید که در مسائل بازیابی اطلاعات، متون معمولاً دارای هزاران بُعد هستند. همان‌طور که پیش‌تر در قضیه کاور بررسی کردیم، در ابعاد بسیار بالا داده‌ها به قدری پراکنده می‌شوند که معمولاً به صورت خطی تفکیک‌پذیر هستند و مشکل کم‌برازش مدل‌های خطی به طرز چشمگیری کاهش می‌یابد.

پاسخنامه مسئله 14.20: مقدمه‌ای بر نفرین ابعاد (Intro to Curse of Dimensionality)

تحلیل اولیه و هدف مسئله

در یادگیری ماشین، غالباً تصور می‌شود که استخراج ویژگی‌های بیشتر (مانند در نظر گرفتن تمام کلمات یکتای 1 زبان) به معنای تغذیه اطلاعات بیشتر به سیستم و در نتیجه افزایش دقت مدل است. اما این قاعده تنها تا آستانه مشخصی صادق است. با ورود به فضاهای با ابعاد بسیار بالا (فضاهای برداری با 100 ها هزار یا میلیون‌ها بُعد)، قوانین و شهود هندسی ما از فضای 3 بعدی کاملاً در هم می‌شکنند؛ پدیده‌ای که به آن **«نفرین ابعاد» (Curse of Dimensionality)** می‌گویند. هدف از این مسئله، بررسی این پدیده ریاضی و دلیل فروپاشی الگوریتم‌های فاصله‌محور مانند **kNN** در چنین فضاهایی است.

بخش اول: پدیده ریاضی و هندسی

در فضاهای 2 یا 3 بعدی، نقاط به راحتی می‌توانند حول 1 مرکز متراکم شوند (تشکیل خوشه دهند) و یا از هم بسیار دور باشند. در این حالت، مفهوم «نزدیک‌ترین همسایه» کاملاً معنادار و قابل اندازه‌گیری است.

اما با افزایش ابعاد فضا به سمت بی‌نهایت ($d \rightarrow \infty$)، حجم فضا به صورت نمایی (**Exponential**) بزرگ می‌شود. در این فضای به شدت وسیع و بی‌نهایت خالی، نقاط داده (اسناد) چنان از هم دور و پراکنده می‌شوند که فاصله هر سند تصادفی تا

«نزدیک‌ترین همسایه‌اش» ($dist_{min}$) و فاصله آن تا «دورترین سند موجود در مجموعه» ($dist_{max}$)، به تدریج با هم برابر می‌شوند.

از نظر ریاضی، این پدیده تلخ با حد زیر توصیف می‌شود:

$$\lim_{d \rightarrow \infty} \frac{dist_{max} - dist_{min}}{dist_{min}} = 0$$

این معادله به این معناست که در فضاهای ابعاد بالا (مانند متن متراکم نشده)، تمام اسناد تقریباً در فاصله‌ای یکسان از یکدیگر قرار می‌گیرند و هیچ سندی به طور معناداری «نزدیک‌تر» از سند دیگر نیست.

بخش دوم: تأثیر مخرب بر دقت دسته‌بندی kNN

الگوریتم **kNN** 1 مدل ذاتاً **فاصله‌محور (Distance-based)** است. تمام منطق تصمیم‌گیری این الگوریتم روی 1 ستون استوار است: «اسنادی که در فضای هندسی به هم نزدیک‌ترند، از نظر معنایی شبیه‌ترند و احتمالاً در 1 کلاس قرار دارند.»

هنگامی که نفرین ابعاد رخ می‌دهد، این ستون منطقی فرو می‌ریزد و اتفاقات زیر در سیستم رخ می‌دهد:

1. بی‌معنی شدن مفهوم همسایگی: وقتی فاصله نزدیک‌ترین همسایه با فاصله دورترین همسایه تنها کسری از 1

درصد تفاوت دارد، مفهوم «نزدیک‌ترین» ارزش تفکیک‌پذیری خود را از دست می‌دهد.

2. حاکمیت نویزها: در غیاب فواصل معنادار، آن مقدار جزئی تفاوت در فواصل، دیگر ناشی از شباهت معنایی و

ساختاری نیست؛ بلکه صرفاً حاصل نویزهای آماری و کلمات بی‌ربط در بردارهای به شدت خلوت (**Sparse**) است.

3. سقوط به سطح حدس تصادفی: در این شرایط، انتخاب **k** همسایه عملاً تبدیل به 1 انتخاب تصادفی

(**Random Selection**) می‌شود. سیستم کنترل خود را روی داده‌ها از دست داده و دقت (**Accuracy**) مدل به شدت

افت می‌کند؛ تا جایی که خروجی الگوریتم **kNN** با 1 حدس تصادفی تفاوتی نخواهد داشت.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: نفرین ابعاد به ما می‌آموزد که اعمال خام الگوریتم **kNN** (به‌ویژه با معیار فاصله اقلیدسی) روی متن‌های بدون پیش‌پردازش، 1 اشتباه مهندسی مهلك است. برای حل این بحران در سیستم‌های بازیابی اطلاعات، 2 اقدام کاملاً ضروری است:

1. استفاده از شباهت کسینوسی به جای فواصل خطی: شباهت کسینوسی به دلیل تمرکز بر زاویه بردارها به جای طول آن‌ها، مقاومت بسیار بیشتری در برابر فضاهای ابعاد بالا دارد.

2. کاهش ابعاد (Dimensionality Reduction): پیش از اجرای هر الگوریتمی، باید ابعاد فضا را از 1 میلیون کلمه به چند 100 ویژگی معنادار کاهش داد. این کار از طریق تکنیک‌هایی مانند **انتخاب ویژگی** (حذف کلمات توقف و اعمال فیلترهایی مانند **Information Gain**) و یا **استخراج ویژگی** (روش‌های جبری مانند **PCA** و **SVD/LSA**) انجام می‌شود تا اطلاعات متراکم شده و فواصل هندسی مجدداً معنادار شوند.

پاسخنامه مسئله 14.21: بهینه‌سازی جستجو با نمایه وارونه

(Search Optimization with Inverted Index)

تحلیل اولیه و هدف مسئله

در سیستم‌های بازیابی اطلاعات (IR) در مقیاس وب، پیمایش خطی (Linear Scan) کل دیتابیس برای پاسخ به 1 کوئری، غیرممکن است. این عملیات پیچیدگی زمانی $O(|D|)$ دارد که برای 10 میلیون سند، عملاً باعث قفل شدن سیستم می‌شود. هدف از این مسئله، بهره‌گیری از ساختار داده‌ای «نمایه وارونه» (Inverted Index) برای تبدیل جستجو از 1 جستجوی سراسری به 1 جستجوی محلی است که تنها اسناد دارای هم‌پوشانی کلمات با کوئری را بررسی می‌کند.

بخش اول: شبه‌کد الگوریتم جستجوی بهینه‌شده

برای بهینه‌سازی kNN، ما از الگوریتم Accumulator استفاده می‌کنیم. در این رویکرد، به جای بررسی کل دیتابیس، تنها اسنادی که در «لیست‌های ارسال» (Posting Lists) کلمات موجود در کوئری ظاهر شده‌اند، بررسی می‌شوند.

```
ALGORITHM Optimized-kNN-Search(Query Q, InvertedIndex Index, k)
    // Initialize a hash map to store scores of potentially relevant documents
    // Key: DocID, Value: SimilarityScore
    Scores = EMPTY-HASH-MAP

    // Step 1: Preprocessing
    QueryTerms = Tokenize(Q)

    // Step 2: Inverted Index Traversal
    FOR EACH Term t IN QueryTerms:
        // Get the posting list: list of documents containing term 't'
        PostingList = Index.GetPostingList(t)

    FOR EACH (DocID, Weight) IN PostingList:
        // Calculate partial similarity (e.g., Dot Product)
        // Sim(Q, D) = Sum of (QueryWeight_t * DocWeight_t)
        PartialSim = CalculatePartialScore(t, Weight)

    // Accumulate score for this document
    Scores[DocID] = Scores[DocID] + PartialSim

    // Step 3: Result Extraction
    // Only sort the documents that were touched (the relevant subset)
    TopK = ExtractTopK(Scores, k)

    RETURN TopK
```

توضیح مراحل:

1. **پیمایش انتخابی:** ما فقط به سراغ کلماتی می‌رویم که کاربر جستجو کرده است. اگر کاربر «الگوریتم‌های ژنتیک» را

جستجو کند، اسنادی که فاقد این 2 کلمه هستند، هرگز لمس نمی‌شوند.

2. **تجمع‌کننده (Accumulator):** نقشه Scores تنها برای اسناد "کاندید" ساخته می‌شود. این یعنی فضای حافظه

مصرفی ما متناسب با تعداد اسناد مرتبط است، نه کل دیتابیس.

بخش دوم: تحلیل پیچیدگی زمانی

تفاوت عملکرد در این 2 رویکرد، 1 تفاوت مرتبه بزرگی (Order of Magnitude) است.

معیار	جستجوی خطی (Naive)	جستجوی بهینه‌شده (Inverted Index)
فضای جستجو	تمام اسناد ($ D $)	فقط اسناد مرتبط ($ D_{relevant} $)
پیچیدگی	$\mathcal{O}(D \cdot V)$	$\mathcal{O}(\sum_{t \in Q} P_t)$

- در اینجا $|P_t|$ طول لیست ارسال (**Posting List**) برای کلمه t است.
- از آنجا که اکثر اسناد در فضای متنی دارای محتوای پراکنده (**Sparse**) هستند، در بسیاری از کوئری‌ها، تعداد اسناد مرتبط بسیار کمتر از تعداد کل اسناد است ($|D_{relevant}| \ll |D|$).

نتیجه کاهش پیچیدگی:

در جستجوی خطی، با اضافه شدن هر سند جدید، زمان جستجو افزایش می‌یابد. اما در الگوریتم بهینه‌شده، پیچیدگی تنها به **طول لیست‌های ارسال کلمات کوئری** وابسته است. اگر دیتابیس از 10 میلیون به 100 میلیون سند افزایش یابد، اما تعداد اسناد مرتبط با کوئری تغییری نکند، زمان جستجو در الگوریتم بهینه‌شده تقریباً ثابت باقی می‌ماند.

جمع‌بندی آموزشی

نکته کلیدی: الگوریتم‌های مبتنی بر بردار (مانند **kNN** و **Rocchio**) زمانی که با **ساختار داده‌ای مناسب (Inverted Index)** ترکیب می‌شوند، از فضای «عددی مطلق» به فضای «جستجوی معنایی» کوچ می‌کنند. این تکنیک، ستون فقرات عملکرد موتورهای جستجوی مدرن است. در سیستم‌های واقعی، ما حتی 1 قدم فراتر می‌رویم و لیست‌های ارسال را بر اساس وزن (**Weight**) یا اهمیت (**PageRank**) مرتب می‌کنیم تا بتوانیم پیش از بررسی تمام اسناد مرتبط، جستجو را قطع کرده و تنها بهترین‌ها را استخراج کنیم (**Pruning**).

پاسخنامه مسئله 14.22: معماری سیستم و مبادله بایاس-واریانس

(System Architecture and Bias-Variance Tradeoff)

تحلیل اولیه و هدف مسئله

در طراحی سیستم‌های هوش مصنوعی برای مقیاس‌های صنعتی (مانند پلتفرم‌های تجارت الکترونیک با میلیون‌ها نظر روزانه)، داده‌ها هرگز ایده‌آل، تمیز و بدون خطا نیستند. نظرات کوتاه کاربران پر از نویزهای متنی (مانند غلط‌های املائی، زبان محاوره، کنایه و طعنه) است. هدف این مسئله، ارزیابی توانایی شما در اتصال مفاهیم انتزاعی (مانند بایاس و واریانس) به تصمیم‌گیری‌های کلان معماری نرم‌افزار است تا نشان دهیم چرا 1 الگوریتم از نظر تئوری ممکن است جذاب باشد، اما در محیط عملیاتی شکست بخورد.

الف) رفتار الگوریتم 1-NN در برابر داده‌های نویزی

استفاده از الگوریتم **1-NN** (نزدیک‌ترین همسایه با $k=1$) در 1 محیط به شدت نویزی، **بدترین تصمیم ممکن** معماری است. رفتار این سیستم از منظر مبادله بایاس و واریانس به شرح زیر تحلیل می‌شود:

1. پروفایل بایاس و واریانس:

الگوریتم **1-NN** دارای **بایاس بسیار پایین** (مدل هیچ پیش‌فرضی درباره شکل مرز تصمیم‌گیری ندارد) و **واریانس بی‌نهایت بالا** (مدل به شدت به تک‌تک نقاط داده حساس است) می‌باشد.

2. حفظ کردن نویزها (Memorization of Noise):

در **1-NN**، سیستم به جای یادگیری «مفهوم کلی رضایت یا نارضایتی»، تمام داده‌های آموزشی از جمله غلط‌های املایی نادرست و نظرات طعنه‌آمیز را به عنوان قوانین قطعی حفظ می‌کند. اگر 1 کاربر به طعنه نوشته باشد: "عالی بود، دقیقاً همون چیزی که می‌خواستم تا بندازمش دور!" و سیستم این نظر را به عنوان نویز در کلاس مثبت داشته باشد، **1-NN** 1 مرز تصمیم‌گیری (1 جزیره کوچک) دقیقاً دور این نظر کنایه‌آمیز رسم می‌کند.

3. نتیجه نهایی (Overfitting):

سیستم دچار بیش‌برازش (**Overfitting**) فاجعه‌بار می‌شود. وقتی نظر جدیدی با 1 غلط املایی تصادفی وارد سیستم می‌شود، اگر بردار آن در فضای متنی بر حسب تصادف به 1ی از این نویزهای آموزشی نزدیک شود، سیستم بر اساس همان 1 نویز تصمیم‌گیری کرده و خروجی کاملاً اشتباهی تولید می‌کند.

ب) پیشنهاد معماری: روکیو یا kNN با k بزرگ؟

به عنوان معمار سیستم، در این شرایط نویزی و مقیاس بالا، قطعاً **الگوریتم Rocchio** (یا طبقه‌بندهای خطی مشابه) پیشنهاد می‌شود. دلایل این انتخاب استراتژیک در 2 بعد کیفیت یادگیری و هزینه عملیاتی قابل بررسی است:

1. تحلیل از منظر مقاومت در برابر نویز (فیلتر کردن از طریق میانگین)

مزیت Rocchio (بایاس بالا، واریانس پایین): در روکیو، ما برای کلاس «نظرات مثبت» و «نظرات منفی» تنها 1 **مرکز ثقل (Centroid)** می‌سازیم. این مرکز ثقل حاصل میانگین‌گیری 10ها هزار نظر است. در فرآیند میانگین‌گیری، کلمات پرتکرار و اصلی احساسی (مثل "خوب"، "خراب"، "ارزش") وزن بسیار بالایی می‌گیرند، در حالی که غلط‌های املایی نادر (مثل "خووووب") یا کنایه‌های پراکنده، وزنشان در میان هزاران داده صحیح تقسیم بر عدد بزرگی شده و عملاً به سمت 0 میل می‌کند. روکیو ذاتاً 1 «فیلتر پایین‌گذر» است که نویزها را محو می‌کند.

ضعف kNN با k بزرگ: هرچند افزایش **k** (مثلاً **k=100**) باعث کاهش واریانس و هموار شدن مرزهای تصمیم‌گیری می‌شود و اثر نویزها را تا حد زیادی با رأی‌گیری اکثریت خنثی می‌کند، اما این الگوریتم همچنان نیازمند نگهداری تمام آن داده‌های نویزی در حافظه است.

2. تحلیل از منظر پیچیدگی زمانی و مقیاس‌پذیری (Scalability)

دیجی‌کالا روزانه 100ها هزار نظر جدید دریافت می‌کند که باید در کسری از ثانیه (**Real-time**) پردازش شوند:

فاجعه kNN در مقیاس صنعتی: الگوریتم **kNN** 1 مدل **Lazy** است. برای تحلیل احساسات تنها 1 نظر جدید، سیستم باید شباهت کسینوسی آن نظر را با **تک‌تک میلیون‌ها نظر موجود در دیتابیس آموزشی** محاسبه کند. این کار نیازمند سرورهای عظیم و زمان پردازش بالاست که برای 1 سیستم **Real-time** کاملاً غیرمنطقی است.

سرعت شگفت‌انگیز Rocchio: در روکیو، تمام فاز آموزش به صورت آفلاین انجام شده و مدل نهایی مستقر روی سرور، تنها شامل **2 بردار** است (مرکز ثقل مثبت و مرکز ثقل منفی). تحلیل 1 نظر جدید تنها نیازمند 2 ضرب داخلی برداری است (مستقل از اینکه دیتابیس شما 10 هزار نظر دارد یا 10 میلیون). پیچیدگی زمانی از $O(|D| \cdot |V|)$ به $O(|V|)$ کاهش می‌یابد.

نتیجه‌گیری: طراحی سیستم‌های بازیابی اطلاعات و پردازش زبان طبیعی همواره هنری در برقراری تعادل است. روی کاغذ و در مقیاس کوچک آزمایشگاهی، **kNN** با تنظیم دقیق پارامتر **k** ممکن است مرزهای پیچیده‌تر و دقت بالاتری نسبت به خط راست **Rocchio** داشته باشد. اما در **مهندسی نرم‌افزار واقعی**، فاکتورهایی مانند «تحمّل نویز از طریق تعمیم خطی» و مهم‌تر از آن «پیچیدگی زمانی $O(1)$ نسبت به حجم داده‌ها در زمان تست»، طبقه‌بندهای خطی (مانند **Rocchio**، **Naive Bayes** و **Linear SVM**) را به حاکمان بلامنازع پردازش متن در مقیاس وب تبدیل کرده است.

پاسخنامه مسئله 14.23: روکیو در توزیع‌های چندوجهی (Rocchio in Multimodal Distributions)

تحلیل اولیه و هدف مسئله

در مسائل پیشین (مانند چالش اخبار حوادث در مسئله 14.11) دیدیم که الگوریتم **Rocchio** در مواجهه با کلاس‌های چندپاره (**Multimodal**) 1 «سند فانتوم» می‌سازد که در فضای خالی بین خوشه‌ها قرار می‌گیرد. اما این سناریو، 1 قدم جلوتر می‌رود و نشان می‌دهد که اگر در آن «فضای خالی»، کلاسی دیگر (در اینجا کلاس اقتصادی) جا خوش کرده باشد، چگونه 1 فاجعه طبقه‌بندی سیستماتیک رخ می‌دهد. این مسئله 1ی از زیباترین اثبات‌های هندسی برای نشان دادن نقاط کور الگوریتم‌های مبتنی بر مرکز ثقل (**Centroid-based**) است.

بخش اول: مدل‌سازی هندسی مسئله در فضای تک‌بعدی (1D Space)

برای ساده‌سازی اثبات هندسی بدون از دست دادن دقت، فرض می‌کنیم اسناد در 1 فضای تک‌بعدی روی محور **X** توزیع شده‌اند (محور **X** را به عنوان 1 بُعد معنایی تصور کنید که از سیاست داخلی به سمت اقتصاد و سپس سیاست خارجی حرکت می‌کند).

1. موقعیت‌دهی به کلاس‌ها (فرضیات هندسی):

کلاستر اول سیاسی (سیاست داخلی): مجموعه‌ای متراکم از اسناد سیاسی که حول نقطه **X = -10** قرار دارند.

کلاستر دوم سیاسی (سیاست خارجی): مجموعه‌ای متراکم از اسناد سیاسی که در منتهی‌الیه دیگر، حول نقطه **X = +10** قرار دارند.

کلاستر اقتصادی: مجموعه‌ای از اسناد اقتصادی که دقیقاً در حد فاصل این 2، مثلاً حول نقطه **X = -4** قرار گرفته‌اند (نزدیک‌تر به سیاست داخلی به دلیل مفاهیمی مانند اقتصاد سیاسی و تورم داخلی).

بخش دوم: اثبات ریاضی و بروز خطای فاحش

برای اثبات خطای **Rocchio**، باید رفتار سیستم را در زمان ورود 1 سند کاملاً مشخص بررسی کنیم.

گام 1: محاسبه مراکز ثقل (Centroids)

مرکز ثقل کلاس سیاسی ($\vec{\mu}_P$): از آنجا که روکیو 1 الگوریتم سراسری (**Global**) است، میانگین تمام اسناد سیاسی را می‌گیرد. نقطه میانی بین کلاستر داخلی (-10) و کلاستر خارجی (+10)، دقیقاً روی مبدأ مختصات می‌افتد.

$$\vec{\mu}_P = \frac{(-10) + (+10)}{2} = 0$$

بنابراین مرکز ثقل کلاس سیاسی در $X = 0$ قرار گرفت.

مرکز ثقل کلاس اقتصادی ($\vec{\mu}_E$): چون این کلاس یک‌تکه (Convex) است، مرکز ثقل آن دقیقاً در مرکز تجمع خودش می‌افتد.

بنابراین مرکز ثقل کلاس اقتصادی در $X = -4$ قرار می‌گیرد.

گام 2: ورود 1 سند جدید به خالص‌ترین نقطه کلاس سیاسی

فرض کنید امروز 1 خبر کاملاً مرتبط با سیاست داخلی (مثلاً درباره انتخابات مجلس) وارد سیستم می‌شود و دقیقاً روی نقطه $X = -10$ قرار می‌گیرد. (این سند درست در قلب کلاستر سیاست داخلی است).

گام 3: محاسبه فواصل و تصمیم‌گیری روکیو

سیستم روکیو برای دسته‌بندی این خبر، فاصله آن را با مراکز ثقل محاسبه می‌کند:

فاصله خبر تا مرکز ثقل سیاسی ($\vec{\mu}_P = 0$): برابر است با **10 واحد**.

فاصله خبر تا مرکز ثقل اقتصادی ($\vec{\mu}_E = -4$): برابر است با **6 واحد** (فاصله بین -10 و -4).

نتیجه‌گیری اثبات: از آنجا که $10 > 6$ است، الگوریتم روکیو این سند کاملاً سیاسی را با اطمینان بالا به عنوان **1 خبر اقتصادی** دسته‌بندی می‌کند!

نکته کلیدی آموزشی: این مثال به زیبایی نشان می‌دهد که چرا در مدل‌های مبتنی بر مرکز ثقل، وجود 1 کلاس چندوجهی، فقط به خودش آسیب نمی‌زند؛ بلکه شعاع تخریب آن باعث ایجاد «نقاط سیاه» در فضای برداری می‌شود که به اشتباه سایر کلاس‌های همجوار را می‌بلعد.

پاسخنامه مسئله 14.24: تحلیل هندسی موزائیک‌بندی ورونوی

(Geometrical Analysis of Voronoi Tessellation)

تحلیل اولیه و هدف مسئله

یکی از زیباترین مفاهیم در هندسه محاسباتی که مستقیماً با یادگیری ماشین گره خورده است، «تسل‌بندی یا موزائیک‌بندی ورونوی» (**Voronoi Tessellation**) است. الگوریتم **1-NN** در فاز تصمیم‌گیری خود، دقیقاً در حال ترسیم 1 دیاگرام ورونوی پنهان در فضای برداری است. هدف این مسئله، درک ساختار هندسی این دیاگرام برای ساخت مرزهای تصمیم، و سپس مشاهده اثر ریاضی افزایش پارامتر **k** بر تغییر شکل (**Deformation**) و هموارسازی (**Smoothing**) این مرزهاست.

بخش اول: ترسیم مرز تصمیم در 1-NN (موزائیک‌بندی ورونوی)

فرض کنید در 1 فضای 2 بعدی، 6 سند (3 عدد از کلاس **A** و 3 عدد از کلاس **B**) داریم. اگر بخواهیم مرز تصمیم الگوریتم **1-NN** را رسم کنیم، سیستم فضا را به روش زیر قطعه‌قطعه می‌کند:

1. ساخت چندضلعی‌های انحصاری (Voronoi Cells):

سیستم بین هر جفت از نقاط مجاور (فارغ از اینکه چه کلاسی دارند)، 1 خط «عمود منصف» رسم می‌کند. تقاطع این عمود منصف‌ها، فضا را به 6 چندضلعی محدب (Convex Polygon) تقسیم می‌کند.

هر چندضلعی تنها حاوی **1 نقطه داده** است. خاصیت ریاضی این چندضلعی (که به آن سلول ورونوی می‌گویند) این است: «هر مختصاتی که درون این چندضلعی قرار داشته باشد، قطعاً و بدون شک به نقطه مرکزی همان سلول نزدیک‌تر است تا به هر نقطه دیگری در کل فضا.»

(تکمیل فرمول ریاضی سلول ورونوی برای دانشجویان): اگر مجموعه نقاط داده‌های ما $P = \{p_1, p_2, \dots, p_n\}$ باشد، منطقه یا سلول ورونوی R_k مرتبط با نقطه p_k با فرمول دقیق زیر توصیف می‌شود:

$$R_k = \{x \in X \mid d(x, p_k) \leq d(x, p_j) \text{ for all } j \neq k\}$$

2. استخراج مرز تصمیم قطعی:

اکنون که فضا به 6 قطعه تقسیم شد، مرز تصمیم‌گیری **1-NN** به سادگی به دست می‌آید:

تمام اضلاعی که 2 سلول هم‌کلاس (مثلاً **A** با **A**) را از هم جدا می‌کنند، نادیده گرفته می‌شوند. اما **اضلاعی که 1 سلول از کلاس A را از 1 سلول از کلاس B جدا می‌کنند، پررنگ شده و به هم متصل می‌شوند.** این خطوط شکسته و دندان‌دار به هم پیوسته، مرز تصمیم نهایی الگوریتم **1-NN** را تشکیل می‌دهند.

بخش دوم: افزایش k به 3 و پدیده هموارسازی (Smoothing)

هنگامی که پارامتر را از **k=1** به **k=3** افزایش می‌دهیم، قانون «حاکمیت مطلق هر نقطه بر سلول خودش» باطل می‌شود. اکنون تصمیم‌گیری در هر نقطه از فضا نیازمند رأی‌گیری از 3 نقطه نزدیک‌تر است. این تغییر، هندسه مرز را به شکل زیر دگرگون می‌کند:

1. محو شدن شبه‌جزیره‌ها و مرزهای تیز:

در دیاگرام ورونوی (**1-NN**)، ممکن است 1 نقطه از کلاس **A** به دلیل موقعیتش، 1 «شبه‌جزیره» تیز و باریک در میان نقاط کلاس **B** ایجاد کرده باشد. در حالت **3-NN**، اگر 1 سند جدید دقیقاً در لبه این شبه‌جزیره بیفتد، اگرچه نزدیک‌ترین نقطه‌اش کلاس **A** است، اما دومین و سومین نقطه نزدیکش از کلاس **B** خواهند بود. با رأی‌گیری (2 به 1 به نفع **B**)، آن بخش از فضای تحت حاکمیت **A** به کلاس **B** واگذار می‌شود.

2. هموار شدن مرز (Boundary Smoothing):

مرزهای تصمیم در **3-NN** دیگر لزوماً منطبق بر اضلاع دیاگرام اولیه ورونوی نیستند. این مرزها عقب‌نشینی کرده و زوایای تیز و شکستگی‌های شدید خود را از دست می‌دهند. مرز جدید به شکلی «نرم‌تر» و کلی‌تر 2 کلاس را از هم جدا می‌کند و از دنبال کردن تک‌تک نویزها یا نقاط پراکنده دست برمی‌دارد.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: مرز تصمیم‌گیری در **1-NN** دقیقاً 1 **مرز چندضلعی (Polygonal)** است که از اضلاع دیاگرام ورونوی تشکیل می‌شود و به همین دلیل به شدت مستعد بیش‌برازش (**Overfitting**) است. با افزایش **k**، ما در واقع در حال اعمال 1 «فیلتر پایین‌گذر مکانی»

پاسخنامه مسئله 14.25: طبقه‌بندی سلسله‌مراتبی در مقیاس بالا

(Large-Scale Hierarchical Classification)

تحلیل اولیه و هدف مسئله

در پروژه‌های در ابعاد وب (مانند ویکی‌پدیا)، سیستم با 1 ساختار طبقه‌بندی تخت (Flat Classification) روبرو نیست، بلکه با 1 درخت عظیم از مفاهیم مواجه است. اگر بخواهیم 1 مقاله جدید را با استفاده از الگوریتم **kNN** تخت در میان 1000 کلاس مختلف دسته‌بندی کنیم، سیستم دچار فروپاشی محاسباتی می‌شود؛ زیرا باید فاصله مقاله را با میلیون‌ها سند موجود در تمام این 1000 کلاس بسنجد. هدف این مسئله، طراحی 1 معماری ترکیبی (Hybrid) است که از **سرعت خطی Rocchio** برای پیمایش در بالای درخت و از **دقت غیرخطی kNN** برای تصمیم‌گیری در برگ‌های درخت استفاده کند.

بخش اول: معماری سیستم ترکیبی (Rocchio + kNN)

این سیستم بر اساس استراتژی «تقسیم و غلبه» (Divide and Conquer) عمل می‌کند. درخت طبقه‌بندی ویکی‌پدیا را در نظر بگیرید: گره‌های بالای درخت شامل دسته‌های بسیار کلی و کلان (مانند علوم، هنر، ورزش) هستند و گره‌های پایین درخت (برگ‌ها) شامل زیردسته‌های بسیار تخصصی (مانند فیزیک کوانتوم، مکانیک سیالات، هنر مینیاتور) می‌باشند.

1. پیمایش کلان در ریشه (Top-Level Routing) – الگوریتم Rocchio:

وقتی سند جدیدی وارد می‌شود، ابتدا در بالاترین سطح درخت پردازش می‌گردد. در این سطح، ما برای هر دسته کلان (مثلاً علوم، هنر، ورزش) تنها 1 مرکز ثقل (Centroid) از پیش محاسبه شده داریم. سیستم با استفاده از الگوریتم **Rocchio**، با چند ضرب داخلی ساده متوجه می‌شود که این سند مثلاً متعلق به شاخه «علوم» است.

2. پیمایش میانی (Intermediate Routing) – الگوریتم Rocchio:

سند به گره «علوم» فرستاده می‌شود. این گره به نوبه خود دارای زیردسته‌هایی مانند «فیزیک»، «شیمی» و «زیست‌شناسی» است. مجدداً **Rocchio** وارد عمل شده و با مقایسه سند با مرکز ثقل این 3 زیردسته، سند را به شاخه «فیزیک» هدایت می‌کند.

3. تصمیم‌گیری نهایی در برگ‌ها (Leaf-Level Classification) – الگوریتم kNN:

اکنون سند به گره «فیزیک» رسیده است که حاوی برگ‌های نهایی و به شدت تخصصی (مانند فیزیک کوانتوم، ترمودینامیک، فیزیک هسته‌ای) است. در این سطح متراکم و پیچیده، **Rocchio** کنار رفته و الگوریتم **kNN** فعال می‌شود. سیستم اکنون فاصله سند جدید را تنها با اسناد آموزشی موجود در زیرشاخه‌های فیزیک مقایسه می‌کند تا کلاس نهایی و دقیق مقاله را استخراج کند.

بخش دوم: اثبات منطق بهینگی (چرا این معماری بی‌نقص است؟)

این طراحی ترکیبی از 2 منظر مهندسی به شدت بهینه است و نقاط ضعف هر 2 الگوریتم را پوشش می‌دهد:

1. بهینگی محاسباتی (حل مشکل کندی kNN):

اگر کل دیتابیس ویکی‌پدیا شامل 10 میلیون مقاله باشد، اجرای **kNN** در حالت تخت نیازمند 10 میلیون محاسبه شباهت است (پیچیدگی $O(|D|)$).

اما در مدل سلسله‌مراتبی، الگوریتم **Rocchio** در مراحل اول و دوم با پیچیدگی بسیار ناچیز $O(1)$ (تنها مقایسه با چند مرکز ثقل)، بخش عظیمی از دیتابیس (مثلاً مقالات هنر و ورزش و شیمی) را **هرس (Prune)** می‌کند.

زمانی که نوبت به **kNN** می‌رسد، فضای جستجو تنها محدود به مقالات «فیزیک» (مثلاً 50 هزار مقاله) شده است. بدین ترتیب بار محاسباتی **kNN** به کسری از 1 درصد کاهش می‌یابد.

2. بهینگی در دقت دسته‌بندی (حل مشکل کم‌برازش Rocchio):

همان‌طور که در مسائل قبل دیدیم، **Rocchio** در کلاس‌های غیرمحدب، چندوجهی و دارای مرزهای پیچیده (**High Bias**) شکست می‌خورد.

- در بالای درخت، مفاهیمی مثل «ورزش» و «فیزیک» از نظر واژگان کاملاً متفاوت و دور از هم هستند؛ بنابراین مرزهای خطی **Rocchio** به راحتی و با دقت بالا آن‌ها را از هم جدا می‌کنند.
- اما در پایین درخت، تفاوت بین «فیزیک هسته‌ای» و «فیزیک ذرات بنیادی» بسیار ظریف و در هم تنیده است. مرز خطی **Rocchio** در اینجا باعث خطای فاحش می‌شود. در این نقطه بحرانی، ما کار را به **kNN** می‌سپاریم که با مرزهای غیرخطی و محلی خود (**Low Bias**)، این الگوهای ظریف را با دقت تمام تفکیک می‌کند.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: طراحی سیستم‌های مقیاس‌پذیر در مهندسی نرم‌افزار، نیازمند فراتر رفتن از «1 الگوریتم واحد» است. ترکیب **Rocchio** (به عنوان 1 مسیر یابِ پرسرعتِ سراسری) و **kNN** (به عنوان 1 متخصصِ دقیقِ محلی)، 1ی از درخشان‌ترین الگوهای طراحی (**Design Patterns**) در توسعه موتورهای جستجو و سیستم‌های توصیه‌گر محتوا است.

پاسخنامه مسئله 14.26: رفتار طبقه‌بندها در فضای خلوت ابعاد بالا

(Classifiers Behavior in High-Dimensional Sparse Space)

تحلیل اولیه و هدف مسئله

در سیستم‌های بازیابی اطلاعات مدرن، ما با واژه‌نامه‌هایی سروکار داریم که 10ها یا 100ها هزار کلمه یکتا (ابعاد) دارند. وقتی 1 سند متنی خام (مثلاً 1 خبر یا ایمیل) به بردار تبدیل می‌شود، معمولاً حاوی کسر بسیار کوچکی از این کلمات است (مثلاً 200 کلمه از واژه‌نامه 100 هزار کلمه‌ای). این یعنی **بیش از 99% از درایه‌های بردار سند، مقدار 0 دارند**.

به این پدیده، **خلوت بودن فضا یا پراکندگی (Sparsity)** می‌گویند. هدف از این مسئله، درک این موضوع است که چگونه این ویژگی ظاهراً منفی (فضای عظیم و خالی)، به طور شگفت‌انگیزی به عنوان 1 کاتالیزور قدرتمند برای سرعت بخشیدن به محاسبات ریاضی (به‌ویژه ضرب داخلی و شباهت کسینوسی) عمل می‌کند.

بخش اول: تحلیل ریاضی حذف درایه‌های 0

برای محاسبه شباهت بین 2 سند A و B ، متداولترین روش استفاده از ضرب داخلی (**Dot Product**) است. فرمول خام و پایه‌ای ضرب داخلی برای فضایی با واژه‌نامه V به این شکل است:

$$\vec{A} \cdot \vec{B} = \sum_{i=1}^{|V|} A_i \times B_i$$

در 1 رویکرد سنتی و ساده‌انگارانه (**Naive**)، کامپیوتر باید 1 حلقه تکرار (**Loop**) با 100 هزار گام اجرا کند تا تک‌تک مؤلفه‌ها را در هم ضرب کرده و جمع کند. اما به دلیل ویژگی **پراکندگی (Sparsity)**، ما 1 قانون ساده جبری داریم:

هر عددی ضرب در 0 برابر با 0 است.

اگر کلمه i ام در سند A وجود نداشته باشد ($A_i = 0$) و یا در سند B وجود نداشته باشد ($B_i = 0$)، حاصل‌ضرب آن‌ها 0 خواهد بود و هیچ تأثیری در جمع نهایی نخواهد داشت. درایه‌ها تنها در صورتی مقداری غیر از 0 تولید می‌کنند که کلمه i ام به‌طور همزمان در هر 2 سند وجود داشته باشد.

بنابراین، فرمول بالا از نظر محاسباتی به این فرمول بسیار بهینه‌تر تقلیل می‌یابد:

$$\vec{A} \cdot \vec{B} = \sum_{i \in (A \cap B)} A_i \times B_i$$

در این فرمول اصلاح‌شده، ما به جای پیمایش کل واژه‌نامه (V)، تنها روی **اشتراک کلمات 2 سند ($A \cap B$)** پیمایش می‌کنیم.

بخش دوم: مزیت محاسباتی پنهان و ساختار داده مناسب

این تغییر فرمول، 1 جهش عظیم در پرفورمنس (**Performance**) سیستم ایجاد می‌کند:

1. کاهش مرتبه زمانی (Time Complexity Reduction):

اگر سند A دارای 200 کلمه و سند B دارای 150 کلمه باشد، بیشترین اشتراک ممکن بین این 2 سند، 150 کلمه است. بنابراین سیستم به جای **100,000 عمل ضرب**، حداکثر به **150 عمل ضرب** نیاز دارد. این یعنی **کاهش 99.8 درصدی بار محاسباتی**.

2. استفاده از ساختارهای داده‌ای بهینه (مانند Hash Map یا Posting Lists):

برای بهره‌برداری از این مزیت، ما بردارهای اسناد را به شکل آرایه‌های 100 هزار خانه‌ای (**Dense Arrays**) ذخیره نمی‌کنیم (چون حافظه رم به شدت هدر می‌رود). در عوض، آن‌ها را به عنوان ساختارهای لیستی ذخیره می‌کنیم که تنها کلمات موجود و وزن آن‌ها را نگه می‌دارند.

• نمایش سنتی (حافظه بالا): $[0, 0, 0, 1.5, 0, 0, \dots, 2.1, 0, 0]$

• نمایش پراکنده (حافظه کم): $\{ \text{کلمه_4: 1.5}, \text{کلمه_890: 2.1} \}$

در این ساختار، پیدا کردن اشتراک 2 سند (**Intersection**) از طریق الگوریتم‌های پرسرعت مانند **Hashing** یا ادغام لیست‌های مرتب‌شونده (**Merge Sort-like traversal**) انجام می‌شود که به شدت بهینه است.

نتیجه‌گیری: پدیده «پراکندگی» (**Sparsity**) دقیقاً همان چیزی است که به الگوریتم‌های کلاسیک مانند **kNN** و **Rocchio** اجازه می‌دهد در ابعاد وسیع متون زنده بمانند. اگر قرار بود اسناد متنی ما مانند تصاویر یا سیگنال‌های صوتی باشند (که در آن‌ها تقریباً هیچ پیکسلی یا نمونه‌ای دقیقاً 0 نیست)، اجرای این الگوریتم‌ها روی مجموعه‌های متنی بزرگ به دلیل کمبود حافظه و زمان پردازش، عملاً غیرممکن می‌شد.

پاسخنامه مسئله 14.27: تقریب همسایگی برای کلان‌داده‌ها

(Neighborhood Approximation for Big Data)

تحلیل اولیه و هدف مسئله

در مسئله 14.21 دیدیم که استفاده از «نمایه وارونه» (**Inverted Index**) می‌تواند با فیلتر کردن اسناد نامرتبط، سرعت الگوریتم **kNN** را به شدت افزایش دهد. اما در مقیاس‌های غول‌آسا (مانند 100 ها میلیون سند در موتورهای جستجوی وب یا سیستم‌های توصیه‌گر یوتیوب)، حتی تعداد اسنادی که حداقل 1 کلمه مشترک با کوئری دارند (**Candidate Set**)، ممکن است به 10 ها میلیون برسد. محاسبه شباهت کسینوسی برای این تعداد سند در زمان واقعی (**Real-time**) غیرممکن است.

هدف این مسئله، معرفی تکنیک **LSH (Locality Sensitive Hashing)** است که معماری سیستم را از «جستجوی دقیق» (**Exact kNN**) به «جستجوی تقریبی» (**Approximate Nearest Neighbors - ANN**) تغییر می‌دهد تا سرعت محاسبات را با فدا کردن درصد بسیار ناچیزی از دقت، 1000 ها بار افزایش دهد.

بخش اول: پارادایم شیف‌ت در مفهوم هشینگ (Hashing)

برای درک **LSH**، ابتدا باید تفاوت آن را با هشینگ سنتی (مانند **MD5** یا **SHA-256**) بدانیم:

هشینگ سنتی (کریپتوگرافیک): هدف اصلی این توابع، **فرار از تصادم (Collision Avoidance)** است. اگر حتی 1 حرف از 1 سند 1000 صفحه‌ای تغییر کند، خروجی هش کاملاً تغییر می‌کند. هدف این است که هیچ 2 سندی در 1 سطل (**Bucket**) نیفتند.

هشینگ حساس به مکان (LSH): این توابع دقیقاً برعکس عمل می‌کنند و هدفشان **ایجاد تصادم عمدی (Collision Maximization)** است. تابع هش **LSH** طوری طراحی می‌شود که اگر 2 سند در فضای برداری به هم شبیه و نزدیک باشند، با احتمال بسیار بالایی هش یکسانی تولید کرده و دقیقاً در 1 سطل مشترک بیفتند.

بخش دوم: تغییر معماری سیستم با استفاده از LSH

در سیستم‌های کلان‌داده، معماری جستجو با پیاده‌سازی **LSH** به 1 پایپ‌لاین (**Pipeline**) 4 مرحله‌ای تبدیل می‌شود:

1. کاهش ابعاد و تولید امضا (Signature Generation)

ابتدا بردارهای متنی با ابعاد بسیار بالا (مثلاً 100 هزار کلمه) را با استفاده از تکنیک‌هایی مانند **MinHashing** (برای شباهت جاکارد) یا **Random Projection** (برای شباهت کسینوسی) به بردارهای بسیار کوچکی به نام «امضا» (**Signature**) تبدیل می‌کنیم. این امضاها معمولاً طول ثابتی (مثلاً 100 عدد صحیح) دارند و ویژگی‌های اصلی سند را حفظ می‌کنند.

2. تکنیک قطعه‌بندی (Banding Technique)

بردار امضای 100 تایی به b قطعه (**Band**) تقسیم می‌شود که هر قطعه شامل r ردیف است (مثلاً 20 قطعه 5 تایی). سپس هر یک از این قطعات به صورت جداگانه از 1 تابع هش عبور کرده و در سطل‌های دیتابیس (**Hash Buckets**) قرار می‌گیرند. اگر 2 سند حداقل در 1 از این 20 قطعه کاملاً مشابه باشند، در 1 سطل مشترک تصادم می‌کنند و به عنوان «جفت‌های کاندید» (**Candidate Pairs**) شناخته می‌شوند.

3. فاز جستجو و استخراج کاندیدها (**Candidate Retrieval**)

هنگامی که کوئری جدیدی وارد سیستم می‌شود، سیستم تمام 100 هزار کلمه آن را پردازش نمی‌کند. بلکه مستقیماً امضای کوئری را می‌سازد، آن را قطعه‌قطعه می‌کند و بررسی می‌کند که این قطعات در کدام سطل‌های هشینگ قرار می‌گیرند. سیستم تمام اسنادی که در آن سطل‌های خاص قرار دارند را به عنوان «کاندیدهای احتمالی» استخراج می‌کند.

4. محاسبه دقیق در مقیاس خرد (**Exact Distance Computation**)

اکنون به جای مقایسه کوئری با 100 میلیون سند، سیستم کاندیدها را به 1 مجموعه بسیار کوچک (مثلاً 1000 سند) تقلیل داده است. در این مرحله نهایی، الگوریتم **kNN** استاندارد وارد عمل شده و شباهت کسینوسی دقیق را تنها برای این 1000 سند محاسبه می‌کند تا برترین نتایج را به کاربر نمایش دهد.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: معماران سیستم‌های کلان‌داده همیشه با 1 بده‌بستان (**Trade-off**) روبرو هستند. الگوریتم **LSH** تضمین نمی‌کند که دقیقاً نزدیک‌ترین همسایه را پیدا کند (ممکن است نزدیک‌ترین سند به دلیل بدشانسی در هیچ سطلی با کوئری تصادم نکند). اما در عوض، پیچیدگی زمانی را از 1 عملیات خطی سنگین به 1 عملیات جستجوی درهم‌سازی (**Hash Lookup**) تقلیل می‌دهد که پیچیدگی آن تقریباً $O(1)$ است. این تقریب هوشمندانه، راز سرعت میلی‌ثانیه‌ای موتورهای جستجو در میان میلیاردها صفحه وب است.

پاسخنامه مسئله 14.28: تزریق نویز و مقاومت مدل (**Noise Injection and Model Robustness**)

تحلیل اولیه و هدف مسئله

در دنیای واقعی و به‌ویژه در امنیت یادگیری ماشین، سیستم‌ها همواره در معرض داده‌های کثیف یا حملات عمدی مانند «مسموم‌سازی داده‌ها» (**Data Poisoning**) قرار دارند. هدف این مسئله، بررسی میزان تاب‌آوری (**Robustness**) 2 الگوریتم پایه یعنی **Rocchio** و **kNN** در برابر نویزهای مخرب است. با درک واکنش این 2 مدل، دانشجویان متوجه خواهند شد که چرا انتخاب معماری تنها به «دقت در شرایط آزمایشگاهی» بستگی ندارد، بلکه «مقاومت در شرایط بحرانی» نیز 1 معیار کلیدی است.

بخش اول: واکنش الگوریتم **Rocchio** (مقاومت بالا، تأثیر ناچیز)

الگوریتم **Rocchio** 1 مدل **سراسری (Global)** مبتنی بر مرکز ثقل (**Centroid**) است. برای درک واکنش آن، باید به ذات ریاضی محاسبه میانگین توجه کنیم.

1. مکانیزم جذب نویز:

مرکز ثقل کلاس «ورزشی» از طریق میانگین‌گیری برداری تمام اسناد این کلاس به دست می‌آید. وقتی 5% از اسناد، کلمات علمی دریافت می‌کنند، این اسناد مسموم در فضای برداری به سمت منطقه کلاس «علمی» کشیده می‌شوند.

اما هنگام محاسبه میانگین، وزن 95% اسناد سالم (که کلماتی مثل "توپ"، "گل" و "ورزشگاه" دارند) در برابر 5% اسناد مسموم، بسیار سنگین‌تر است.

2. تغییر هندسی مرز:

لنگر عظیم 95 درصدی باعث می‌شود که مرکز ثقل کلاس ورزشی، تنها 1 **جابجایی میکروسکوپی** به سمت منطقه علمی داشته باشد. به تبع آن، مرز تصمیم‌گیری خطی (عمود منصف) نیز تنها کسری از درجه منحرف می‌شود.

نتیجه برای Rocchio: این الگوریتم در برابر مسموم‌سازی 5 درصدی، مانند 1 ضربه‌گیر عمل می‌کند. روکیو ذاتاً 1 فیلتر پایین‌گذر است که نویزهای اقلیت را در میان اکثریت سالم «حل» و بی‌اثر می‌کند.

بخش دوم: واکنش الگوریتم kNN (شکنندگی شدید، تخریب محلی)

الگوریتم **kNN** (به‌ویژه با **k** کوچک مانند **k=1** یا **k=3**) 1 مدل کاملاً **محلی (Local)** است. این مدل هیچ مرکز ثقل یا درک کلانی از «اکثریت 95 درصدی» ندارد.

1. مکانیزم تشکیل جزیره‌های مخرب:

آن 5% اسناد مسموم، در فضای برداری پرواز کرده و دقیقاً در قلب منطقه کلاس «علمی» (یا مرز آن) فرود می‌آیند. چون **kNN** هیچ‌یک از این اسناد را با هم ادغام نمی‌کند، این 5% به شکل نقاطی مستقل در فضا باقی می‌مانند. آن‌ها به «سلول‌های انتحاری» یا **جزیره‌های کوچک ورزشی** در میان دریای کلاس علمی تبدیل می‌شوند.

2. تغییر هندسی مرز (فاجعه طبقه‌بندی):

زمانی که 1 مقاله علمی جدید و کاملاً سالم وارد سیستم می‌شود، اگر بردار آن از نظر هندسی در نزدیکی 1ی از این «جزیره‌های مسموم» بیفتد، سیستم فقط به همان همسایه نزدیک نگاه می‌کند. با وجود اینکه هزاران مقاله علمی دیگر کمی دورتر وجود دارند، مدل فریب می‌خورد و مقاله علمی سالم را به عنوان «ورزشی» طبقه‌بندی می‌کند.

نتیجه برای kNN: این الگوریتم به شدت در برابر **Data Poisoning** آسیب‌پذیر است (**Robustness** بسیار پایین). مرزهای تصمیم‌گیری آن به شدت دندانه‌دار می‌شوند تا دورِ تکتک این نویزها بپیچند، که منجر به پدیده **Overfitting** محلی و خطاهای فاحش در فاز تست می‌شود.

جمع‌بندی و نکات آموزشی

نتیجه‌گیری: این مسئله 1 قانون طلایی در مهندسی یادگیری ماشین را به ما می‌آموزد: **«انعطاف‌پذیری بالا، همواره بهای امنیتی دارد.»** الگوریتم **kNN** به دلیل بایاس پایین، انعطافِ خارق‌العاده‌ای برای یادگیری مرزهای پیچیده دارد، اما همین ویژگی آن را در برابر نویزها و حملات مسموم‌سازی به شدت شکننده می‌کند (واریانس بالا). در پروژه‌های صنعتی که کیفیت داده‌ها قابل تضمین نیست (مانند نظرات کاربران یا ایمیل‌های اسپم)، استفاده از مدل‌های دارای بایاسِ بالاتر (مانند **Naive Bayes**، **Rocchio** یا **SVM** با **Margin** وسیع) برای حفظ امنیت و پایداری سیستم کاملاً الزامی است.

پاسخنامه مسئله 14.29: مسیریابی نظرات در تجارت الکترونیک

(Review Routing in E-commerce)

جمله «کالا عالی بود اما خیلی دیر رسید» همزمان دو پیام دارد: رضایت از کالا و شکایت از ارسال. سیستم فعلی **Rocchio** با معماری **One-of** فرض می‌کند هر سند دقیقاً یک کلاس دارد، پس این نظر را ناچار به یک مرکز ثقل می‌چسباند یا، وقتی هیچ کلاسی برتری روشنی ندارد، آن را رد می‌کند. هدف مسئله این است که نشان دهیم مسیریابی نظرات ذاتاً یک مسئله **Any-of** (چندبرچسبی) است و منطق آستانه‌گذاری آن را روی کاغذ طراحی کنیم.

بخش اول: چرا معماری One-of شکست می‌خورد؟

در **Rocchio** برای هر کلاس c یک مرکز ثقل ساخته می‌شود و در حالت **One-of** سند به نزدیک‌ترین مرکز ثقل می‌رود:

$$\vec{\mu}_c = \frac{1}{|D_c|} \sum_{d \in D_c} \vec{v}(d)$$

$$\hat{c} = \arg \max_c \cos(\vec{d}, \vec{\mu}_c)$$

برای یک مدل کاغذی، واژگان را به شش واژه محدود می‌کنیم: **[عالی، کیفیت، خراب، دیر، ارسال، کالا]** و سه مرکز ثقل (بردارهای وزن نمونه) در نظر می‌گیریم:

$$\begin{aligned}\vec{\mu}_{\text{Logistics}} &= (0, 0, 0, 0.7, 0.5, 0.3) \\ \vec{\mu}_{\text{Sales}} &= (0, 0.6, 0.6, 0, 0, 0.4) \\ \vec{\mu}_{\text{Praise}} &= (0.8, 0.3, 0, 0, 0, 0.5)\end{aligned}$$

نظر کاربر شامل واژه‌های «کالا»، «عالی» و «دیر» است: $\vec{d}_1 = (1, 0, 0, 1, 0, 1)$. شباهت‌ها:

$$\cos(\vec{d}_1, \vec{\mu}_{\text{Praise}}) = 0.758, \quad \cos(\vec{d}_1, \vec{\mu}_{\text{Logistics}}) = 0.634, \quad \cos(\vec{d}_1, \vec{\mu}_{\text{Sales}}) = 0.240$$

نتیجه برای One-of: برنده **Praise** است و نظر به هیچ دپارتمانی نمی‌رسد؛ شکایت تأخیر کاملاً گم می‌شود. اگر سیستم برای جلوگیری از خطا قانون «رد در صورت حاشیه کم» داشته باشد، اختلاف دو کلاس اول فقط $0.758 - 0.634 = 0.124$ است و نظر مسدود می‌شود؛ همان خطایی که در صورت مسئله دیده‌ایم. ریشه مشکل الگوریتم نیست، **فرض انحصاری بودن کلاس‌ها** است.

بخش دوم: بازطراحی با معماری Any-of

در **Any-of** به جای یک تصمیم مشترک، برای هر دپارتمان یک **طبقه‌بند دودویی مستقل** داریم («آیا این نظر به لجستیک مربوط است؟» و جداگانه «آیا به فروش مربوط است؟»). هر طبقه‌بند آستانه مخصوص خودش را دارد و نظر به همه دپارتمان‌هایی فرستاده می‌شود که از آستانه‌شان عبور کند:

$$\text{Route}(d) = \{ c \in \{\text{Logistics}, \text{Sales}\} : \cos(\vec{d}, \vec{\mu}_c) \geq \theta_c \}$$

کلاس **Praise** دیگر رقیب دپارتمان‌ها نیست؛ بالا بودن امتیاز رضایت، مانع ارسال شکایت تأخیر نمی‌شود. با $\theta_{\text{Logistics}} = \theta_{\text{Sales}} = 0.5$ دو نظر نمونه را روی کاغذ مسیریابی می‌کنیم ($d_2 =$ «کیفیت کالا خراب بود و دیر هم رسید»):

نظر	Logistics با cos	Sales با cos	Praise با cos	لجستیک (θ = 0.5)	فروش (θ = 0.5)
d1: «عالی بود اما دیر رسید»	0.634	0.246	0.758	✓ ارسال	X
d2: «کیفیت خراب بود و دیر رسید»	0.549	0.853	0.404	✓ ارسال	✓ ارسال

نظر اول فقط به لجستیک می‌رود (تعریف از کیفیت، شکایت فروش نیست) و نظر دوم هم‌زمان به هر دو دپارتمان؛ رفتاری که **One-of** اصلاً قادر به نمایش آن نیست.

بخش سوم: منطق تصمیم‌گیری و تنظیم آستانه‌ها

1. تنظیم آستانه هر کلاس به‌صورت مستقل:

آستانه‌ها حدسی انتخاب نمی‌شوند؛ روی یک مجموعه اعتبارسنجی برچسب‌خورده، برای هر دپارتمان مقداری انتخاب می‌شود که **F1** همان کلاس را بیشینه کند:

$$\theta_c^* = \arg \max_{\theta} F_1^{(c)}(\theta)$$

2. در نظر گرفتن هزینه خطا:

در پشتیبانی مشتری، گم شدن یک شکایت (**False Negative**) معمولاً گران‌تر از یک ارجاع اضافه (**False Positive**) است؛ پس می‌توان آستانه را کمی پایین‌تر از θ_c^* گذاشت تا **Recall** دپارتمان‌ها بالا برود.

3. نظرهایی که از هیچ آستانه‌ای عبور نمی‌کنند:

اگر هیچ دپارتمانی انتخاب نشد، نظر نباید مسدود یا حذف شود؛ به صف بررسی انسانی می‌رود. کد زیر همین منطق را خلاصه می‌کند:

```
review_router.py

def route_review(d, centroids, thresholds):
    # Any-of: one independent yes/no decision per department
    targets = []
    for dept in ("logistics", "sales"):
        score = cosine(d, centroids[dept])
        if score >= thresholds[dept]:
            targets.append(dept)
    if not targets:
        targets.append("human_review")    # nothing passed: never drop a complaint
    return targets
```

نکته طلایی: هر وقت یک سند ممکن است هم‌زمان به چند مقصد تعلق داشته باشد (نظرات، تیکت‌ها، برچسب‌گذاری خبر)، مسئله را به چند طبقه‌بند دودویی مستقل با آستانه‌های جداگانه بشکنید. **One-of** فقط وقتی درست است که کلاس‌ها واقعاً انحصاری باشند.

پاسخنامه مسئله 14.30: انتخاب پویای پارامتر k در زمان اجرا (Dynamic k Selection at Runtime)

تحلیل اولیه و طراحی الگوریتم (Adaptive-kNN)

در نسخه کلاسیک kNN ، ثابت بودن پارامتر k یک نقطه ضعف ساختاری است. اگر k روی عدد کوچکی تنظیم شود، سیستم در فضاهای خلوت فریب نویزها را می‌خورد؛ و اگر روی عدد بزرگی تنظیم شود، در فضاهای متراکم مرزهای ظریف بین کلاس‌ها را از بین می‌برد (Over-smoothing).

برای حل این چالش، ما الگوریتمی به نام **Adaptive-kNN** (**kNN تطبیقی**) طراحی می‌کنیم. منطق این الگوریتم بر پایه محاسبه «تراکم محلی» (**Local Density**) استوار است:

ما ابتدا شباهت m همسایه اول کوئری را بررسی می‌کنیم. اگر این همسایگان به شدت به کوئری شبیه بودند (تراکم بالا)، سیستم با اعتماد به نفس بالا و با یک k کوچک تصمیم‌گیری می‌کند تا نویزهای دورتر وارد چرخه نشوند. اما اگر فضا خلوت بود و شباهت‌ها پایین بود، سیستم دایره جستجو را منبسط می‌کند (افزایش k) تا با جمع‌آوری آرای بیشتر، از خطای تصادفی جلوگیری کند.

مدل‌سازی ریاضی: تبدیل تراکم به پارامتر k

ابتدا متغیرهای پایه را تعریف می‌کنیم:

- k_{min} : حداقل تعداد همسایه مجاز (مثلاً 3).
- k_{max} : حداکثر تعداد همسایه مجاز برای جلوگیری از افت سرعت (مثلاً 50).
- m : تعداد همسایگان اولیه‌ای که برای سنجش تراکم فضا استفاده می‌شوند (مثلاً $m = 3$).

گام 1: محاسبه تراکم محلی (ρ)

تراکم فضا پیرامون کوئری Q را به عنوان میانگین شباهت کسینوسی m همسایه اول تعریف می‌کنیم:

$$\rho = \frac{1}{m} \sum_{i=1}^m \text{Sim}(Q, D_i)$$

مقدار ρ همواره عددی بین 0 (فضای به شدت خلوت) و 1 (فضای به شدت متراکم و مشابه) خواهد بود.

گام 2: محاسبه k پویا ($k_{dynamic}$)

اکنون با استفاده از یک انتقال خطی معکوس، ρ را به بازه مجاز k نگاشت می‌کنیم:

$$k_{dynamic} = k_{min} + \lfloor (k_{max} - k_{min}) \cdot (1 - \rho) \rfloor$$

اگر فضا متراکم باشد ($\rho \approx 1$): عبارت $(1 - \rho)$ صفر می‌شود و $k_{dynamic}$ برابر با k_{min} خواهد شد.

اگر فضا خلوت باشد ($\rho \approx 0$): عبارت $(1 - \rho)$ یک می‌شود و $k_{dynamic}$ به حداکثر خود یعنی k_{max} منبسط می‌گردد.

شبه‌کد الگوریتم (Pseudo-code)

ساختار کد زیر با رویکردی مشابه زبان پایتون طراحی شده است تا منطق جریان داده در آن بهینه‌ترین حالت ممکن را داشته باشد:

```
adaptive_knn.txt

ALGORITHM Adaptive_kNN(Q, Index, k_min=3, k_max=50, m=3):

# Step 1: Retrieve maximum bounding neighbors to avoid multiple DB calls
# O(log N) or O(1) depending on Index structure (e.g., LSH / Inverted Index)
Candidates = Index.GetTopK(Q, k_max)

# Edge case fallback for extremely small datasets
IF length(Candidates) < k_min:
    RETURN MajorityVote(Candidates)

# Step 2: Sample the immediate local neighborhood to estimate density
SumSim = 0.0
FOR i = 1 TO m:
    SumSim = SumSim + Candidates[i].Similarity

rho = SumSim / m

# Step 3: Compute the dynamic k
# Inverse relationship: higher density -> smaller k
ScalingFactor = 1.0 - rho
k_dynamic = k_min + FLOOR((k_max - k_min) * ScalingFactor)

# Step 4: Extract the precise adaptive subset
AdaptiveNeighbors = Candidates[1 TO k_dynamic]

# Step 5: Return the classification result based on the dynamic subset
RETURN MajorityVote(AdaptiveNeighbors)
```

تحلیل پیچیدگی و بهینگی (Complexity Analysis)

شاید در نگاه اول به نظر برسد که پویا کردن **k** بار محاسباتی سنگینی دارد، اما از نظر پیچیدگی زمانی الگوریتم، هزینه این خلاقیت **تقریباً صفر** است:

ما در همان گام اول (همانند الگوریتم **kNN** استاندارد)، k_{max} همسایه را بازیابی کرده و آرایه کاندیدها را در حافظه (**RAM**) در اختیار داریم. محاسبه پارامتر ρ تنها نیازمند $O(m)$ عملیات جمع است (که عدد ثابت و کوچکی مانند 3 است) و تخصیص $k_{dynamic}$ نیز با یک عملیات حسابی با پیچیدگی $O(1)$ انجام می‌شود. در نتیجه، ما بدون هیچ افت سرعتی نسبت به الگوریتم پایه، یک مدل تصمیم‌گیری کاملاً هوشمند و سازگار با توزیع داده‌ها (**Data-Aware**) خلق کرده‌ایم.

پاسخنامه مسئله 14.31: تأثیر کاهش ابعاد بر خطی بودن

(Impact of Dimensionality Reduction on Linearity)

تحلیل اولیه و هدف مسئله

در مسائل پیشین (مانند مسئله 14.13 و قضیه کاور) دیدیم که فضاهای متنی با ابعاد بسیار بالا (مثلاً 100 هزار کلمه)، به صورت ذاتی دارای تفکیک‌پذیری خطی (**Linear Separability**) هستند. اما تکنیک‌های کاهش ابعاد مانند **SVD** در تحلیل پنهان معنایی (**LSA**)، این فضای غول‌آسا و خلوت (**Sparse**) را فشرده کرده و به 1 فضای کوچک، متراکم (**Dense**) و پیوسته (مثلاً 200 بعدی) تبدیل می‌کنند.

هدف این مسئله، بررسی این پارادایم شیفت است: وقتی فضا فشرده می‌شود، آیا قوانین هندسی حاکم بر مرزهای تصمیم‌گیری نیز تغییر می‌کنند؟

بخش اول: سقوط تفکیک‌پذیری خطی خام (Reverse Cover's Theorem)

کاهش ابعاد از 100 هزار به 200، صرفاً حذف کلمات نیست؛ بلکه استخراج «مفاهیم پنهان» (**Latent Concepts**) است. در این فرآیند، مترادف‌ها (مانند "ماشین" و "خودرو") که قبلاً روی محورهای کاملاً مجزا و متعامد قرار داشتند، اکنون روی 1 محور مشترک معنایی ترکیب می‌شوند.

تأثیر هندسی روی تفکیک‌پذیری:

با فشردن فضا، اسنادی که در فضای 100 هزار بعدی کیلومترها از هم فاصله داشتند و به راحتی با 1 ابرصفحه جدا می‌شدند، اکنون در فضای 200 بعدی به هم نزدیک شده و روی هم می‌افتند.

ما در واقع 99,800 «درجه آزادی» را از دست داده‌ایم. در نتیجه، مرزهای تصمیم‌گیری که قبلاً می‌توانستند از میان هزاران روزنه در فضای خلوت عبور کنند، اکنون با توده‌های متراکم و در هم تنیده اسناد مواجه می‌شوند. بنابراین، **تفکیک‌پذیری خطی خام داده‌ها به شدت کاهش می‌یابد** و کلاس‌ها ممکن است در مرزهای خود دچار هم‌پوشانی (**Overlap**) یا ساختارهای غیرمحدب (**Non-convex**) شوند.

بخش دوم: مرگ روکیو و تولد مجدد kNN

با توجه به تغییر ماهیت فضا از **Sparse** به **Dense**، استراتژی انتخاب الگوریتم دقیقاً 180 درجه برعکس می‌شود:

1. چرا Rocchio دیگر منطقی نیست؟

الگوریتم **Rocchio** (با بایاس بالا) بر این فرض استوار است که کلاس‌ها به شکل کره‌هایی مجزا هستند که با 1 خط راست جدا می‌شوند. در فضای تقلیل‌یافته 200 بعدی، کلاس‌ها اغلب به شکل خوشه‌های پیچیده، کشیده (مانند اشکال هلالی) یا دارای هم‌پوشانی درمی‌آیند. پافشاری **Rocchio** بر استفاده از 1 خط راست در این فضای متراکم، منجر به **کم‌برازش (Underfitting)** و خطای دسته‌بندی بالا می‌شود.

2. چرا kNN و مدل‌های غیرخطی پادشاهی می‌کنند؟

در مسئله 14.20 خواندیم که الگوریتم **kNN** در فضاهای ابعاد بالا به دلیل «نفرین ابعاد» و بی‌معنی شدن فواصل، شکست می‌خورد. اما اکنون با کاهش ابعاد به 200 (و حذف نویزها توسط **SVD**)، **نفرین ابعاد شکسته شده است!**

- فواصل اقلیدسی و کسینوسی دوباره کاملاً معنادار و دقیق شده‌اند.
- فضای حافظه و زمان پردازش به شدت کاهش یافته، بنابراین محاسبه **kNN** بسیار سریع شده است.
- قدرت **kNN** در ساخت مرزهای تصمیم‌گیری غیرخطی و محلی، دقیقاً همان چیزی است که برای جدا کردن کلاس‌های در هم تنیده در این فضای فشرده 200 بعدی به آن نیاز داریم.

نتیجه‌گیری: در پایپ‌لاین‌های مدرن ماشین لرنینگ، 1 قانون نانوشته وجود دارد: اگر قصد دارید داده‌های متنی خام را مستقیماً پردازش کنید، از طبقه‌بندهای خطی پرسرعت استفاده کنید (**High-D = Linear**). اما به محض اینکه داده‌ها را از فیلترهای کاهش ابعاد (مثل **SVD**، **PCA** یا **Embeddings** مانند **Word2Vec**) عبور دادید، باید بلافاصله به سمت طبقه‌بندهای قدرتمندتر و غیرخطی (مانند شبکه‌های عصبی عمیق، **SVM** با کرنل **RBF** یا **kNN**) سوئیچ کنید (**Low-D = Non-Linear**).

پاسخنامه مسئله 14.32: طراحی جامع سیستم تشخیص بات (Bot Detection Architecture)

تحلیل اولیه و هدف پروژه

مدیریت پلتفرمی با 50 میلیون کاربر، نیازمند معماری است که بتواند با حجم عظیمی از داده‌ها (**Scale**) در زمان واقعی (**Real-time**) مقابله کند. اگر بخواهیم تمام متون کاربران را با 1 الگوریتم دقیق مانند **kNN** ارزیابی کنیم، سیستم به دلیل بار پردازشی کلاپس می‌کند.

از طرفی، توزیع داده‌ها نامتقارن است: کاربران واقعی متونی به شدت خلاقانه، متنوع و پراکنده می‌نویسند (**High Variance**)، در حالی که بات‌ها معمولاً کمپین‌های هماهنگ با متون تکراری و متمرکز اجرا می‌کنند (**Low Variance / High Density**).

هدف این معماری، استفاده از استراتژی «**فیلتر 2 مرحله‌ای**» (**Two-Stage Pipeline**) است. ما از 1 الگوریتم با بایاس بالا برای دفع سریع توده‌های مشخص، و از 1 الگوریتم با بایاس پایین برای بررسی جراحی‌گونه موارد مشکوک استفاده می‌کنیم.

طراحی نقشه جریان داده (Data Flow Architecture)

در این معماری، ترافیک ورودی از 2 فیلتر متوالی عبور می‌کند:

1. فاز اول: دروازه‌بان خطی (Fast Screening) – الگوریتم Rocchio

سیستم به صورت آفلاین، مرکز ثقل متون تمام بات‌های شناخته‌شده ($\vec{\mu}_{bot}$) را محاسبه کرده است. وقتی متن جدیدی وارد می‌شود، سیستم تنها 1 ضرب داخلی برای محاسبه فاصله متن تا $\vec{\mu}_{bot}$ انجام می‌دهد.

- اگر فاصله بسیار کم بود (قطعاً بات است) $\Leftarrow$ مسدودسازی فوری.
- اگر فاصله بسیار زیاد بود (قطعاً انسان است) $\Leftarrow$ عبور مجاز.
- اگر فاصله در منطقه خاکستری (**Grey Area**) بود $\Leftarrow$ ارسال به فاز دوم.

2. فاز دوم: کارآگاه محلی (Deep Inspection) – Inverted Index + kNN

تنها درصد کوچکی از متون (موارد مشکوک) به این فاز می‌رسند. در اینجا، متن کاربر به عنوان 1 کوئری در «نمایه وارونه» جستجو می‌شود تا اسنادی که دقیقاً کلمات مشابهی دارند پیدا شوند. سپس **kNN** روی این زیرمجموعه کوچک اجرا شده و با بررسی دقیق‌ترین همسایه‌ها، رأی نهایی را صادر می‌کند.

توجیه ریاضی بر اساس مبادله بایاس و واریانس

معماری فوق، تجسم عملی مدیریت **Bias-Variance Tradeoff** است:

1. چرا Rocchio در فاز اول؟ (استفاده از بایاس بالا)

چون بات‌ها متون تکراری تولید می‌کنند، بردارهای آن‌ها در فضای متنی 1 **تراکم کروی** (**Dense Cluster**) می‌سازند. الگوریتم **Rocchio** با بایاس بالای خود، این فرض را دارد که کلاس‌ها ساختار کروی دارند. بنابراین مرکز ثقلِ بات‌ها، نماینده‌ای به شدت دقیق از رفتار آن‌هاست.

مزیت اصلی در فاز اول، **پیچیدگی زمانی** $O(1)$ است. ما با تحمیل بایاس بالا، سرعت را به حداکثر می‌رسانیم و 90% ترافیک واضح (قطعات متراکم بات‌ها یا پراکندگی‌های دوردست انسان‌ها) را در کسری از میلی‌ثانیه تعیین تکلیف می‌کنیم.

2. چرا kNN در فاز دوم؟ (استفاده از واریانس بالا)

منطقه خاکستری جایی است که بات‌های هوشمندتر (که سعی می‌کنند کلمات انسانی را با متون اسپم ترکیب کنند) و انسان‌های تنبل (که متن‌های کلیشه‌ای می‌نویسند) با هم درگیر می‌شوند. در این مرزهای پیچیده و در هم تنیده، فرض خطی بودن

Rocchio شکست می‌خورد (باعث **Underfitting** می‌شود).

اینجا ما به مدلی با بایاس پایین و **واریانس بالا** نیاز داریم. **kNN** به جای نگاه به مرکز ثقل، روی «ریزساختارها» زوم می‌کند.

3. نقش حیاتی نمایه وارونه (Inverted Index):

اجرای **kNN** حتی برای همان 10% ترافیک مشکوک، اگر قرار باشد با 50 میلیون رکورد مقایسه شود، فاجعه‌بار است. اما انسان‌ها (**High Variance**) از کلمات به شدت متنوعی استفاده می‌کنند. نمایه وارونه با استفاده از ساختار پراکنده (**Sparse**) واژگان، اسناد آموزشی نامربوط را در $O(1)$ هرس می‌کند. فرمول جستجو به شکل زیر تقلیل می‌یابد:

$$\text{Candidates} = \bigcup_{t \in \text{Query}} \text{PostingList}(t)$$

بنابراین **kNN** تنها روی چند 100 همسایه بالقوه اجرا می‌شود.

نتیجه‌گیری: طراحی سیستم‌های مقیاس‌پذیر، هنرِ چیدمان الگوریتم‌هاست. هیچ الگوریتمی به تنهایی کامل نیست؛ **Rocchio** سریع است اما در مرزهای پیچیده کور می‌شود، **kNN** دقیق است اما به شدت کند عمل می‌کند. با ترکیب استراتژیک این 2 در 1 معماری آبشاری (**Cascade Architecture**) و استفاده از ساختارهای داده‌ای مانند **Inverted Index**، ما سیستمی خلق کردیم که همزمان سرعت پردازش‌های خطی و دقت مرزهای غیرخطی را به ارث می‌برد.